Contents

0.1 Wolfgang Ertel

Introduction
to Artificial
Intelligence

Second Edition

Contents

0.1.1 What Is Artificial Intelligence?

The term artificial intelligence stirs emotions. For one thing there is our fascination with intelligence, which seemingly imparts to us humans a special place among life forms. Questions arise such as ”What is intelligence?”, ”How can one measure intelligence?” or ”How does the brain work?”. All these questions are meaningful when trying to understand artificial intelligence. However, the central question for the engineer, especially for the computer scientist, is the question of the intelligent machine that behaves like a person, showing intelligent behavior.

The attribute artificial might awaken much different associations. It brings up fears of intelligent cyborgs. It recalls images from science fiction novels. It raises the question of whether our highest good, the soul, is something we should try to understand, model, or even reconstruct.

With such different offhand interpretations, it becomes difficult to define the term artificial intelligence or AI simply and robustly. Nevertheless I would like to try, using examples and historical definitions, to characterize the field of AI. In 1955, John McCarthy, one of the pioneers of AI, was the first to define the term artificial intelligence, roughly as follows:

The goal of AI is to develop machines that behave as though they were intelligent.

To test this definition, the reader might imagine the following scenario. Fifteen or so small robotic vehicles are moving on an enclosed four by four meter square surface. One can observe various behavior patterns. Some vehicles form small groups with relatively little movement. Others move peacefully through the space and gracefully avoid any collision. Still others appear to follow a leader. Aggressive behaviors are also observable. Is what we are seeing intelligent behavior?

According to McCarthy's definition the aforementioned robots can be described as intelligent. The psychologist Valentin Braitenberg has shown that this seemingly

complex behavior can be produced by very simple electrical circuits [[**?**]]. So-called Braitenberg vehicles have two wheels, each of which is driven by an independent electric motor. The speed of each motor is influenced by a light sensor on the front of the vehicle as shown in Fig. 1.1. The more light that hits

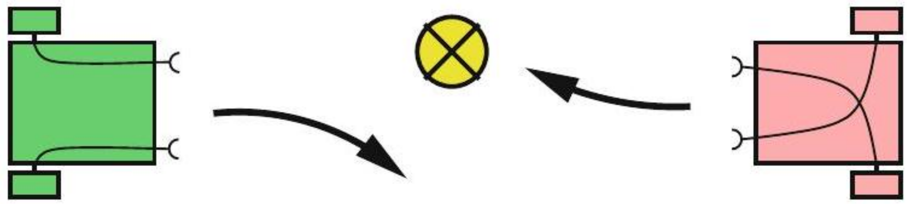


Figure 1: Two very simple Braitenberg vehicles and their reactions to a light source

the sensor, the faster the motor runs. Vehicle 1 in the left part of the figure, according to its configuration, moves away from a point light source. Vehicle 2 on the other hand moves toward the light source. Further small modifications can create other behavior patterns, such that with these very simple vehicles we can realize the impressive behavior described above.

Clearly the above definition is insufficient because AI has the goal of solving difficult practical problems which are surely too demanding for the Braitenberg vehicle. In the Encyclopedia Britannica [[**?**]] one finds a Definition that goes like:

> AI is the ability of digital computers or computer controlled robots to solve problems that are normally associated with the higher intellectual processing capabilities of humans ...

But this definition also has weaknesses. It would admit for example that a computer with large memory that can save a long text and retrieve it on demand displays intelligent capabilities, for memorization of long texts can certainly be considered a higher intellectual processing capability of humans, as can for example the quick multiplication of two 20-digit numbers. According to this definition, then, every computer is an AI system. This dilemma is solved elegantly by the following definition by Elaine Rich [[**?**]]:

> Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.

Rich, tersely and concisely, characterizes what AI researchers have been doing for the last 50 years. Even in the year 2050, this definition will be up to date.

Tasks such as the execution of many computations in a short amount of time are the strong points of digital computers. In this regard they outperform humans by many multiples. In many other areas, however, humans are far superior to machines. For instance, a person entering an unfamiliar room will recognize the surroundings within fractions of a second and, if necessary, just as swiftly make decisions and plan actions. To date, this task is too demanding for autonomous robots.[1] According to Rich's definition, this is therefore a task

[1] An autonomous robot works independently, without manual support, in particular without remote control.

for AI. In fact, research on autonomous robots is an important, current theme in AI. Construction of chess computers, on the other hand, has lost relevance because they already play at or above the level of grandmasters.

It would be dangerous, however, to conclude from Rich's definition that AI is only concerned with the pragmatic implementation of intelligent processes. Intelligent systems, in the sense of Rich's definition, cannot be built without a deep understanding of human reasoning and intelligent action in general, because of which neuroscience (see Sect. 1.1.1) is of great importance to AI. This also shows that the other cited definitions reflect important aspects of AI.

A particular strength of human intelligence is adaptivity. We are capable of adjusting to various environmental conditions and change our behavior accordingly through learning. Precisely because our learning ability is so vastly superior to that of computers, machine learning is, according to Rich's definition, a central subfield of AI.

0.1.2 Brain Science and Problem Solving

Through research of intelligent systems we can try to understand how the human brain works and then model or simulate it on the computer. Many ideas and principles in the field of neural networks (see Chap. 9) stem from brain science with the related field of neuroscience.

A very different approach results from taking a goal-oriented line of action, starting from a problem and trying to find the most optimal solution. How humans solve the problem is treated as unimportant here. The method, in this approach, is secondary. First and foremost is the optimal intelligent solution to the problem. Rather than employing a fixed method (such as, for example, predicate logic) AI has as its constant goal the creation of intelligent agents for as many different tasks as possible. Because the tasks may be very different, it is unsurprising that the methods currently employed in AI are often also quite different. Similar to medicine, which encompasses many different, often life-saving diagnostic and therapy procedures, AI also offers a broad palette of effective solutions for widely varying applications. For mental inspiration, consider Fig. 1.2 on page 4. Just as in medicine, there is no universal method for all application areas of AI, rather a great number of possible solutions for the great number of various everyday problems, big and small.

Cognitive science is devoted to research into human thinking at a somewhat higher level. Similarly to brain science, this field furnishes practical AI with many important ideas. On the other hand, algorithms and implementations lead to further important conclusions about how human reasoning functions. Thus these three fields benefit from a fruitful interdisciplinary exchange. The subject of this book, however, is primarily problem-oriented AI as a subdiscipline of computer science.

There are many interesting philosophical questions surrounding intelligence and artificial intelligence. We humans have consciousness; that is, we can think about

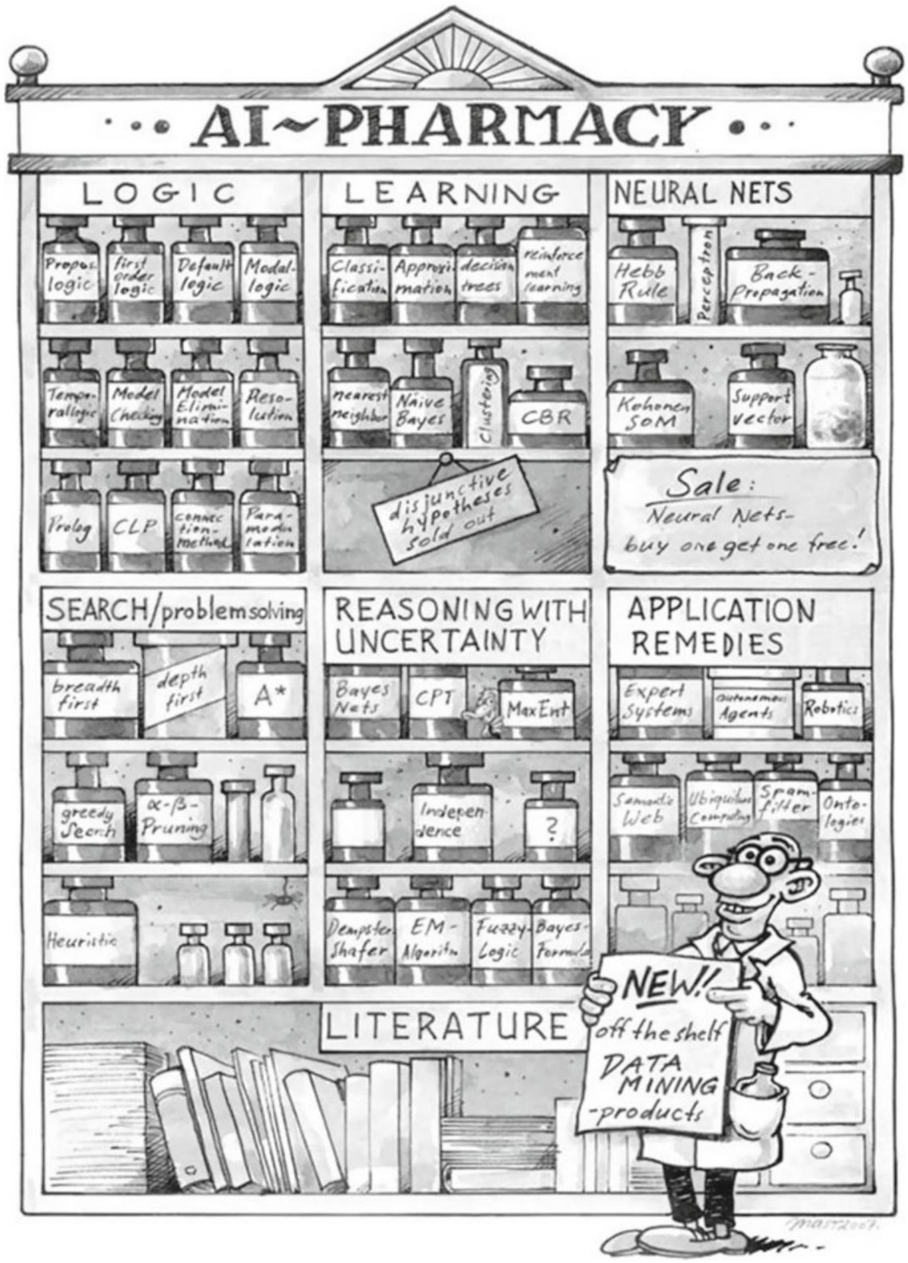


Figure 2: A small sample of the solutions offered by AI

ourselves and even ponder that we are able to think about ourselves. How does consciousness come to be? Many philosophers and neurologists now believe that the mind and consciousness are linked with matter, that is, with the brain. The
question of whether machines could one day have a mind or consciousness could at some point in the future become relevant. The mind-body problem in particular concerns whether or not the mind is bound to the body. We will not discuss these questions here. The interested reader may consult [[**?**], [**?**]] and is invited, in the course of AI technology studies, to form a personal opinion about these questions.

0.1.3 The Turing Test and Chatterbots

Alan Turing made a name for himself as an early pioneer of AI with his definition of an intelligent machine, in which the machine in question must pass the following test. The test person Alice sits in a locked room with two computer terminals. One terminal is connected to a machine, the other with a non-malicious person Bob. Alice can type questions into both terminals. She is given the task of deciding, after five minutes, which terminal belongs to the machine. The machine passes the test if it can trick Alice at least 30% of the time [[**?**]].

While the test is very interesting philosophically, for practical AI, which deals with problem solving, it is not a very relevant test. The reasons for this are similar to those mentioned above related to Braitenberg vehicles (see Exercise 1.3 on page 21).

The AI pioneer and social critic Joseph Weizenbaum developed a program named Eliza, which is meant to answer a test subject's questions like a human psychologist [[**?**]]. He was in fact able to demonstrate success in many cases. Supposedly his secretary often had long discussions with the program. Today in the internet there are many so-called chatterbots, some of whose initial responses are quite impressive. After a certain amount of time, however, their artificial nature becomes apparent. Some of these programs are actually capable of learning, while others possess extraordinary knowledge of various subjects, for example geography or software development. There are already commercial applications for chatterbots in online customer support and there may be others in the field of e-learning. It is conceivable that the learner and the e-learning system could communicate through a chatterbot. The reader may wish to compare several chatterbots and evaluate their intelligence in Exercise 1.1 on page 20.

0.1.4 The History of AI

AI draws upon many past scientific achievements which are not mentioned here, for AI as a science in its own right has only existed since the middle of the Twentieth Century. Table 1.1 on page 6, with the most important AI milestones, and a graphical representation of the main movements of AI in Fig. 1.3 on page 8 complement the following text.

Table 1.1 Milestones in the development of AI from Gödel to today

1931	The Austrian Kurt Gödel shows that in first-order predicate logic all true statements are deriva
1937	Alan Turing points out the limits of intelligent machines with the halting problem [[**?**]].
1943	McCulloch and Pitts model neural networks and make the connection to propositional logic.
1950	Alan Turing defines machine intelligence with the Turing test and writes about learning machi
1951	Marvin Minsky develops a neural network machine. With 3000 vacuum tubes he simulates 40
1955	Arthur Samuel (IBM) builds a learning checkers program that plays better than its developer [
1956	McCarthy organizes a conference in Dartmouth College. Here the name Artificial Intelligence Newell and Simon of Carnegie Mellon University (CMU) present the Logic Theorist, the first s
1958	McCarthy invents at MIT (Massachusetts Institute of Technology) the high-level language LIS
1959	Gelernter (IBM) builds the Geometry Theorem Prover.
1961	The General Problem Solver (GPS) by Newell and Simon imitates human thought [[**?**]].
1963	McCarthy founds the AI Lab at Stanford University.
1965	Robinson invents the resolution calculus for predicate logic [[**?**]] (Sect. 3.5).
1966	Weizenbaum's program Eliza carries out dialog with people in natural language [Wei66] (Sect
1969	Minsky and Papert show in their book Perceptrons that the perceptron, a very simple neural n
1972	French scientist Alain Colmerauer invents the logic programming language PROLOG (Chap. 5 British physician de Dombal develops an expert system for diagnosis of acute abdominal pain
1976	Shortliffe and Buchanan develop MYCIN, an expert system for diagnosis of infectious diseases
1981	Japan begins, at great expense, the "Fifth Generation Project" with the goal of building a powe
1982	R1, the expert system for configuring computers, saves Digital Equipment Corporation 40 mill
1986	Renaissance of neural networks through, among others, Rumelhart, Hinton and Sejnowski [[**?**
1990	Pearl [[**?**]], Cheeseman [[**?**]], Whittaker, Spiegelhalter bring probability theory into AI with Ba

Table 1.1 (continued)

1992	Tesauros TD-gammon program demonstrates the advantages of reinforcement learning.
1993	Worldwide RoboCup initiative to build soccer-playing autonomous robots [[**?**]].
1995	From statistical learning theory, Vapnik develops support vector machines, which are very im
1997	IBM's chess computer Deep Blue defeats the chess world champion Gary Kasparov.
	First international RoboCup competition in Japan.
2003	The robots in RoboCup demonstrate impressively what AI and robotics are capable of achievin
2006	Service robotics becomes a major AI research area.
2009	First Google self-driving car drives on the California freeway.
2010	Autonomous robots begin to improve their behavior through learning.
2011	IBM's "Watson" beats two human champions on the television game show "Jeopardy!". Watso
2015	Daimler premiers the first autonomous truck on the Autobahn.
	Google self-driving cars have driven over one million miles and operate within cities.
	Deep learning (Sect. 11.9) enables very good image classification.
	Paintings in the style of the Old Masters can be automatically generated with deep learning. A
2016	The Go program AlphaGo by Google DeepMind [[**?**]] beats the European champion 5:0 in Jan

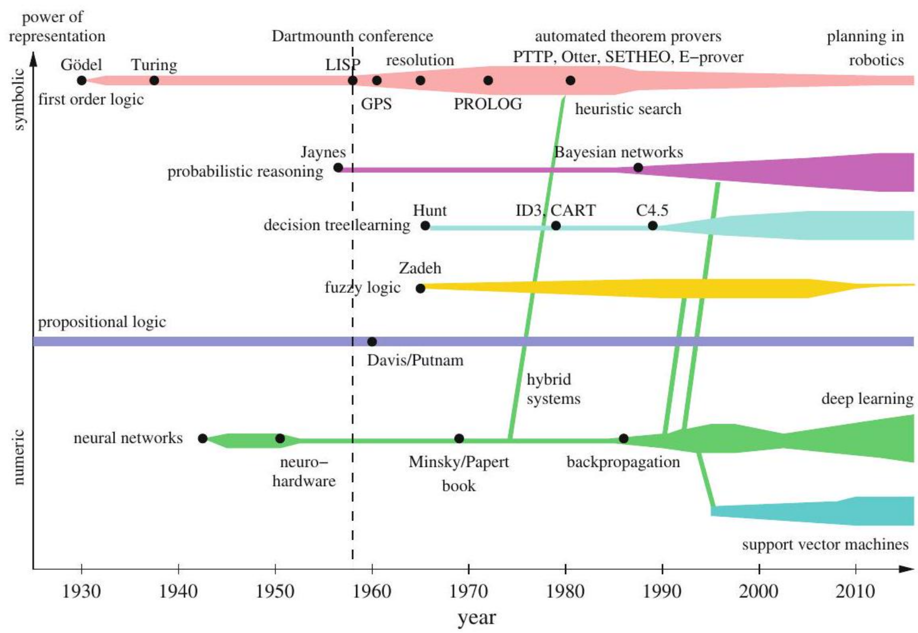


Figure 3: History of the various AI areas. The width of the bars indicates prevalence of the method's use.

0.1.5 The First Beginnings

In the 1930s Kurt Gödel, Alonso Church, and Alan Turing laid important foundations for logic and theoretical computer science. Of particular interest for AI are Gödel's theorems. The completeness theorem states that first-order predicate logic is complete. This means that every true statement that can be formulated in predicate logic is provable using the rules of a formal calculus. On this basis, automatic theorem provers could later be constructed as implementations of formal calculi. With the incompleteness theorem, Gödel showed that in higher-order logics there exist true statements that are unprovable. [2] With this he uncovered painful limits of formal systems.

Alan Turing's proof of the undecidability of the halting problem also falls into this time period. He showed that there is no program that can decide whether a given arbitrary program (and its respective input) will run in an infinite loop. With

this Turing also identified a limit for intelligent programs. It follows, for example, that there will never be a universal program verification system. [3]

In the 1940s, based on results from neuroscience, McCulloch, Pitts and Hebb designed the first mathematical models of neural networks. However, comput-

[12] Higher-order logics are extensions of predicate logic, in which not only variables, but also function symbols or predicates can appear as terms in a quantification. Indeed, Gödel only showed that any system that is based on predicate logic and can formulate Peano arithmetic is incomplete.

ers at that time lacked sufficient power to simulate simple brains.

0.1.6 Logic Solves (Almost) All Problems

AI as a practical science of thought mechanization could of course only begin once there were programmable computers. This was the case in the 1950s. Newell and Simon introduced Logic Theorist, the first automatic theorem prover, and thus also showed that with computers, which actually only work with numbers, one can also process symbols. At the same time McCarthy introduced, with the language LISP, a programming language specially created for the processing of symbolic structures. Both of these systems were introduced in 1956 at the historic Dartmouth Conference, which is considered the birthday of AI.

In the US, LISP developed into the most important tool for the implementation of symbol-processing AI systems. Thereafter the logical inference rule known as resolution developed into a complete calculus for predicate logic.

In the 1970s the logic programming language PROLOG was introduced as the European counterpart to LISP. PROLOG offers the advantage of allowing direct programming using Horn clauses, a subset of predicate logic. Like LISP, PROLOG has data types for convenient processing of lists.

Until well into the 1980s, a breakthrough spirit dominated AI, especially among many logicians. The reason for this was the string of impressive achievements in symbol processing. With the Fifth Generation Computer Systems project in Japan and the ESPRIT program in Europe, heavy investment went into the construction of intelligent computers.

For small problems, automatic provers and other symbol-processing systems sometimes worked very well. The combinatorial explosion of the search space, however, defined a very narrow window for these successes. This phase of AI was described in [[**?**]] as the "Look, Ma, no hands!" era.

Because the economic success of AI systems fell short of expectations, funding for logic-based AI research in the United States fell dramatically during the 1980s.

0.1.7 The New Connectionism

During this phase of disillusionment, computer scientists, physicists, and Cognitive scientists were able to show, using computers which were now sufficiently powerful, that mathematically modeled neural networks are capable of learning using training examples, to perform tasks which previously required costly programming. Because of the fault-tolerance of such systems and their ability to recognize patterns, considerable successes became possible, especially in pattern recognition. Facial recognition in photos and handwriting recognition are two example applications. The system Nettalk was able to learn speech from

[13] This statement applies to "total correctness", which implies a proof of correct execution as well as a proof of termination for every valid input.

example texts [SR86]. Under the name connectionism, a new subdiscipline of AI was born.

Connectionism boomed and the subsidies flowed. But soon even here feasibility limits became obvious. The neural networks could acquire impressive capabilities, but it was usually not possible to capture the learned concept in simple formulas or logical rules. Attempts to combine neural nets with logical rules or the knowledge of human experts met with great difficulties. Additionally, no satisfactory solution to the structuring and modularization of the networks was found.

0.1.8 Reasoning Under Uncertainty

AI as a practical, goal-driven science searched for a way out of this crisis. One wished to unite logic's ability to explicitly represent knowledge with neural networks' strength in handling uncertainty. Several alternatives were suggested.

The most promising, probabilistic reasoning, works with conditional probabilities for propositional calculus formulas. Since then many diagnostic and expert systems have been built for problems of everyday reasoning using Bayesian networks. The success of Bayesian networks stems from their intuitive comprehensibility, the clean semantics of conditional probability, and from the centuries-old, mathematically grounded probability theory. The weaknesses of logic, which can only work with two truth values, can be solved by fuzzy logic, which pragmatically introduces infinitely many values between zero and one. Though even today its theoretical foundation is not totally firm, it is being successfully utilized, especially in control engineering.

A much different path led to the successful synthesis of logic and neural networks under the name hybrid systems. For example, neural networks were employed to learn heuristics for reduction of the huge combinatorial search space in proof discovery [[**?**]].

Methods of decision tree learning from data also work with probabilities. Systems like CART, ID3 and C4.5 can quickly and automatically build very accurate decision trees which can represent propositional logic concepts and then be used as expert systems. Today they are a favorite among machine learning techniques (Sect. 8.4).

Since about 1990, data mining has developed as a subdiscipline of AI in the area of statistical data analysis for extraction of knowledge from large databases. Data mining brings no new techniques to AI, rather it introduces the requirement of using large databases to gain explicit knowledge. One application with great market potential is steering ad campaigns of big businesses based on analysis of many millions of purchases by their customers. Typically, machine learning techniques such as decision tree learning come into play here.

0.1.9 Distributed, Autonomous and Learning Agents

Distributed artificial intelligence, DAI, has been an active area research since about 1985. One of its goals is the use of parallel computers to increase the efficiency of problem solvers. It turned out, however, that because of the high computational complexity of most problems, the use of ”intelligent” systems is more beneficial than parallelization itself.

A very different conceptual approach results from the development of autonomous software agents and robots that are meant to cooperate like human teams. As with the aforementioned Braitenberg vehicles, there are many cases in which an individual agent is not capable of solving a problem, even with unlimited resources. Only the cooperation of many agents leads to the intelligent behavior or to the solution of a problem. An ant colony or a termite colony is capable of erecting buildings of very high architectural complexity, despite the fact that no single ant comprehends how the whole thing fits together. This is similar to the situation of provisioning bread for a large city like New York [RN10]. There is no central planning agency for bread, rather there are hundreds of bakers that know their respective areas of the city and bake the appropriate amount of bread at those locations.

Active skill acquisition by robots is an exciting area of current research. There are robots today, for example, that independently learn to walk or to perform
various motorskills related to soccer (Chap. 10). Cooperative learning of multiple robots to solve problems together is still in its infancy.

0.1.10 Al Grows Up

The above systems offered by AI today are not a universal recipe, but a workshop with a manageable number of tools for very different tasks. Most of these tools are well-developed and are available as finished software libraries, often with convenient user interfaces. The selection of the right tool and its sensible use in each individual case is left to the AI developer or knowledge engineer. Like any other artisanship, this requires a solid education, which this book is meant to promote.

More than nearly any other science, AI is interdisciplinary, for it draws upon interesting discoveries from such diverse fields as logic, operations research, statistics, control engineering, image processing, linguistics, philosophy, psychology, and neurobiology. On top of that, there is the subject area of the particular application. To successfully develop an AI project is therefore not always so simple, but almost always extremely exciting.

0.1.11 The Al Revolution

Around the year 2010 after about 25 years of research on neural networks, scientists could start harvesting the fruits of their research. The very powerful deep learning networks can for example learn to classify images with very high arruracy. Since image classification is of crucial importance for all types of smart robots, this initiated the AI revolution which in turn leads to smart self-driving cars and service robots.

0.1.12 AI and Society

There have been many scientific books and science fiction novels written on all aspects of this subject. Due to great advances in AI research, we have been on the brink of the age of autonomous robots and the Internet of Things since roughly 2005. Thus we are increasingly confronted with AI in everyday life. The reader, who may soon be working as an AI developer, must also deal with the social impact of this work. As an author of a book on AI techniques, I have the crucial task of examining this topic. I would like to deal with some particularly important aspects of AI which are of great practical relevance for our lives.

0.1.13 Does Al Destroy Jobs?

In January 2016, the World Econonic Forum published a study [[**?**]], frequently cited by the German press, predicting that ”industry 4.0 ” would destroy over five

million jobs in the next five years. This forecast is hardly surprising because automation in factories, offices, administration, transportation, in the home and in many other areas has led to continually more work being done by computers, machines and robots. AI has been one of the most important factors in this trend since about 2010. Presumably, the majority of people would gladly leave physically hard, dirty and unhealthy jobs and tasks to machines. Thus automation is a complete blessing for humanity, assuming it does not result in negative side effects, such as harm to the environment. Many of the aforementioned unpleasant jobs can be done faster, more precisely, and above all cheaper by machines. This seems almost like a trend towards paradise on Earth, where human beings do less and less unpleasant work and have correspondingly more time for the good things in life. This seems almost like a trend towards paradise on earth. We have to do less and less unpleasant work and in turn have more time for the good things in life. [4] All the while, we would enjoy the same (or potentially even increasing) prosperity, for the economy would not employ these machines if they did not markedly raise productivity.

Unfortunately we are not on the road to paradise. For several decades, we have worked more than 40 hours per week, have been stressed, complained of burnout and other sicknesses, and suffered a decline in real wages. How can this be, if productivity is continually increasing? Many economists say that the reason for this is competitive pressure. In an effort to compete and deliver the lowest priced goods to market, companies need to lower production costs and thus lay off workers. This results in the aforementioned unemployment. In order to avoid a drop in sales volume due to reduced prices, more products need to be manufactured and sold. The economy must grow!

If the economy continues to grow in a country in which the population is no longer growing (as is the case in most modern industrialized countries), each citizen must necessarily consume more. For that to happen, new markets must be created, [5] and marketing has the task of convincing us that we want the new products. This is-allegedly-the only way to ”sustainably” ensure prosperity. Apparently there seems to be no escape from this growth/consumption spiral. This has two fatal consequences. For one thing, this increase in consumption should make people happier, but it is having quite the opposite effect: mental illness is increasing.

Even more obvious and, above all, fatal, are economic growth's effects on our living conditions. It is no secret that the earth's growth limit has long been exceeded [[**?**], [**?**]], and that we are overexploiting nature's nonrenewable resources. We are therefore living at the expense of our children and grandchildren, who consequently will have poorer living conditions than we have today. It is also known that every additional dollar of economic growth is an additional burden on the environment-for example through additional CO_2 concentration in the atmosphere and the resulting climate change [[**?**]]. We are destroying our own basis of

existence. Thus it is obvious that we should abandon this path of growth for the sake of a livable future. But how? Let's think back to the road to paradise that AI is supposedly preparing for us. Apparently, as we practice it, it does not lead to paradise. Understanding this problem and finding the right path is one of the central tasks of today. Because of inherent complexities, this problem can not be fully dealt with in an introductory AI textbook. However, I would like to provide the reader with a little food for thought.

Although productivity is growing steadily in almost all areas of the economy, workers are required to work as hard as ever. They do not benefit from the increase in productivity. So, we must ask, where do the profits go? Evidently not to the people to whom they are owed, i.e. the workers. Instead, part of the profits is spent on investment and thus on further growth and the rest is taken by the capital owners, while employees work the same hours for declining real wages [[**?**]]. This leads to ever-increasing capital concentration among a few rich individuals and private banks, while on the other hand increasing poverty around the world is creating political tensions that result in war, expulsion and flight.

What is missing is a fair and just distribution of profits. How can this be achieved? Politicians and economists are continually trying to optimize our economic system, but politics has not offered a sustainable solution, and too few economists are investigating this highly exciting economic question. Obviously the attempt to optimize the parameters of our current capitalist economic system has not lead to a more equitable distribution of wealth, but to the opposite.

This is why economists and financial scientists must begin to question the system and look for alternatives. We should ask ourselves how to change the rules and laws of the economy so that all people profit from increased productivity. A growing community of economists and sustainability scientists have offered interesting solutions, a few of which I will briefly describe here.

---

[14] Those of us, such as scientists, computer scientists and engineers, who enjoy it may of course continue our work.

[5] Many EU and German Ministry of Education and Research funding programs for example require that scientists who submit proposals show evidence that their research will open up new markets.

Problem Number One is the creation of fiat money by the banks. New moneywhich is needed, among other things, to keep our growing economy going-is now being created by private banks. This is made possible by the fact that banks have to own only a small part, namely the minimum cash reserve ratio, of the money they give as loans. In the EU in 2016, the minimum cash reserve ratio is one percent.

States then borrow this money from private banks in the form of government bonds and thus fall into debt. This is how our current government debt crises have developed. This problem can be solved easily by prohibiting creation of money by the banks by increasing the minimum cash reserve ratio to 100%. State central banks will then get back the monopoly on creating money, and the newly created money can be used directly by the state for the purposes of social welfare. It should be evident that this simple measure would significantly ease the problem of public debt.

Further interesting components of such an economic reform could be the conversion of the current interest rate system to the so-called natural economic order [[**?**]], and the introduction of the "economy for the common good" [[**?**]] and the biophysical economy [[**?**], Küm11]. The practical implementation of the economy for the common good would involve a tax reform, the most important elements of which would be the abolition of the income tax and substantially increased value

added tax on energy and resource consumption. We would thus arrive at a highly prosperous, more sustainable human world with less environmental damage and more local trade. The reader may study the literature and assess whether the ideas quoted here are interesting and, if necessary, help to make the required changes. To conclude this section, I would like to quote the famous physicist Stephen Hawking. In a community-driven interview on www.reddit.com he gave the following answer to whether he had any thoughts about unemployment caused by automation:

> If machines produce everything we need, the outcome will depend on how things are distributed. Everyone can enjoy a life of luxurious leisure if the machine-produced wealth is shared, or most people can end up miserably poor if the machine-owners successfully lobby against wealth redistribution. So far, the trend seems to be toward the second option, with technology driving ever-increasing inequality.

Another Hawking quotation is also fitting. During the same interview, [6] to an AI professor's question about which moral ideas he should impart to his students, Hawking answered:

> ... Please encourage your students to think not only about how to create AI, but also about how to ensure its beneficial use.

As a consequence we should question the reasonableness of AI applications such as the export of intelligent cruise missiles to "allied" Arab states, the deployment of humanoid combat robots, etc.

0.1.14 AI and Transportation

In the past 130 years, automotive industry engineers have made great strides. In Germany, one out of every two people owns their own car. These cars are highly reliable. This makes us very mobile and we use this very convenient mobility in work, everyday life and leisure. Moreover, we are dependent on it. Today, we can not get by without a motor vehicle, especially in rural areas with weak public transportation infrastructure, as for instance in Upper Swabia, where the author and his students live.

The next stage of increased convenience in road transportation is now imminent. In a few years, we will be able to buy electric self-driving cars, i.e. robotic cars, which will autonomously bring us to almost any destination. All passengers in the robotic car would be able to read, work or sleep during the trip. This is possible on public transit already, but passengers in a robotic car would be able to do this at any time and on any route.

Autonomous vehicles that can operate independently could also travel without passengers. This will lead to yet another increase in convenience: robotic taxis. Via a smartphone app, we will be able to order the optimal taxi, in terms of size and equipment, for any conceivable transportation purpose. We will be able to choose whether we want to travel alone in the taxi or whether we are willing to share a ride with

other passengers. We will not need our own car anymore. All associated responsibilities and expenses, such as refueling, technical service, cleaning, searching for parking, buying and selling, garage rent, etc. are void, which saves us money and effort. Besides the immediate gains in comfort and convenience, robotic cars will offer other significant advantages. For example, according to a McKinsey study [[**?**]], we will need far fewer cars and, above all, far fewer parking places in the era of self-driving cars, which will lead to an immense reduction in resource consumption. According to a Lawrence Berkeley National Laboratory study [[**?**]], electric self-driving cars will cause a 90% reduction in green house emissions per passenger mile due to the vehicles' energy efficiency and the optimized fit between the vehicle and its purpose. Due to their optimal resource utilization, robotic taxis will be much more environmentally friendly than, for example, heavy buses, which often run at low capacity, especially in rural areas. Overall, robot taxis will contribute dramatically to energy savings and thus, among other things, to a significant improvement in CO_2 and climate problems.

Passenger safety will be much higher than it is today. Experts currently estimate future accident rates between zero and ten percent compared to today. Emotional driving ("road rage"), distracted driving and driving under the influence of drugs and alcohol will no longer exist.

Taxi drivers losing their jobs is often cited as a disadvantage of robotic cars. It is almost certain that there will no longer be taxi drivers from about 2030 onwards, but that is not necessarily a problem. As explained in the previous

[16] https://www.reddit.com/user/Prof-Stephen-Hawking.

section, our society just needs to deal with the newly gained productivity properly.

In addition to the many advantages mentioned above, robotic cars have two critical problems. Firstly, the so-called rebound effect will nullify at least some of the gains in resource, energy and time savings. Shorter driving times as well as more comfortable and cheaper driving will tempt us to drive more. We can only deal with this problem by rethinking our attitude towards consumption and quality of life. Do we have to use the entire time saved for more activities? Here we are all invited to critical reflection.

Another problem we should take seriously is that the robotic cars will need to be networked. In principle, this gives hackers and terrorists the ability to access and manipulate the vehicles' controls through security holes in their network protocols. If a hacker manages to do this once, he could repeat the attack on a grand scale, potentially bringing entire vehicle fleets to a halt, causing accidents, spying on vehicle occupants, or initiating other criminal actions. Here, as in other areas such as home automation and the Internet of Things, IT security experts will be needed to ensure the highest possible security guarantees using tools of the trade such as cryptographic methods. By the way, improved machine learning algorithms will be useful in detecting hacking attacks.

0.1.15 Service Robotics

In a few years, shortly after self-driving cars, the next bit of consumption bait on the shelves of electronics stores will be service robots. Recently the Google subsidiary

Boston Dynamics provided an impressive example in its humanoid robot Atlas.[7] Like the new cars, service robots offer a large gain in comfort and convenience which we would probably like to enjoy. One need only imagine such a robot dutifully cleaning and scrubbing after a party from night until morning without a grumble. Or think of the help that an assistance robot like Marvin, shown in Fig. 1.4, could provide to the elderly[8] or to people with disabilities [[**?**]].

In contrast to the robotic cars, however, these benefits come with costlier trade-offs. Completely new markets would be created, more natural resources and more energy would be consumed, and it is not even certain that people's lives would be simplified by the use of service robots in all areas. One of the first applications for robots like Atlas, developed by Boston Dynamics in contract with Google, will probably be military combat.

It is therefore all the more important that, before these robots come to market, we engage in social discourse on this topic. Science fiction films, such as "Ex Machina" (2015) with its female androids, the chilling "I, Robot" (2004) or the humorous "Robot and Frank" (2012), which depicts the pleasant side of a service robot as an old man's helper, can also contribute to such a discussion.

[17] https://youtu.be/rVlhMGQgDkY.

[8] In the coming demographic shift, assistance robots could become important for the elderly and

0.1.16 Agents

Although the term intelligent agents is not new to AI, only in recent years has it gained prominence through [RN10], among others. Agent denotes rather generally a system that processes information and produces an output from an input. These agents may be classified in many different ways.

In classical computer science, software agents are primarily employed (Fig. 1.5). In this case the agent consists of a program that calculates a result from user input.

In robotics, on the other hand, hardware agents (also called autonomous robots) are employed, which additionally have sensors and actuators at their disposal (Fig. 1.6). The agent can perceive its environment with the sensors. With the actuators it carries out actions and changes its environment.

With respect to the intelligence of the agent, there is a distinction between reflex agents, which only react to input, and agents with memory, which can also include the past in their decisions. For example, a driving robot that through its sensors knows its exact position (and the time) has no way, as a reflex agent, of determining its velocity. If, however, it saves the position, at short, discrete time steps, it can thus easily calculate its average velocity in the previous time interval.

If a reflex agent is controlled by a deterministic program, it represents a function of the set of all inputs to the set of all outputs. An agent with memory, on the other hand, is in general not a function. Why? (See Exercise 1.5 on page 21.) Reflex agents are sufficient in cases where the problem to be solved involves a Markov decision process. This is a process in which only the current state is needed to determine the optimal next action (see Chap. 10).

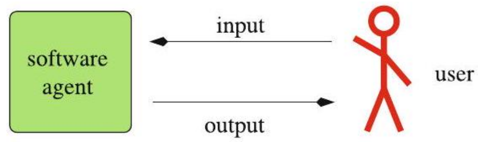
A mobile robot which should move from room 112 to room 179 in a building takes actions different from those of a robot that should move to room 105. In other words, the actions depend on the goal. Such agents are called goal-based.

Fig. 1.6 A hardware agent Example 1.1 A spam filter is an agent that puts incoming emails into wanted or unwanted (spam) categories, and deletes any unwanted emails. Its goal as a goalbased agent is to put all emails in the right category. In the course of this not-so-simple task, the agent can occasionally make mistakes. Because its goal is to classify all emails correctly, it will attempt to make as few errors as possible. However, that is not always what the user has in mind. Let us compare the following two agents. Out of 1,000 emails, Agent 1 makes only 12 errors. Agent 2 on the other hand makes 38 errors with the same 1,000 emails. Is it therefore worse than Agent 1? The errors of both agents are shown in more detail in the following table, the so-called ”confusion matrix”:

---

thus for our whole society.

.5 A software agent
ıser interaction



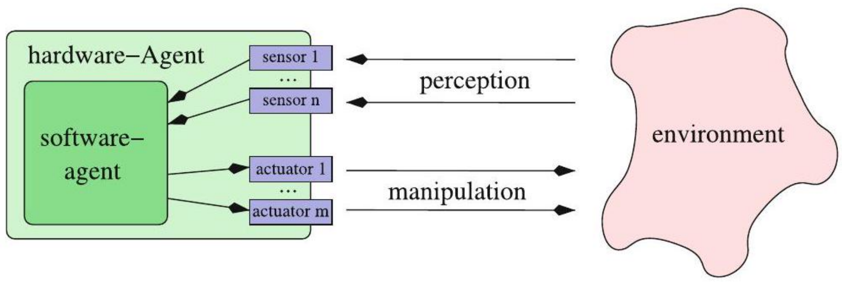


Figure 5: A software agent with user interaction

Agent 1:					Agent 2:				
		correct class					correct class		
		wanted	spam				wanted	spam	
spam filter decides	wanted	189	1		spam filter decides	wanted	200	38	
	spam	11	799			spam	0	762	

Agent 1 in fact makes fewer errors than Agent 2, but those few errors are severe because the user loses 11 potentially important emails. Because there are in this case two types of errors of differing severity, each error should be weighted with the appropriate cost factor (see Sect. 7.3.5 and Exercise 1.7 on page 21).

The sum of all weighted errors gives the total cost caused by erroneous decisions. The goal of a cost-based agent is to minimize the cost of erroneous decisions in the long term, that is, on average. In Sect. 7.3 we will become familiar with the medical diagnosis system LEXMED as an example of a cost-based agent.

Analogously, the goal of a utility-based agent is to maximize the utility derived from correct decisions in the long term, that is, on average. The sum of all decisions weighted by their respective utility factors gives the total utility.

Of particular interest in AI are Learning agents, which are capable of changing themselves given training examples or through positive or negative feedback, such that the average utility of their actions grows over time (see Chap. 8).

As mentioned in Sect. 1.2.5, distributed agents are increasingly coming into use, whose intelligence are not localized in one agent, but rather can only be seen through cooperation of many agents.

The design of an agent is oriented, along with its objective, strongly toward its environment, or alternately its picture of the environment, which strongly depends on it sensors. The environment is observable if the agent always knows the complete state of the world. Otherwise the environment is only partially observable. If an action always leads to the same result, then the environment is deterministic. Otherwise it is nondeterministic. In a discrete environment only finitely many states and actions occur, whereas a continuous environment boasts infinitely many states or actions.

0.1.17 Knowledge-Based Systems

An agent is a program that implements a mapping from perceptions to actions. For simple agents this way of looking at the problem is sufficient. For complex applications in which the agent must be able to rely on a large amount of information and is meant to do a difficult task, programming the agent can be very costly and unclear how to proceed. Here AI provides a clear path to follow that will greatly simplify the work. First we separate knowledge from the system or program, which uses the knowledge to, for example, reach conclusions, answer queries, or come up with a plan. This system is called the inference mechanism. The knowledge is stored in a knowledge base (KB). Acquisition of knowledge in the knowledge base is denoted Knowledge Engineering and is based on various knowledge sources such as human experts, the knowledge engineer, and databases. Active learning systems can also acquire knowledge through active exploration of the world (see Chap. 10). In Fig. 1.7 the general architecture of knowledge-based systems is presented.

Moving toward a separation of knowledge and inference has several crucial advantages. The separation of knowledge and inference can allow inference systems to be implemented in a largely application-independent way. For example, application of a medical expert system to other diseases is much easier by replacing the knowledge base rather than by programming a whole new system.

Through the decoupling of the knowledge base from inference, knowledge can be stored declaratively. In the knowledge base there is only a description of the knowledge, which is independent from the inference system in use. Without this clear separation, knowledge and processing of inference steps would be interwoven, and any changes to the knowledge would be very costly.

Formal language as a convenient interface between man and machine lends itself to the representation of knowledge in the knowledge base. In the following chapters we will get to know a whole series of such languages. First, in Chaps. 2 and 3 there are propositional calculus and first-order predicate logic (PL1). But other formalisms such as probabilistic logic and decision trees are also presented. We start with propositional calculus and the related inference systems. Building on that, we will present predicate logic, a powerful language that is accessible by machines and very important in AI. As an example for a large scale knowledge based system we want to refer to the software agent ”Watson”. Developed at IBM together with a number of universities, Watson is a question answering program, that can be fed with clues given in natural language. It

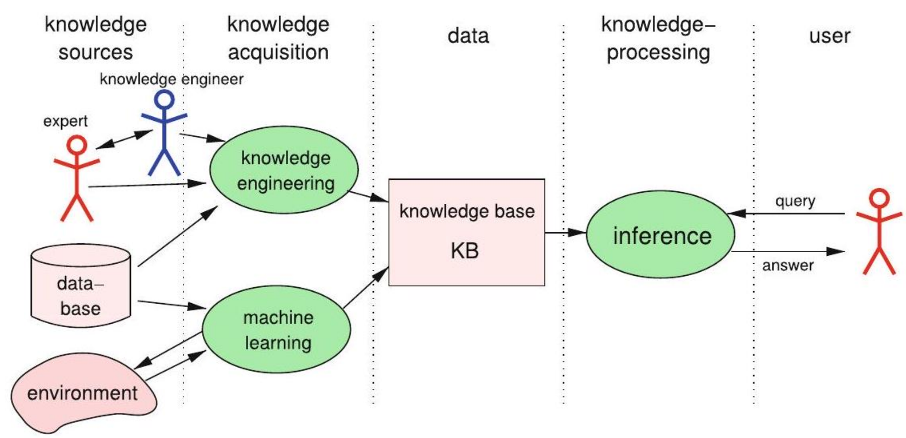


Figure 6: Structure of a classic knowledge-processing system

works on a knowledge base comprising four terabytes of hard disk storage, including the full text of Wikipedia [[**?**]]. Watson was developed within IBM's DeepQA project which is characterized in [[**?**]] as follows:

> The DeepQA project at IBM shapes a grand challenge in Computer Science that aims to illustrate how the wide and growing accessibility of natural language content and the integration and advancement of Natural Language Processing, Information Retrieval, Machine Learning, Knowledge Representation and Reasoning, and massively parallel computation can drive open-domain automatic Question Answering technology to a point where it clearly and consistently rivals the best human performance.

In the U.S. television quiz show "Jeopardy!", in February 2011, Watson defeated the two human champions Brad Rutter and Ken Jennings in a two-game, combined-point match and won the one million dollar price. One of Watson's particular strengths was its very fast reaction to the questions with the result that Watson often hit the buzzer (using a solenoid) faster than its human competitors and then was able to give the first answer to the question.

The high performance and short reaction times of Watson were due to an implementation on 90 IBM Power 750 servers, each of which contains 32 processors, resulting in 2880 parallel processors.

0.1.18 Exercises

Exercise 1.1 Test some of the chatterbots available on the internet. Start for example with www.hs-weingarten.de/ ~ ertel/aibook in the collection of links under Turingtest/Chatterbots, or at www.simonlaven.com or www.alicebot.org.

Write down a starting question and measure the time it takes, for each of the various programs, until you know for certain that it is not a human.

菜菜 Exercise 1.2 At www.pandorabots.com you will find a server on which you can build a chatterbot with the markup language AIML quite easily. Depending on your interest level, develop a simple or complex chatterbot, or change an existing one.

Exercise 1.3 Give reasons for the unsuitability of the Turing test as a definition of ”artificial intelligence” in practical AI.

0.2 Exercise 1.4

Many well-known inference processes, learning processes, etc. are NP-complete or even undecidable. What does this mean for AI?

0.3 Exercise 1.5

(a) Why is a deterministic agent with memory not a function from the set of all inputs to the set of all outputs, in the mathematical sense?
(b) How can one change the agent with memory, or model it, such that it becomes equivalent to a function but does not lose its memory?

0.4 Exercise 1.6

Let there be an agent with memory that can move within a plane. From its sensors, it receives at clock ticks of a regular interval Δt its exact position (x, y) in Cartesian coordinates.
(a) Give a formula with which the agent can calculate its velocity from the current time t and the previous measurement of $t - \Delta t$.
(b) How must the agent be changed so that it can also calculate its acceleration? Provide a formula here as well.

0.5 Exercise 1.7

(a) Determine for both agents in Example 1.1 on page 18 the costs created by the errors and compare the results. Assume here that having to manually delete a spam email costs one cent and retrieving a deleted email, or the loss of an email, costs one dollar.
(b) Determine for both agents the profit created by correct classifications and compare the results. Assume that for every desired email recognized, a profit of one dollar accrues and for every correctly deleted spam email, a profit of one cent.

0.6 Propositional Logic

In propositional logic, as the name suggests, propositions are connected by logical operators. The statement “the street is wet” is a proposition, as is “it is raining”. These two propositions can be connected to form the new proposition if it is raining the street is wet.
Written more formally
it is raining $\Rightarrow$ the street is wet.
This notation has the advantage that the elemental propositions appear again in unaltered form. So that we can work with propositional logic precisely, we will begin with a definition of the set of all propositional logic formulas.

0.6.1 Syntax

Definition 2.1 Let $Op = \{\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow, (,)\}$ be the set of logical operators and let Σ be a set of symbols. The sets Op, Σ and $\{t, f\}$ are pairwise disjoint. Σ is called the signature and its elements are the proposition variables. The set of propositional logic formulas is now recursively defined:

- t and f are (atomic) formulas.
- All proposition variables, that is all elements from Σ, are (atomic) formulas.
- If A and B are formulas, then $\neg A, (A), A \wedge B, A \vee B, A \Rightarrow B, A \Leftrightarrow B$ are also formulas.

This elegant recursive definition of the set of all formulas allows us to generate infinitely many formulas. For example, given $\Sigma = \{A, B, C\}$,

$$A \wedge B, \quad A \wedge B \wedge C, \quad A \wedge A \wedge A, \quad C \wedge B \vee A, \quad (\neg A \wedge B) \Rightarrow (\neg C \vee A)$$

are formulas. $(((A)) \vee B)$ is also a syntactically correct formula.
Definition 2.2 We read the symbols and operators in the following way:

t :	”true”	
f :	”false”	
$\neg A$:	”not A”	(negation)
$A \wedge B$:	”A and B”	(conjunction)
$A \vee B$:	”A or B”	(disjunction)
$A \Rightarrow B$:	”if A then B”	(implication (also called *material implication*))

The formulas defined in this way are so far purely syntactic constructions without meaning. We are still missing the semantics.

0.6.2 Semantics

In propositional logic there are two truth values: t for "true" and f for "false". We begin with an example and ask ourselves whether the formula $A \wedge B$ is true. The answer is: it depends on whether the variables A and B are true. For example, if A stands for "It is raining today" and B for "It is cold today" and these are both true, then $A \wedge B$ is true. If, however, B represents "It is hot today" (and this is false), then $A \wedge B$ is false.

We must obviously assign truth values that reflect the state of the world to proposition variables. Therefore we define

> Definition 2.3 A mapping $I : \Sigma \to \{t, f\}$, which assigns a truth value to every proposition variable, is called an interpretation.

Because every proposition variable can take on two truth values, every propositional logic formula with n different variables has 2^n different interpretations. We define the truth values for the basic operations by showing all possible interpretations in a truth table (see Table 2.1 on page 25).

Table 2.1 Definition of the logical operators by truth table

A	B	(A)	$\neg A$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
t	t	t	f	t	t	t	t
t	f	t	f	f	t	f	f
f	t	f	t	f	t	t	f
f	f	f	t	f	f	t	t

The empty formula is true for all interpretations. In order to determine the truth value for complex formulas, we must also define the order of operations for logical operators. If expressions are parenthesized, the term in the parentheses is evaluated first. For unparenthesized formulas, the priorities are ordered as follows, beginning with the strongest binding: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.

To clearly differentiate between the equivalence of formulas and syntactic equivalence, we define

Definition 2.4 Two formulas F and G are called semantically equivalent if they take on the same truth value for all interpretations. We write $F \equiv G$.

Semantic equivalence serves above all to be able to use the meta-language, that is, natural language, to talk about the object language, namely logic. The statement "$A \equiv B$" conveys that the two formulas A and B are semantically equivalent. The statement "$A \Leftrightarrow B$" on the other hand is a syntactic object of the formal language of propositional logic.

According to the number of interpretations in which a formula is true, we can divide formulas into the following classes:

Definition 2.5 A formula is called

- Satisfiable if it is true for at least one interpretation.
- Logically valid or simply valid if it is true for all interpretations. True formulas are also called tautologies.

- Unsatisfiable if it is not true for any interpretation.

Every interpretation that satisfies a formula is called a model of the formula.

Clearly the negation of every generally valid formula is unsatisfiable. The negation of a satisfiable, but not generally valid formula F is satisfiable.

We are now able to create truth tables for complex formulas to ascertain their truth values. We put this into action immediately using equivalences of formulas which are important in practice.

Theorem 2.1 The operations $\wedge, \vee$ are commutative and associative, and the following equivalences are generally valid:

$$
\begin{array}{rcll}
\neg A \vee B & \Leftrightarrow & A \Rightarrow B & (\textit{implication}) \\
A \Rightarrow B & \Leftrightarrow & \neg B \Rightarrow \neg A & (\textit{contraposition}) \\
(A \Rightarrow B) \wedge (B \Rightarrow A) & \Leftrightarrow & (A \Leftrightarrow B) & (\textit{equivalence}) \\
\neg(A \wedge B) & \Leftrightarrow & \neg A \vee \neg B & (\textit{De Morgan's law}) \\
\neg(A \vee B) & \Leftrightarrow & \neg A \wedge \neg B & \\
A \vee (B \wedge C) & \Leftrightarrow & (A \vee B) \wedge (A \vee C) & (\textit{distributive law}) \\
A \wedge (B \vee C) & \Leftrightarrow & (A \wedge B) \vee (A \wedge C) & \\
A \vee \neg A & \Leftrightarrow & w & (\textit{tautology}) \\
A \wedge \neg A & \Leftrightarrow & f & (\textit{contradiction}) \\
A \vee f & \Leftrightarrow & A & \\
A \vee w & \Leftrightarrow & w & \\
A \wedge f & \Leftrightarrow & f & \\
A \wedge w & \Leftrightarrow & A &
\end{array}
$$

Proof To show the first equivalence, we calculate the truth table for $\neg A \vee B$ and $A \Rightarrow B$ and see that the truth values for both formulas are the same for all interpretations. The formulas are therefore equivalent, and thus all the values of the last column are "t"s.

A	B	$\neg A$	$\neg A \vee B$	$A \Rightarrow B$	$(\neg A \vee B) \Leftrightarrow (A \Rightarrow B)$
t	t	f	t	t	t
t	f	f	f	f	t
f	t	t	t	t	t
f	f	t	t	t	t

The proofs for the other equivalences are similar and are recommended as exercises for the reader (Exercise 2.2 on page 37).

0.6.3 Proof Systems

In AI we are interested in taking existing knowledge and from that deriving new knowledge or answering questions. In propositional logic this means showing that a knowledge base KB-that is, a (possibly extensive) propositional logic formulaa formula Q[1] follows. Thus, we first define the term "entailment".

[11] Here Q stands for query.

Definition 2.6 A formula KB entails a formula Q (or Q follows from KB) if every model of KB is also a model of Q. We write $KB \models Q$.

In other words, in every interpretation in which KB is true, Q is also true. More succinctly, whenever KB is true, Q is also true. Because, for the concept of entailment, interpretations of variables are brought in, we are dealing with a semantic concept.

Every formula that is not valid chooses so to speak a subset of the set of all interpretations as its model. Tautologies such as $A \vee \neg A$, for example, do not restrict the number of satisfying interpretations because their proposition is empty. The empty formula is therefore true in all interpretations. For every tautology T then $\emptyset \models T$. Intuitively this means that tautologies are always true, without restriction of the interpretations by a formula. For short we write $\models T$. Now we show an important connection between the semantic concept of entailment and syntactic implication.

Theorem 2.2 (Deduktionstheorem)

$$A \models B \text{ if and only if } \models A \Rightarrow B.$$

Proof Observe the truth table for implication:

A	B	$A \Rightarrow B$
t	t	t
t	f	f
f	t	t
f	f	t

An arbitrary implication $A \Rightarrow B$ is clearly always true except with the interpretation $A \longmapsto t, B \longmapsto f$. Assume that $A \models B$ holds. This means that for every interpretation that makes A true, B is also true. The critical second row of the truth table does not even apply in that case. Therefore $A \Rightarrow B$ is true, which means that $A \Rightarrow B$ is a tautology. Thus one direction of the statement has been shown.

Now assume that $A \Rightarrow B$ holds. Thus the critical second row of the truth table is also locked out. Every model of A is then also a model of B. Then $A \models B$ holds.

If we wish to show that KB entails Q, we can also demonstrate by means of the truth table method that $KB \Rightarrow Q$ is a tautology. Thus we have our first proof system for propositional logic, which is easily automated. The disadvantage of this method is the very long computation time in the worst case. Specifically, in the worst case with n proposition variables, for all 2^n interpretations of the variables the formula $KB \Rightarrow Q$ must be evaluated. The computation time grows therefore exponentially with the number of variables. Therefore this process is unusable for large variable counts, at least in the worst case. If a formula KB entails a formula Q, then by the deduction theorem $KB \Rightarrow Q$ is a tautology. Therefore the negation $\neg(KB \Rightarrow Q)$ is unsatisfiable. We have

$$\neg(KB \Rightarrow Q) \equiv \neg(\neg KB \vee Q) \equiv KB \wedge \neg Q.$$

Therefore, $KB \wedge \neg Q$ is also unsatisfiable. We formulate this simple, but important consequence of the deduction theorem as a theorem.

Theorem 2.3 (Proof by contradiction) $KB \models Q$ if and only if $KB \wedge \neg Q$ is unsatisfiable.

To show that the query Q follows from the knowledge base KB, we can also add the negated query $\neg Q$ to the knowledge base and derive a contradiction. Because of the equivalence $A \wedge \neg A \Leftrightarrow f$ from Theorem 2.1 on page 26 we know that a contradiction is unsatisfiable. Therefore, Q has been proved. This procedure, which is frequently used in mathematics, is also used in various automatic proof calculi such as the resolution calculus and in the processing of PROLOG programs.

One way of avoiding having to test all interpretations with the truth table method is the syntactic manipulation of the formulas KB and Q by application of inference rules with the goal of greatly simplifying them, such that in the end we can instantly see that $KB \models Q$. We call this syntactic process derivation and write $KB \vdash Q$. Such syntactic proof systems are called calculi. To ensure that a calculus does not generate errors, we define two fundamental properties of calculi.

Definition 2.7 A calculus is called sound if every derived proposition follows semantically. That is, if it holds for formulas KB and Q that

$$\text{if } KB \vdash Q \text{ then } KB \models Q.$$

A calculus is called complete if all semantic consequences can be derived. That is, for formulas KB and Q the following holds:

$$\text{if} \quad KB \models Q \quad \text{then} \quad KB \vdash Q.$$

The soundness of a calculus ensures that all derived formulas are in fact semantic consequences of the knowledge base. The calculus does not produce any "false consequences". The completeness of a calculus, on the other hand, ensures that the calculus does not overlook anything. A complete calculus always finds a proof if

the formula to be proved follows from the knowledge base. If a calculus is sound and complete, then syntactic derivation and semantic entailment are two equivalent relations (see Fig. 2.1).

To keep automatic proof systems as simple as possible, these are usually made to operate on formulas in conjunctive normal form.

Definition 2.8 A formula is in conjunctive normal form (CNF) if and only if it consists of a conjunction

$$K_1 \wedge K_2 \wedge \cdots \wedge K_m$$

of clauses. A clause K_i consists of a disjunction

$$(L_{i1} \vee L_{i2} \vee \cdots \vee L_{in_i})$$

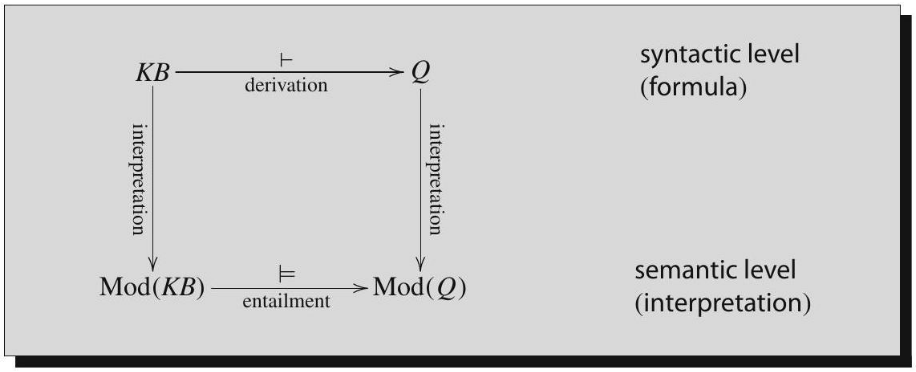


Figure 7: Syntactic derivation and semantic entailment. $\mathrm{Mod}(X)$ represents the set of models of a formula X.

of literals. Finally, a literal is a variable (positive literal) or a negated variable (negative literal).

The formula $(A \vee B \vee \neg C) \wedge (A \vee B) \wedge (\neg B \vee \neg C)$ is in conjunctive normal form. The conjunctive normal form does not place a restriction on the set of formulas because:

Theorem 2.4 Every propositional logic formula can be transformed into an equivalent conjunctive normal form.

Example 2.1 We put $A \vee B \Rightarrow C \wedge D$ into conjunctive normal form by using the equivalences from Theorem 2.1 on page 26:

$$\begin{aligned}
& A \vee B \Rightarrow C \wedge D \\
& \quad \equiv \neg(A \vee B) \vee (C \wedge D) && \text{(implication)} \\
& \quad \equiv (\neg A \wedge \neg B) \vee (C \wedge D) && \text{(de Morgan)} \\
& \quad \equiv (\neg A \vee (C \wedge D)) \wedge (\neg B \vee (C \wedge D)) && \text{(distributive law)} \\
& \quad \equiv ((\neg A \vee C) \wedge (\neg A \vee D)) \wedge ((\neg B \vee C) \wedge (\neg B \vee D)) && \text{(distributive law)} \\
& \quad \equiv (\neg A \vee C) \wedge (\neg A \vee D) \wedge (\neg B \vee C) \wedge (\neg B \vee D) && \text{(associative law)}
\end{aligned}$$

We are now only missing a calculus for syntactic proof of propositional logic formulas. We start with the modus ponens, a simple, intuitive rule of inference, which, from the validity of A and $A \Rightarrow B$, allows the derivation of B. We write this formally as

$$\frac{A, \quad A \Rightarrow B}{B}.$$

This notation means that we can derive the formula(s) below the line from the comma-separated formulas above the line. Modus ponens as a rule by itself, while sound, is not complete. If we add additional rules we can create a complete calculus, which, however, we do not wish to consider here. Instead we will investigate the resolution rule

$$\frac{A \vee B, \quad \neg B \vee C}{A \vee C} \tag{2.1}$$

as an alternative. The derived clause is called resolvent. Through a simple transformation we obtain the equivalent form

$$\frac{A \vee B, \quad B \Rightarrow C}{A \vee C}.$$

If we set A to f, we see that the resolution rule is a generalization of the modus ponens. The resolution rule is equally usable if C is missing or if A and C are missing. In the latter case the empty clause can be derived from the contradiction $B \wedge \neg B$ (Exercise 2.7 on page 38).

0.6.4 Resolution

We now generalize the resolution rule again by allowing clauses with an arbitrary number of literals. With the literals $A_1, \ldots, A_m, B, C_1, \ldots, C_n$ the general resolution rule reads

$$\frac{(A_1 \vee \cdots \vee A_m \vee B), \quad (\neg B \vee C_1 \vee \cdots \vee C_n)}{(A_1 \vee \cdots \vee A_m \vee C_1 \vee \cdots \vee C_n)}. \tag{2.2}$$

We call the literals B and $\neg B$ complementary. The resolution rule deletes a pair of complementary literals from the two clauses and combines the rest of the literals into a new clause. To prove that from a knowledge base KB, a query Q follows, we carry out a proof by contradiction. Following Theorem 2.3 on page 28 we must show that a contradiction can be derived from $KB \wedge \neg Q$. In formulas in conjunctive normal form, a contradiction appears in the form of two clauses (A) and $(\neg A)$, which lead to the empty clause as their resolvent. The following theorem ensures us that this process really works as desired.

For the calculus to be complete, we need a small addition, as shown by the following example. Let the formula $(A \vee A)$ be given as our knowledge base. To show by the resolution rule that from there we can derive $(A \wedge A)$, we must show that the empty clause can be derived from $(A \vee A) \wedge (\neg A \vee \neg A)$. With the resolution rule alone, this is impossible. With factorization, which allows deletion of copies of literals from clauses, this problem is eliminated. In the example, a double application of factorization leads to $(A) \wedge (\neg A)$, and a resolution step to the empty clause.

> Theorem 2.5 The resolution calculus for the proof of unsatisfiability of formulas in conjunctive normal form is sound and complete.

Because it is the job of the resolution calculus to derive a contradiction from $KB \wedge \neg Q$, it is very important that the knowledge base KB is consistent:

Definition 2.9 A formula KB is called consistent if it is impossible to derive from it a contradiction, that is, a formula of the form $\phi \wedge \neg \phi$.

Otherwise anything can be derived from KB (see Exercise 2.8 on page 38). This is true not only of resolution, but also for many other calculi.

Of the calculi for automated deduction, resolution plays an exceptional role. Thus we wish to work a bit more closely with it. In contrast to other calculi, resolution has only two inference rules, and it works with formulas in conjunctive normal form. This makes its implementation simpler. A further advantage compared to many calculi lies in its reduction in the number of possibilities for the application of inference rules in every step of the proof, whereby the search space is reduced and computation time decreased.

As an example, we start with a simple logic puzzle that allows the important steps of a resolution proof to be shown.

Example 2.2 Logic puzzle number 7, entitled A charming English family, from the German book [[**?**]] reads (translated to English): Despite studying English for seven long years with brilliant success, I must admit that when I hear English people speaking English I'm totally perplexed. Recently, moved by noble feelings, I picked up three hitchhikers, a father, mother, and daughter, who I quickly realized were English and only spoke English. At each of the sentences that follow I wavered between two possible interpretations. They told me the following (the second possible meaning is in parentheses): The father: ”We are going to Spain (we are from Newcastle).” The mother: ”We are not going to Spain and are from Newcastle (we stopped in Paris and are not going to Spain).” The daughter: ”We are not from Newcastle (we stopped in Paris).” What about this charming English family?

To solve this kind of problem we proceed in three steps: formalization, transformation into normal form, and proof. In many cases formalization is by far the most difficult step because it is easy to make mistakes or forget small details. (Thus practical exercise is very important. See Exercises 2.9-2.11 on page 38.)

Here we use the variables S for ”We are going to Spain”, N for ”We are from Newcastle”, and P for ”We stopped in Paris” and obtain as a formalization of the three propositions of father, mother, and daughter

$$(S \vee N) \wedge [(\neg S \wedge N) \vee (P \wedge \neg S)] \wedge (\neg N \vee P).$$

Factoring out $\neg S$ in the middle sub-formula brings the formula into CNF in one step. Numbering the clauses with subscripted indices yields

$$KB \equiv (S \vee N)_1 \wedge (\neg S)_2 \wedge (P \vee N)_3 \wedge (\neg N \vee P)_4.$$

Now we begin the resolution proof, at first still without a query Q. An expression of the form ” Res(m, n) : $\langle$ clause $\rangle_k$ ” means that $\langle$ clause $\rangle$ is obtained by resolution of clause m with clause n and is numbered k.

$$\begin{array}{ll}
\text{Res}(1,2): & (N)_5 \\
\text{Res}(3,4): & (P)_6 \\
\text{Res}(1,4): & (S \vee P)_7
\end{array}$$

We could have derived clause P also from Res$(4, 5)$ or Res$(2, 7)$. Every further resolution step would lead to the derivation of clauses that are already available. Because it does not allow the derivation of the empty clause, it has therefore been shown that the knowledge base is non-contradictory. So far we have derived N and P. To show that $\neg S$ holds, we add the clause $(S)_8$ to the set of clauses as a negated query. With the resolution step

$$\text{Res}(2, 8): \quad ()_9$$

the proof is complete. Thus $\neg S \wedge N \wedge P$ holds. The "charming English family" evidently comes from Newcastle, stopped in Paris, but is not going to Spain.

Example 2.3 Logic puzzle number 28 from [Ber89], entitled The High Jump, reads

Three girls practice high jump for their physical education final exam. The bar is set to 1.20 meters. "I bet", says the first girl to the second, "that I will make it over if, and only if, you don't". If the second girl said the same to the third, who in turn said the same to the first, would it be possible for all three to win their bets?

We show through proof by resolution that not all three can win their bets. Formalization:

The first girl's jump succeeds: $A,$ First girl's bet: $(A \Leftrightarrow \neg B)$,
the second girl's jump succeeds: $B,$ second girl's bet: $(B \Leftrightarrow \neg C)$,
the third girl's jump succeeds: $C.$ third girl's bet: $(C \Leftrightarrow \neg A)$.
Claim: the three cannot all win their bets:

$$Q \equiv \neg((A \Leftrightarrow \neg B) \wedge (B \Leftrightarrow \neg C) \wedge (C \Leftrightarrow \neg A))$$

It must now be shown by resolution that $\neg Q$ is unsatisfiable.
Transformation into CNF: First girl's bet:

$$(A \Leftrightarrow \neg B) \equiv (A \Rightarrow \neg B) \wedge (\neg B \Rightarrow A) \equiv (\neg A \vee \neg B) \wedge (A \vee B)$$

The bets of the other two girls undergo analogous transformations, and we obtain the negated claim

$$\begin{aligned}\neg Q \equiv &(\neg A \vee \neg B)_1 \wedge (A \vee B)_2 \wedge (\neg B \vee \neg C)_3 \wedge (B \vee C)_4 \wedge (\neg C \vee \neg A)_5 \\ &\wedge (C \vee A)_6.\end{aligned}$$

From there we derive the empty clause using resolution:

$$\begin{array}{ll}\text{Res}(1, 6): & (C \vee \neg B)_7 \\ \text{Res}(4, 7): & (C)_8 \\ \text{Res}(2, 5): & (B \vee \neg C)_9 \\ \text{Res}(3, 9): & (\neg C)_{10} \\ \text{Res}(8, 10): & ()\end{array}$$

Thus the claim has been proved.

0.6.5 Horn Clauses

A clause in conjunctive normal form contains positive and negative literals and can be represented in the form

$$(\neg A_1 \vee \cdots \vee \neg A_m \vee B_1 \vee \cdots \vee B_n)$$

with the variables $A_1, \ldots, A_m$ and $B_1, \ldots, B_n$. This clause can be transformed in two simple steps into the equivalent form

$$A_1 \wedge \cdots \wedge A_m \Rightarrow B_1 \vee \cdots \vee B_n.$$

This implication contains the premise, a conjunction of variables and the conclusion, a disjunction of variables. For example, "If the weather is nice and there is snow on the ground, I will go skiing or I will work." is a proposition of this form. The receiver of this message knows for certain that the sender is not going swimming. A significantly clearer statement would be "If the weather is nice and there is snow on the ground, I will go skiing.". The receiver now has definite information. Thus we call clauses with at most one positive literal definite clauses. These clauses have the advantage that they only allow one conclusion and are thus distinctly simpler to interpret. Many relations can be described by clauses of this type. We therefore define

Definition 2.10 Clauses with at most one positive literal of the form

$$(\neg A_1 \vee \cdots \vee \neg A_m \vee B) \quad \text{or} \quad (\neg A_1 \vee \cdots \vee \neg A_m) \quad \text{or} \quad B$$

or (equivalently)

$$A_1 \wedge \cdots \wedge A_m \Rightarrow B \quad \text{or} \quad A_1 \wedge \cdots \wedge A_m \Rightarrow f \quad \text{or} \quad B.$$

are named Horn clauses (after their inventor). A clause with a single positive literal is a fact. In clauses with negative and one positive literal, the positive literal is called the head.

To better understand the representation of Horn clauses, the reader may derive them from the definitions of the equivalences we have currently been using (Exercise 2.12 on page 38).

Horn clauses are easier to handle not only in daily life, but also in formal reasoning, as we can see in the following example. Let the knowledge base consist of the following clauses (the " $\wedge$ " binding the clauses is left out here and in the text that follows):

$$\begin{aligned}
&(\text{nice_weather})_1 \\
&(\text{snowfall})_2 \\
&(\text{snowfall} \Rightarrow \text{snow})_3 \\
&(\text{nice_weather} \wedge \text{snow} \Rightarrow \text{skiing})_4
\end{aligned}$$

If we now want to know whether skiing holds, this can easily be derived. A slightly generalized modus ponens suffices here as an inference rule:

$$\frac{A_1 \wedge \cdots \wedge A_m, \quad A_1 \wedge \cdots \wedge A_m \Rightarrow B}{B}.$$

The proof of ”skiing” has the following form (MP $(i_1, \ldots, i_k)$ represents application of the modus ponens on clauses i_1 to i_k) :

$$\begin{array}{ll} \text{MP}(2,3): & (\text{ snow })_5 \\ \text{MP}(1,5,4): & (\text{ skiing })_6. \end{array}$$

With modus ponens we obtain a complete calculus for formulas that consist of propositional logic Horn clauses. In the case of large knowledge bases, however, modus ponens can derive many unnecessary formulas if one begins with the wrong clauses. Therefore, in many cases it is better to use a calculus that starts with the query and works backward until the facts are reached. Such systems are designated backward chaining, in contrast to forward chaining systems, which start with facts and finally derive the query, as in the above example with the modus ponens.

For backward chaining of Horn clauses, SLD resolution is used. SLD stands for ”Selection rule driven linear resolution for definite clauses”. In the above example, augmented by the negated query (skiing $\Rightarrow f$)

$$\begin{array}{l} (\text{nice\_weather})_1 \\ (\text{snowfall})_2 \\ (\text{snowfall} \Rightarrow \text{snow})_3 \\ (\text{nice\_weather} \wedge \text{snow} \Rightarrow \text{skiing})_4 \\ (\text{skiing} \Rightarrow f)_5 \end{array}$$

we carry out SLD resolution beginning with the resolution steps that follow from this clause

$$\begin{array}{ll} \text{Res}(5,4): & (\text{ nice\_weather } \wedge \text{ snow } \Rightarrow f)_6 \\ \text{Res}(6,1): & (\text{ snow } \Rightarrow f)_7 \\ \text{Res}(7,3): & (\text{ snowfall } \Rightarrow f)_8 \\ \text{Res}(8,2): & () \end{array}$$

and derive a contradiction with the empty clause. Here we can easily see ”linear resolution”, which means that further processing is always done on the currently derived clause. This leads to a great reduction of the search space. Furthermore, the
literals of the current clause are always processed in a fixed order (for example, from right to left) (”Selection rule driven”). The literals of the current clause are called subgoal. The literals of the negated query are the goals. The inference rule for one step reads

$$\frac{A_1 \wedge \cdots \wedge A_m \Rightarrow B_1, \quad B_1 \wedge B_2 \wedge \cdots \wedge B_n \Rightarrow f}{A_1 \wedge \cdots \wedge A_m \wedge B_2 \wedge \cdots \wedge B_n \Rightarrow f}.$$

Before application of the inference rule, $B_1, B_2, \ldots, B_n$—the current subgoals—must be proved. After the application, B_1 is replaced by the new subgoal $A_1 \wedge \cdots \wedge A_m$. To show that B_1 is true, we must now show that $A_1 \wedge \cdots \wedge A_m$ are true. This process continues until the list of subgoals of the current clauses (the so-called goal stack) is empty. With that, a contradiction has been found. If, for a subgoal $\neg B_i$, there is no clause with the complementary literal B_i as its clause head, the proof terminates and no contradiction can be found. The query is thus unprovable.

SLD resolution plays an important role in practice because programs in the logic programming language PROLOG consist of predicate logic Horn clauses, and their processing is achieved by means of SLD resolution (see Exercise 2.13 on page 38, or Chap. 5).

0.6.6 Computability and Complexity

The truth table method, as the simplest semantic proof system for propositional logic, represents an algorithm that can determine every model of any formula in finite time. Thus the sets of unsatisfiable, satisfiable, and valid formulas are decidable. The computation time of the truth table method for satisfiability grows in the worst case exponentially with the number n of variables because the truth table has 2^n rows. An optimization, the method of semantic trees, avoids looking at variables that do not occur in clauses, and thus saves computation time in many cases, but in the worst case it is likewise exponential.

In resolution, in the worst case the number of derived clauses grows exponentially with the number of initial clauses. To decide between the two processes, we can therefore use the rule of thumb that in the case of many clauses with few variables, the truth table method is preferable, and in the case of few clauses with many variables, resolution will probably finish faster.

The question remains: can proof in propositional logic go faster? Are there better algorithms? The answer: probably not. After all, S. Cook, the founder of complexity theory, has shown that the 3-SAT problem is NP-complete. 3-SAT is the set of all CNF formulas whose clauses have exactly three literals. Thus it is clear that there is probably (modulo the P/NP problem) no polynomial algorithm for 3-SAT, and thus probably not a general one either. For Horn clauses, however, there is an algorithm in which the computation time for testing satisfiability grows only linearly as the number of literals in the formula increases.

0.6.7 Applications and Limitations

Theorem provers for propositional logic are part of the developer's everyday toolset in digital technology. For example, the verification of digital circuits and the generation of test patterns for testing of microprocessors in fabrication are some of these tasks. Special proof systems that work with binary decision diagrams (BDD) are also employed as a data structure for processing propositional logic formulas. In AI, propositional logic is employed in simple applications.

For example, simple expert systems can certainly work with propositional logic. However, the variables must all be discrete, with only a few values, and there may not be any cross-relations between variables. Complex logical connections can be expressed much more elegantly using predicate logic.

Probabilistic logic is a very interesting and current combination of propositional logic and probabilistic computation that allows modeling of uncertain knowledge. It is handled thoroughly in Chap. 7.

0.6.8 Exercises

#7 Exercise 2.1 Give a Backus-Naur form grammar for the syntax of propositional logic.

Exercise 2.2 Show that the following formulas are tautologies:
(a) $\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$
(b) $A \Rightarrow B \Leftrightarrow \neg B \Rightarrow \neg A$
(c) $((A \Rightarrow B) \wedge (B \Rightarrow A)) \Leftrightarrow (A \Leftrightarrow B)$
(d) $(A \vee B) \wedge (\neg B \vee C) \Rightarrow (A \vee C)$

Exercise 2.3 Transform the following formulas into conjunctive normal form:
(a) $A \Leftrightarrow B$
(b) $A \wedge B \Leftrightarrow A \vee B$
(c) $A \wedge (A \Rightarrow B) \Rightarrow B$

Exercise 2.4 Check the following statements for satisfiability or validity.
(a) (play_lottery $\wedge$ six_right) $\Rightarrow$ winner
(b) (play_lottery $\wedge$ six_right $\wedge$ (six_right $\Rightarrow$ win)) $\Rightarrow$ win
(c) $\neg$ ($\neg$ gas_in_tank $\wedge$ (gas_in_tank $\vee\neg$ car_starts) $\Rightarrow$ $\neg$ car_starts)

Exercise 2.5 Using the programming language of your choice, program a theorem prover for propositional logic using the truth table method for formulas in conjunctive normal form. To avoid a costly syntax check of the formulas, you may represent clauses as lists or sets of literals, and the formulas as lists or sets of clauses. The program should indicate whether the formula is unsatisfiable, satisfiable, or true, and output the number of different interpretations and models.

Exercise 2.6
(a) Show that modus ponens is a valid inference rule by showing that $A \wedge (A \Rightarrow B) \models B$.
(b) Show that the resolution rule (2.1) is a valid inference rule.

- Exercise 2.7 Show by application of the resolution rule that, in conjunctive normal form, the empty clause is equivalent to the false statement.

- Exercise 2.8 Show that, with resolution, one can ”derive” any arbitrary clause from a knowledge base that contains a contradiction.

Exercise 2.9 Formalize the following logical functions with the logical operators and show that your formula is valid. Present the result in CNF.
(a) The XOR operation (exclusive or) between two variables.
(b) The statement at least two of the three variables A, B, C are true.

- Exercise 2.10 Solve the following case with the help of a resolution proof: ”If the criminal had an accomplice, then he came in a car. The criminal had no accomplice and did not have the key, or he had the key and an accomplice. The criminal had the key. Did the criminal come in a car or not?”

Exercise 2.11 Show by resolution that the formula from
(a) Exercise 2.2(d) is a tautology.
(b) Exercise 2.4(c) is unsatisfiable.
Exercise 2.12 Prove the following equivalences, which are important for working with Horn clauses:
(a) $(\neg A_1 \vee \cdots \vee \neg A_m \vee B) \equiv A_1 \wedge \cdots \wedge A_m \Rightarrow B$
(b) $(\neg A_1 \vee \cdots \vee \neg A_m) \equiv A_1 \wedge \cdots \wedge A_m \Rightarrow f$
(c) $A \equiv w \Rightarrow A$
Exercise 2.13 Show by SLD resolution that the following Horn clause set is unsatisfiable.

$$\begin{array}{lll} (A)_1 & (D)_4 & (A \wedge D \Rightarrow G)_7 \\ (B)_2 & (E)_5 & (C \wedge F \wedge E \Rightarrow H)_8 \\ (C)_3 & (A \wedge B \wedge C \Rightarrow F)_6 & (H \Rightarrow f)_9 \end{array}$$

$\rightarrow$ Exercise 2.14 In Sect. 2.6 it says: ”Thus it is clear that there is probably (modulo the P/NP problem) no polynomial algorithm for 3-SAT, and thus probably not a general one either.” Justify the ”probably” in this sentence.

0.6.9 First-order Predicate Logic

Many practical, relevant problems cannot be or can only very inconveniently be formulated in the language of propositional logic, as we can easily recognize in the following example. The statement

“Robot 7 is situated at the xy position (35, 79)”

can in fact be directly used as the propositional logic variable

```
"Robot_7_is_situated_at_xy_position_(35, 79)"
```

for reasoning with propositional logic, but reasoning with this kind of proposition is very inconvenient. Assume 100 of these robots can stop anywhere on a grid of 100×100 points. To describe every position of every robot, we would need $100 \cdot 100 \cdot 100 = 1000000 = 10^6$ different variables. The definition of relationships between objects (here robots) becomes truly difficult. The relation

“Robot A is to the right of robot B.”

is semantically nothing more than a set of pairs. Of the 10000 possible pairs of x-coordinates there are $(99 \cdot 98)/2 = 4851$ ordered pairs. Together with all 10000 combinations of possible y-values for both robots, there are $(100 \cdot 99) = 9900$ formulas of the type

Robot_7_is_to_the_right_of_robot_12 $\Leftrightarrow$ Robot_7_is_situated_-at_xy_position_(35, 79) $\wedge$ Robot_12_is_situated_at_xy_position_-(10, 93) $\vee \ldots$

defining these relations, each of them with $\left(10^4\right)^2 \cdot 0.485 = 0.485 \cdot 10^8$ alternatives on the right side. In first-order predicate logic, we can define a predicate

Position(number, xPosition, yPosition). The above relation must no longer be enumerated as a huge number of pairs, rather it is described abstractly with a rule of the form

$\forall u \forall v$ is_further_right $(u, v) \Leftrightarrow \exists x_u \exists y_u \exists x_v \exists y_v$ position $(u, x_u, y_u) \wedge$ position $(v, x_v, y_v) \wedge x_u > x_v$,

Where $\forall u$ is read as "for every u" and $\exists v$ as "there exists v".
In this chapter we will define the syntax and semantics of first-order predicate logic (PL1), show that many applications can be modeled with this language and that there is a complete and sound calculus for this language.

0.6.10 Syntax

First we solidify the syntactic structure of terms.

Definition 3.1 Let V be a set of variables, K a set of constants, and F a set of function symbols. The sets V, K and F are pairwise disjoint. We define the set of terms recursively:

- All variables and constants are (atomic) terms.
- If $t_1, \ldots, t_n$ are terms and f an n-place function symbol, then $f\left(t_1, \ldots, t_n\right)$ is also a term.

Some examples of terms are $f(\sin(\ln(3)), \exp(x))$ and $g(g(g(x)))$. To be able to establish logical relationships between terms, we build formulas from terms.

Definition 3.2 Let P be a set of predicate symbols. Predicate logic formulas are built as follows:

- If $t_1, \ldots, t_n$ are terms and p an n-place predicate symbol, then $p\left(t_1, \ldots, t_n\right)$ is an (atomic) formula.
- If A and B are formulas, then $\neg A, (A), A \wedge B, A \vee B, A \Rightarrow B, A \Leftrightarrow B$ are also formulas.
- If x is a variable and A a formula, then $\forall x A$ and $\exists x A$ are also formulas. $\forall$ is the universal quantifier and $\exists$ the existential quantifier.
- $p\left(t_1, \ldots, t_n\right)$ and $\neg p\left(t_1, \ldots, t_n\right)$ are called literals.

Table 3.1 Examples of formulas in first-order predicate logic. Please note that mother here is a function symbol

Formula	Description
$\forall x\,(\text{frog}(x) \Rightarrow \text{green}(x))$	All frogs are green
$\forall x\,((\text{frog}(x) \wedge \text{brown}(x)) \Rightarrow \text{big}(x))$	All brown frogs are big
$\forall x\ \text{likes}(x, \text{cake})$	Everyone likes cake
$\neg\forall x\ \text{likes}(x, \text{cake})$	Not everyone likes cake
$\neg\exists x\ \text{likes}(x, \text{cake})$	No one likes cake
$\exists x\,\forall y\ \text{likes}(y, x)$	There is something that everyone likes
$\exists x\,\forall y\ \text{likes}(x, y)$	There is someone who likes everything
$\forall x\,\exists y\ \text{likes}(y, x)$	Everything is loved by someone
$\forall x\,\exists y\ \text{likes}(x, y)$	Everyone likes something
$\forall x\,(\text{customer}(x) \Rightarrow \text{likes}(\text{bob}, x))$	Bob likes every customer
$\exists x\,(\text{customer}(x) \wedge \text{likes}(x, \text{bob}))$	There is a customer whom bob likes
$\exists x\,\big(\text{baker}(x) \wedge \forall y\,(\text{customer}(y) \Rightarrow \text{likes}(x, y))\big)$	There is a baker who likes all of his customers
$\forall x\ \text{older}(\text{mother}(x), x)$	Every mother is older than her child
$\forall x\ \text{older}(\text{mother}(\text{mother}(x)), x)$	Every grandmother is older than her daughter's child
$\forall x\,\forall y\,\forall z\,((\text{rel}(x, y) \wedge \text{rel}(y, z)) \Rightarrow \text{rel}(x, z))$	rel is a transitive relation

- Formulas in which every variable is in the scope of a quantifier are called first-order sentences or closed formulas. Variables which are not in the scope of a quantifier are called free variables.
- Definitions 2.8 (CNF) and 2.10 (Horn clauses) hold for formulas of predicate logic literals analogously.

In Table 3.1 several examples of PL1 formulas are given along with their intuitive interpretations.

0.6.11 Semantics

In propositional logic, every variable is directly assigned a truth value by an interpretation. In predicate logic, the meaning of formulas is recursively defined over the construction of the formula, in that we first assign constants, variables, and function symbols to objects in the real world.

Definition 3.3 An interpretation II is defined as

- A mapping from the set of constants and variables $K \cup V$ to a set W of names of objects in the world.
- A mapping from the set of function symbols to the set of functions in the world. Every n-place function symbol is assigned an n-place function.
- A mapping from the set of predicate symbols to the set of relations in the world. Every n-place predicate symbol is assigned an n-place relation.

Example 3.1 Let c_1, c_2, c_3 be constants, "plus" a two-place function symbol, and "gr" a two-place predicate symbol. The truth of the formula

$$F \equiv \mathrm{gr}\left(plus\left(c_1, c_3\right), c_2\right)$$

depends on the interpretation $\mathbb{I}$. We first choose the following obvious interpretation of constants, the function, and of the predicates in the natural numbers:

$$\mathbb{I}_1 : c_1 \mapsto 1, c_2 \mapsto 2, c_3 \mapsto 3, \quad \text{plus} \mapsto +, \quad gr \mapsto > .$$

Thus the formula is mapped to

$$1 + 3 > 2, \quad \text{or after evaluation} \quad 4 > 2.$$

The greater-than relation on the set $\{1, 2, 3, 4\}$ is the set of all pairs (x, y) of numbers with $x > y$, meaning the set $G = \{(4, 3), (4, 2), (4, 1), (3, 2), (3, 1), (2, 1)\}$. Because $(4, 2) \in G$, the formula F is true under the interpretation $\mathbb{I}_1$. However, if we choose the interpretation

$$\mathbb{I}_2 : c_1 \mapsto 2, c_2 \mapsto 3, c_3 \mapsto 1, \quad \text{plus} \mapsto -, \quad gr \mapsto >,$$

we obtain

$$2 - 1 > 3, \text{ or } 1 > 3.$$

The pair $(1, 3)$ is not a member of G. The formula F is false under the interpretation $\mathbb{I}_2$. Obviously, the truth of a formula in PL1 depends on the interpretation. Now, after this preview, we define truth.

0.7 Definition

- An atomic formula $p\left(t_1, \ldots, t_n\right)$ is true (or valid) under the interpretation $\mathbb{I}$ if, after interpretation and evaluation of all terms $t_1, \ldots, t_n$ and interpretation of the predicate p through the n-place relation r, it holds that

$$\left(\mathbb{I}\left(t_1\right), \ldots, \mathbb{I}\left(t_n\right)\right) \in r.$$

- The truth of quantifierless formulas follows from the truth of atomic formulas-as in propositional calculus-through the semantics of the logical operators defined in Table 2.1 on page 25.

- A formula $\forall x F$ is true under the interpretation $\mathbb{I}$ exactly when it is true given an arbitrary change of the interpretation for the variable x (and only for x)

- A formula $\exists x F$ is true under the interpretation $\mathbb{I}$ exactly when there is an interpretation for x which makes the formula true.
  The definitions of semantic equivalence of formulas, for the concepts satisfiable, true, unsatisfiable, and model, along with semantic entailment (Definitions 2.4, 2.5, 2.6) carry over unchanged from propositional calculus to predicate logic.

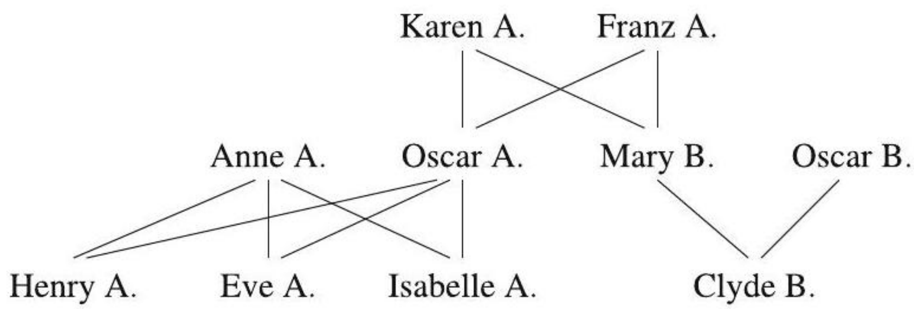


Figure 8: A family tree. The edges going from Clyde B. upward to Mary B. and Oscar B. represent the element (Clyde B., Mary B., Oscar B.) as a child relationship

Theorem 3.1 Theorems 2.2 (deduction theorem) and 2.3 (proof by contradiction) hold analogously for PL1.

Example 3.2 The family tree given in Fig. 3.1 graphically represents (in the semantic level) the relation

$$
\begin{array}{ll}
 & (\ \text{Oscar A., Karen A., Frank A. }), \\
\text{Child} = & (\text{ Henry A., Anne A., Oscar A. }), \\
 & (\text{ Isabelle A., Anne A., Oscar A. }), \\
 & (\text{ Mary B., Karen A., Frank A. }), \\
 & (\text{ Eve A., Anne A., Oscar A. }), \\
 & (\text{ Clyde B., Mary B., Oscar B. })\}
\end{array}
$$

For example, the triple (Oscar A., Karen A., Frank A.) stands for the proposition "Oscar A. is a child of Karen A. and Frank A." From the names we read off the one-place relation

$$\text{Female} = \{\text{Karen A., Anne A., Mary B., Eve A., Isabelle A.}\}$$

of the women. We now want to establish formulas for family relationships. First we define a three-place predicate child(x, y, z) with the semantic

$$\mathbb{I}(\text{child}(x, y, z)) = w \equiv (\mathbb{I}(x), \mathbb{I}(y), \mathbb{I}(z)) \in \text{Kind}\,.$$

Under the interpretation $\mathbb{I}(\text{oscar}) = \text{Oscar A.}$, $\mathbb{I}(\text{eve}) = \text{Eve A.}$, $\mathbb{I}(\text{anne}) = \text{Anne A.}$, it is also true that child(eve, anne, oscar). For child(eve, oscar, anne) to be true, we require, with

$$\forall x \forall y \forall z \ \text{child}(x, y, z) \Leftrightarrow \text{child}(x, z, y),$$

symmetry of the predicate child in the last two arguments. For further definitions we refer to Exercise 3.1 on page 63 and define the predicate descendant recursively as

$$\forall x \forall y \ \text{descendant}(x, y) \Leftrightarrow \exists z \ \ \text{child}(x, y, z) \lor \\ \big(\exists u \exists v \ \ \text{child}(x, u, v) \land \text{descendant}(u, y)\big).$$

Now we build a small knowledge base with rules and facts. Let

$$\begin{aligned} KB \equiv\ & \text{female}(\text{karen}) \land \text{female}(\text{anne}) \land \text{female}(\text{mary}) \\ & \land \text{female}(\text{eve}) \land \text{female}(\text{isabelle}) \\ & \land \text{child}(\text{oscar}, \text{karen}, \text{franz}) \land \text{child}(\text{mary}, \text{karen}, \text{franz}) \\ & \land \text{child}(\text{eve}, \text{anne}, \text{oscar}) \land \text{child}(\text{henry}, \text{anne}, \text{oscar}) \\ & \land \text{child}(\text{isabelle}, \text{anne}, \text{oscar}) \land \text{child}(\text{clyde}, \text{mary}, \text{oscar}) \\ & \land \big(\forall x \forall y \forall z \ \text{child}(x, y, z) \Rightarrow \text{child}(x, z, y)\big) \\ & \land \big(\forall x \forall y \ \text{descendant}(x, y) \Leftrightarrow \exists z \ \text{child}(x, y, z) \\ & \qquad \lor \ (\exists u \exists v \ \text{child}(x, u, v) \land \text{descendant}(u, y))\big) \end{aligned}$$

We can now ask, for example, whether the propositions child(eve, oscar, anne) or descendant(eve, franz) are derivable. To that end we require a calculus.

0.7.1 Equality

To be able to compare terms, equality is a very important relation in predicate logic. The equality of terms in mathematics is an equivalence relation, meaning it is reflexive, symmetric and transitive. If we want to use equality in formulas, we must either incorporate these three attributes as axioms in our knowledge base, or we must integrate equality into the calculus. We take the easy way and define a predicate "=" which, deviating from Definition 3.2 on page 40, is written using infix notation as is customary in mathematics. (An equation $x = y$ could of course also be written in the form $eq(x, y)$.) Thus, the equality axioms have the form

$$\begin{array}{rll} \forall x & x = x & \text{(reflexivity)} \\ \forall x \forall y & x = y \Rightarrow y = x & \text{(symmetry)} \\ \forall x \forall y \forall z & x = y \land y = z \Rightarrow x = z & \text{(transitivity).} \end{array} \tag{3.1}$$

To guarantee the uniqueness of functions, we additionally require

$$\forall x \forall y \ x = y \Rightarrow f(x) = f(y) \quad \text{(substitution axiom)} \tag{3.2}$$

for every function symbol. Analogously we require for all predicate symbols

$$\forall x \forall y \ x = y \Rightarrow p(x) \Leftrightarrow p(y) \quad \text{(substitution axiom).} \tag{3.3}$$

We formulate other mathematical relations, such as the "<" relation, by similar means (Exercise 3.4 on page 64).

Often a variable must be replaced by a term. To carry this out correctly and describe it simply, we give the following definition.

Definition 3.5 We write $\varphi[x/t]$ for the formula that results when we replace every free occurrence of the variable x in φ with the term t. Thereby we do not allow any variables in the term t that are quantified in φ. In those cases variables must be renamed to ensure this.

Example 3.3 If, in the formula $\forall x\ x = y$, the free variable y is replaced by the term $x + 1$, the result is $\forall x\ x = x + 1$. With correct substitution we obtain the formula $\forall x\ x = y + 1$, which has a very different semantic.

0.7.2 Quantifiers and Normal Forms

By Definition 3.4 on page 43, the formula $\forall x\ p(x)$ is true if and only if it is true for all interpretations of the variable x. Instead of the quantifier, one could write $p(a_1) \wedge \cdots \wedge p(a_n)$ for all constants $a_1 \cdots a_n$ in K. For $\exists x\ p(x)$ one could write $p(a_1) \vee \cdots \vee p(a_n)$. From this it follows with de Morgan's law that

$$\forall x\ \varphi \equiv \neg \exists x\ \neg \varphi.$$

Through this equivalence, universal, and existential quantifiers are mutually replaceable. Example 3.4 The proposition ”Everyone wants to be loved” is equivalent to the proposition ”Nobody does not want to be loved”.

Quantifiers are an important component of predicate logic's expressive power. However, they are disruptive for automatic inference in AI because they make the structure of formulas more complex and increase the number of applicable inference rules in every step of a proof. Therefore our next goal is to find, for every predicate logic formula, an equivalent formula in a standardized normal form with as few quantifiers as possible. As a first step we bring universal quantifiers to the beginning of the formula and thus define

Definition 3.6 A predicate logic formula φ is in prenex normal form if it holds that

- $\varphi = Q_1 x_1 \cdots Q_n x_n \psi$.
- ψ is a quantifierless formula.
- $Q_i \in \{\forall, \exists\}$ for $i = 1, \ldots, n$.

Caution is advised if a quantified variable appears outside the scope of its quantifier, as for example x in

$$\forall x p(x) \Rightarrow \exists x q(x)$$

Here one of the two variables must be renamed, and in

$$\forall x p(x) \Rightarrow \exists y q(y)$$

the quantifier can easily be brought to the front, and we obtain as output the equivalent formula

$$\forall x \exists y p(x) \Rightarrow q(y)$$

If, however, we wish to correctly bring the quantifier to the front of

$$(\forall x p(x)) \Rightarrow \exists y q(y) \tag{3.4}$$

we first write the formula in the equivalent form

$$\neg(\forall x p(x)) \vee \exists y q(y).$$

The first universal quantifier now turns into

$$(\exists x \neg p(x)) \vee \exists y q(y)$$

and now the two quantifiers can finally be pulled forward to

$$\exists x \exists y \neg p(x) \vee q(y),$$

which is equivalent to

$$\exists x \exists y p(x) \Rightarrow q(y)$$

We see then that in (3.4) on page 46 we cannot simply pull both quantifiers to the front. Rather, we must first eliminate the implications so that there are no negations on the quantifiers. It holds in general that we may only pull quantifiers out if negations only exist directly on atomic sub-formulas.

Example 3.5 As is well known in analysis, convergence of a series $(a_n)_{n\in\mathbb{N}}$ to a limit a is defined by

$$\forall \varepsilon > 0 \exists n_0 \in \mathbb{N} \forall n > n_0 \, |a_n - a| < \varepsilon$$

With the function abs(x) for $|x|$, $a(n)$ for a_n, minus(x, y) for $x - y$ and the predicates $el(x, y)$ for $x \in y$, $gr(x, y)$ for $x > y$, the formula reads

$$\forall \varepsilon \left(gr(\varepsilon, 0) \Rightarrow \exists n_0 \left(el\left(n_0, \mathbb{N}\right) \Rightarrow \forall n \left(gr\left(n, n_0\right) \Rightarrow gr(\varepsilon, \text{abs}(\text{minus}(a(n), a)))\right)\right)\right) \tag{3.5}$$

This is clearly not in prenex normal form. Because the variables of the inner quantifiers $\exists n_0$ and $\forall n$ do not occur to the left of their respective quantifiers, no variables must be renamed. Next we eliminate the implications and obtain

$$\forall \varepsilon \left(\neg gr(\varepsilon, 0) \vee \exists n_0 \left(\neg el\left(n_0, \mathbb{N}\right) \vee \forall n \left(\neg gr\left(n, n_0\right) \vee gr(\varepsilon, \text{abs}(\text{minus}(a(n), a)))\right)\right)\right).$$

Because every negation is in front of an atomic formula, we bring the quantifiers forward, eliminate the redundant parentheses, and with

$$\forall \varepsilon \exists n_0 \forall n \left(\neg gr(\varepsilon, 0) \vee \neg el\left(n_0, \mathbb{N}\right) \vee \neg gr\left(n, n_0\right) \vee gr(\varepsilon, \text{abs}(\text{minus}(a(n), a)))\right)$$

it becomes a quantified clause in conjunctive normal form.

The transformed formula is equivalent to the output formula. The fact that this transformation is always possible is guaranteed by

Theorem 3.2 Every predicate logic formula can be transformed into an equivalent formula in prenex normal form.

In addition, we can eliminate all existential quantifiers. However, the formula resulting from the so-called Skolemization is no longer equivalent to the output formula. Its satisfiability, however, remains unchanged. In many cases, especially when one wants to show the unsatisfiability of $KB \wedge \neg Q$, this is sufficient. The following formula in prenex normal form will now be skolemized:

$$\forall x_1 \forall x_2 \exists y_1 \forall x_3 \exists y_2 p\left(f\left(x_1\right), x_2, y_1\right) \vee q\left(y_1, x_3, y_2\right).$$

Because the variable y_1 apparently depends on x_1 and x_2, every occurrence of y_1 is replaced by a Skolem function $g\left(x_1, x_2\right)$. It is important that g is a new function symbol that has not yet appeared in the formula. We obtain

$$\forall x_1 \forall x_2 \forall x_3 \exists y_2 p\left(f\left(x_1\right), x_2, g\left(x_1, x_2\right)\right) \vee q\left(g\left(x_1, x_2\right), x_3, y_2\right)$$

and replace y_2 analogously by $h\left(x_1, x_2, x_3\right)$, which leads to

$$\forall x_1 \forall x_2 \forall x_3 p\left(f\left(x_1\right), x_2, g\left(x_1, x_2\right)\right) \vee q\left(g\left(x_1, x_2\right), x_3, h\left(x_1, x_2, x_3\right)\right).$$

Because now all the variables are universally quantified, the universal quantifiers can be left out, resulting in

$$p\left(f\left(x_1\right), x_2, g\left(x_1, x_2\right)\right) \vee q\left(g\left(x_1, x_2\right), x_3, h\left(x_1, x_2, x_3\right)\right)$$

Now we can eliminate the existential quantifier (and thereby also the universal quantifier) in (3.5) on page 47 by introducing the Skolem function $n_0(\varepsilon)$. The skolemized prenex and conjunctive normal form of (3.5) on page 47 thus reads

$$\neg gr(\varepsilon, 0) \vee \neg el\left(n_0(\varepsilon), \mathbb{N}\right) \vee \neg gr\left(n, n_0(\varepsilon)\right) \vee gr(\varepsilon, \text{abs}(\text{minus}(a(n), a))).$$

By dropping the variable n_0, the Skolem function can receive the name n_0. When skolemizing a formula in prenex normal form, all existential quantifiers are eliminated from the outside inward, where a formula of the form $\forall x_1 \ldots \forall x_n \exists y \varphi$ is replaced by $\forall x_1 \ldots \forall x_n \varphi\left[y / f\left(x_1, \ldots, x_n\right)\right]$, during which f may not appear in φ. If an existential quantifier is on the far outside, such as in $\exists y p(y)$, then y must be replaced by a constant (that is, by a zero-place function symbol).

The procedure for transforming a formula in conjunctive normal form is summarized in the pseudocode represented in Fig. 3.2. Skolemization has polynomial runtime in the number of literals. When transforming into normal form, the number of literals in the normal form can grow exponentially, which can lead to exponential computation time and exponential memory usage. The

NORMALFORMTRANSFORMATION(*Formula*):
1. *Transformation into prenex normal form*:
    Transformation into conjunctive normal form (Theorem 2.1):
        Elimination of equivalences.
        Elimination of implications.
        Repeated application of de Morgan's law and distributive law.
    Renaming of variables if necessary.
    Factoring out universal quantifiers.
2. *Skolemization*:
    Replacement of existentially quantified variables by new Skolem functions.
    Deletion of resulting universal quantifiers.

Figure 9: Transformation of predicate logic formulas into normal form

reason for this is the repeated application of the distributive law. The actual problem, which results from a large number of clauses, is the combinatorial explosion of the search space for a subsequent resolution proof. However, there is an optimized transformation algorithm which only spawns polynomially many literals [**?**].

0.7.3 Proof Calculi

For reasoning in predicate logic, various calculi of natural reasoning such as Gentzen calculus or sequent calculus, have been developed. As the name suggests, these calculi are meant to be applied by humans, since the inference rules are more or less intuitive and the calculi work on arbitrary PL1 formulas. In the next section we will primarily concentrate on the resolution calculus, which is in practice the most important efficient, automatizable calculus for formulas in conjunctive normal form. Here, using Example 3.2 on page 43 we will give a very small "natural" proof. We use the inference rule

$$\frac{A, \quad A \Rightarrow B}{B} \quad \text{(modus ponens, MP) and} \quad \frac{\forall x\, A}{A[x/t]} \quad (\forall\text{-elimination}, \forall E).$$

The modus ponens is already familiar from propositional logic. When eliminating universal quantifiers one must keep in mind that the quantified variable x must be replaced by a ground term t, meaning a term that contains no variables. The proof of child(eve, oscar, anne) from an appropriately reduced knowledge base is presented in Table 3.2.

The two formulas of the reduced knowledge base are listed in rows 1 and 2.

Table 1: Simple proof with modus ponens and quantifier elimination

WB:	1	child(eve, anne, oscar)
WB:	2	$\forall x \, \forall y \, \forall z \, \big(child(x, y, z) \Rightarrow child(x, z, y)\big)$
$\forall E(2)$: x/eve, y/anne, z/oscar	3	child(eve, anne, oscar) $\Rightarrow$ child(eve, oscar, anne)
MP(1,3)	4	child(eve, oscar, anne)

In row 3 the universal quantifiers from row 2 are eliminated, and in row 4 the claim is derived with modus ponens.

The calculus consisting of the two given inference rules is not complete. However, it can be extended into a complete procedure by addition of further inference rules. This nontrivial fact is of fundamental importance for mathematics and AI. The Austrian logician Kurt Gödel proved in 1931 that [Göd31a].

Theorem 3.3 (Gödel's completeness theorem) First-order predicate logic is complete. That is, there is a calculus with which every proposition that is a consequence of a knowledge base KB can be proved. If $KB \models \varphi$, then it holds that $KB \vdash \varphi$.

Every true proposition in first-order predicate logic is therefore provable. But is the reverse also true? Is everything we can derive syntactically actually true? The answer is "yes":

Theorem 3.4 (Correctness) There are calculi with which only true propositions can be proved. That is, if $KB \vdash \varphi$ holds, then $KB \models \varphi$.

In fact, nearly all known calculi are correct. After all, it makes little sense to work with incorrect proof methods. Provability and semantic consequence are therefore equivalent concepts, as long as correct and complete calculus is being used. Thereby first-order predicate logic becomes a powerful tool for mathematics and AI. The aforementioned calculi of natural deduction are rather unsuited for automatization. Only resolution calculus, which was introduced in 1965 and essentially works with only one simple inference rule, enabled the construction of powerful automated theorem provers, which later were employed as inference machines for expert systems.

0.7.4 Resolution

Indeed, the correct and complete resolution calculus triggered a logic euphoria during the 1970s. Many scientists believed that one could formulate almost every task of knowledge representation and reasoning in PL1 and then solve it with an automated prover. Predicate logic, a powerful, expressive language, together with a complete proof calculus seemed to be the universal intelligent machine for representing knowledge and solving many difficult problems (Fig. 3.3).

If one feeds a set of axioms (that is, a knowledge base) and a query into such a logic machine as input, the machine searches for a proof and returns it-for one exists and will be found-as output. With Gödel's completeness theorem and the work of Herbrand as a foundation, much was invested into the mechanization of logic. The vision of a machine that could, with an arbitrary non-contradictory PL1 knowledge base, prove any true query was very enticing. Accordingly, until now many proof calculi for PL1 are being developed and realized in the form of theorem provers. As an example, here we describe the historically important and widely used resolution calculus and show its capabilities. The reason for selecting resolution as an example of a proof calculus in this book is, as stated, its historical and

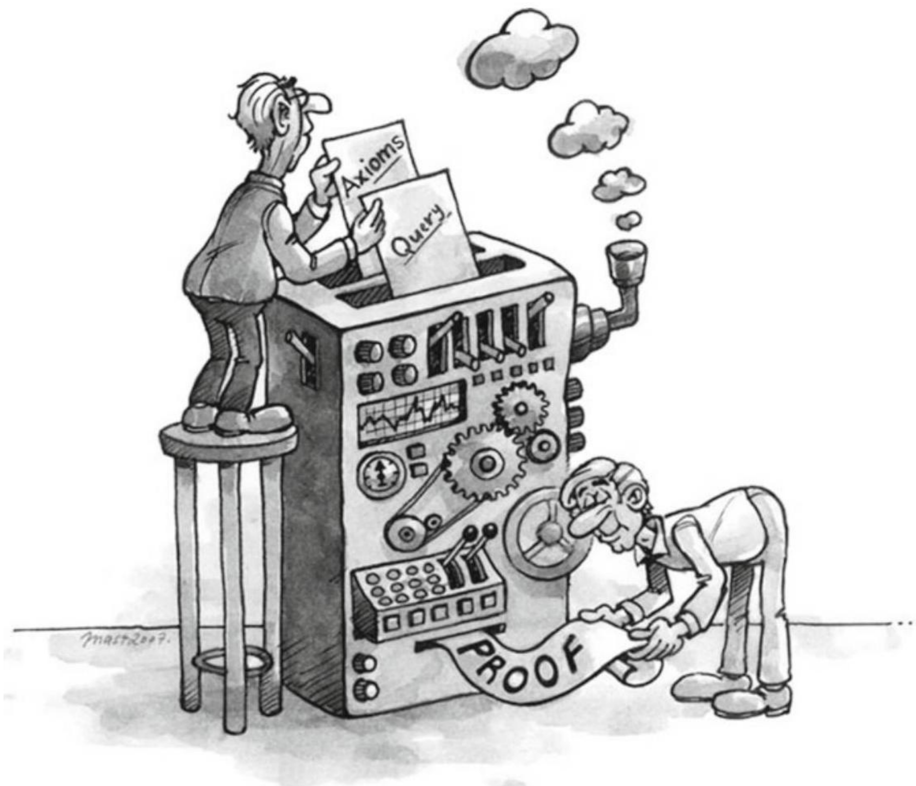


Figure 10: The universal logic machine

didactic importance. Today, resolution represents just one of many calculi used in high-performance provers. We begin by trying to compile the proof in Table 3.2 on page 50 with the knowledge base of Example 3.2 on page 43 into a resolution proof. First the formulas are transformed into conjunctive normal form and the negated query

$$\neg Q \equiv \neg \text{ child(eve, oscar, anne)}$$

is added to the knowledge base, which gives

$$\begin{aligned} KB \wedge \neg Q \equiv &(\text{ child (eve , anne , oscar)})_1 \wedge \\ &(\neg \text{ child } (x, y, z) \vee \text{ child } (x, z, y))_2 \wedge \\ &(\neg \text{ child (eve , oscar , anne)})_3. \end{aligned}$$

The proof could then look something like

$$\begin{aligned} x/\text{ eve }, y/\text{ anne }, z/\text{ oscar } :&(\neg \text{ child (eve , anne , oscar)} \vee \\ &\text{child (eve , oscar , anne)})_4 \\ \text{Res}(3,4) :&(\neg \text{ child (eve , anne , oscar)})_5 \\ \text{Res}(1,5) :&()_6, \end{aligned} \tag{2}$$

where, in the first step, the variables x, y, z are replaced by constants. Then two resolution steps follow under application of the general resolution rule from (2.2), which was taken unchanged from propositional logic.

The circumstances in the following example are somewhat more complex. We assume that everyone knows his own mother and ask whether Henry knows anyone. With the function symbol "mother" and the predicate "knows", we have to derive a contradiction from

$$(\text{ knows } (x, \text{ mother } (x)))_1 \wedge (\neg \text{ knows (henry }, y))_2.$$

By the replacement x /henry, y /mother (henry) we obtain the contradictory clause pair

$$(\text{ knows (henry , mother (henry)))}_1 \wedge (\neg \text{ knows (henry , mother (henry)))}_2.$$

This replacement step is called unification. The two literals are complementary, which means that they are the same other than their signs. The empty clause is now derivable with a resolution step, by which it has been shown that Henry does know someone (his mother). We define

Definition 3.7 Two literals are called unifiable if there is a substitution σ for all variables which makes the literals equal. Such a σ is called a unifier. A unifier is called the most general unifier (MGU) if all other unifiers can be obtained from it by substitution of variables. Example 3.6 We want to unify the literals $p(f(g(x)), y, z)$ and $p(u, u, f(u))$. Several unifiers are

$$\begin{array}{lllll} \sigma_1 : & y/f(g(x)), & z/f(f(g(x))), & u/f(g(x)), & \\ \sigma_2 : & x/h(v), & y/f(g(h(v))), & z/f(f(g(h(v)))), & u/f(g(h(v))) \\ \sigma_3 : & x/h(h(v)), & y/f(g(h(h(v)))), & z/f(f(g(h(h(v))))), & u/f(g(h(h(v)))) \\ \sigma_4 : & x/h(a), & y/f(g(h(a))), & z/f(f(g(h(a)))), & u/f(g(h(a))) \\ \sigma_5 : & x/a, & y/f(g(a)), & z/f(f(g(a))), & u/f(g(a)) \end{array}$$

where σ_1 is the most general unifier. The other unifiers result from σ_1 through the substitutions $x/h(v), x/h(h(v)), x/h(a), x/a$.

We can see in this example that during unification of literals, the predicate symbols can be treated like function symbols. That is, the literal is treated like a term. Implementations of unification algorithms process the arguments of functions sequentially. Terms are unified recursively over the term structure. The simplest unification algorithms are very fast in most cases. In the worst case, however, the computation time can grow exponentially with the size of the terms. Because for automated provers the overwhelming number of unification attempts fail or are very simple, in most cases the worst case complexity has no dramatic effect. The fastest unification algorithms have nearly linear complexity even in the worst case [[**?**]].

We can now give the general resolution rule for predicate logic:

Definition 3.8 The resolution rule for two clauses in conjunctive normal form reads

$$\frac{(A_1 \vee \cdots \vee A_m \vee B)\,, \quad (\neg B' \vee C_1 \vee \cdots \vee C_n) \quad \sigma(B) = \sigma\,(B')}{(\sigma\,(A_1) \vee \cdots \vee \sigma\,(A_m) \vee \sigma\,(C_1) \vee \cdots \vee \sigma\,(C_n))}, \tag{3.6}$$

where σ is the MGU of B and B'.

Theorem 3.5 The resolution rule is correct. That is, the resolvent is a semantic consequence of the two parent clauses.

For Completeness, however, we still need a small addition, as is shown in the following example. Example 3.7 The famous Russell paradox reads "There is a barber who shaves everyone who does not shave himself." This statement is contradictory, meaning it is unsatisfiable. We wish to show this with resolution. Formalized in PL1, the paradox reads

$$\forall x \text{ shaves (barber }, x) \Leftrightarrow \neg \text{shaves}(x, x)$$

and transformation into clause form yields (see Exercise 3.6 on page 64) $(\neg \text{ shaves (barbier }, x) \vee \neg \text{ shaves } (x, x))_1 \wedge (\text{ shaves (barbier }, x) \vee \text{ shaves } (x, x))_2$.

From these two clauses we can derive several tautologies, but no contradiction. Thus resolution is not complete. We need yet a further inference rule.

Definition 3.9 Factorization of a clause is accomplished by

$$\frac{(A_1 \vee A_2 \vee \cdots \vee A_n) \quad \sigma\,(A_1) = \sigma\,(A_2)}{(\sigma\,(A_2) \vee \cdots \vee \sigma\,(A_n))},$$

where σ is the MGU of A_1 and A_2.

Now a contradiction can be derived from (3.7)

$$\begin{aligned} \text{Fak}(1, \sigma : x / \text{ barber }) &: (\neg \text{ shaves (barber , barber)})_3 \\ \text{Fak}(2, \sigma : x / \text{ barber }) &: (\text{ shaves (barber , barber)})_4 \\ \text{Res}(3, 4) &: ()_5 \end{aligned}$$

and we assert:

Theorem 3.6 The resolution rule (3.6) together with the factorization rule (3.9) is refutation complete. That is, by application of factorization and resolution steps, the empty clause can be derived from any unsatisfiable formula in conjunctive normal form.

0.7.5 Resolution Strategies

While completeness of resolution is important for the user, the search for a proof can be very frustrating in practice. The reason for this is the immense combinatorial search space. Even if there are only very few pairs of clauses in $KB \wedge \neg Q$ in the beginning, the prover generates a new clause with every resolution step, which increases the number of possible resolution steps in the next iteration. Thus it has long been attempted to reduce the search space using special strategies, preferably without losing completeness. The most important strategies are the following. Unit resolution prioritizes resolution steps in which one of the two clauses consists of only one literal, called a unit clause. This strategy preserves completeness and leads in many cases, but not always, to a reduction of the search space. It therefore is a heuristic process (see Sect. 6.3).

One obtains a guaranteed reduction of the search space by application of the set of support strategy. Here a subset of $KB \wedge \neg Q$ is defined as the set of support (SOS). Every resolution step must involve a clause from the SOS, and the resolvent is added to the SOS. This strategy is incomplete. It becomes complete when it is ensured that the set of clauses is satisfiable without the SOS (see Exercise 3.7 on page 64). The negated query $\neg Q$ is often used as the initial SOS.

In input resolution, a clause from the input set $KB \wedge \neg Q$ must be involved in every resolution step. This strategy also reduces the search space, but at the cost of completeness.

With the pure literal rule all clauses that contain literals for which there are no complementary literals in other clauses can be deleted. This rule reduces the search space and is complete, and therefore it is used by practically all resolution provers.

If the literals of a clause K_1 represent a subset of the literals of the clause K_2, then K_2 can be deleted. For example, the clause

$$\big(\text{raining(today)} \Rightarrow \text{street\_wet(today)}\big)$$

is redundant if street_wet(today) is already valid. This important reduction step is called subsumption. Subsumption, too, is complete.

0.7.6 Equality

Equality is an especially inconvenient cause of explosive growth of the search space. If we add (3.1) on page 45 and the equality axioms formulated in (3.2) on page 45 to the knowledge base, then the symmetry clause $\neg x = y \vee y = x$ can be unified with every positive or negated equation, for example. This leads to the derivation of new clauses and equations upon which equality axioms can again be applied, and so on. The transitivity and substitution axioms have similar consequences. Because of this, special inference rules for equality have been developed which get by without explicit equality axioms and, in particular, reduce the search space. Demodulation, for example, allows substitution of a term t_2 for t_1, if the equation $t_1 = t_2$ exists. An equation $t_1 = t_2$ is applied by means of unification to a term t as follows:

$$\frac{t_1 = t_2, \quad (\ldots t \ldots), \sigma(t_1) = \sigma(t)}{(\ldots \sigma(t_2) \ldots)}.$$

Somewhat more general is paramodulation, which works with conditional equations [Bib82, [**?**]].

The equation $t_1 = t_2$ allows the substitution of the term t_1 by t_2 as well as the substitution t_2 by t_1. It is usually pointless to reverse a substitution that has already been carried out. On the contrary, equations are frequently used to simplify terms. They are thus often used in one direction only. Equations which are only used in one direction are called directed equations. Efficient processing of directed equations is accomplished by so-called term rewriting systems. For formulas with many equations there exist special equality provers.

0.7.7 Automated Theorem Provers

Implementations of proof calculi on computers are called theorem provers. Along with specialized provers for subsets of PL1 or special applications, there exist today a whole line of automated provers for the full predicate logic and higher-order logics, of which only a few will be discussed here. An overview of the most important systems can be found in [[**?**]].

One of the oldest resolution provers was developed at the Argonne National Laboratory in Chicago. Based on early developments starting in 1963, Otter [[**?**]], was created in 1984. Above all, Otter was successfully applied in specialized areas of mathematics, as one can learn from its home page:

> ”Currently, the main application of Otter is research in abstract algebra and formal logic. Otter and its predecessors have been used to answer many open questions in the areas of finite semigroups, ternary Boolean algebra, logic calculi, combinatory logic, group theory, lattice theory, and algebraic geometry.”

Several years later the University of Technology, Munich, created the high-performance prover SETHEO [[**?**]] based on fast PROLOG technology. With the goal of reaching even higher performance, an implementation for parallel computers was developed under the name PARTHEO. It turned out that it was not worthwhile to use special hardware in theorem provers, as is also the case in other areas of AI, because these computers are very quickly overtaken by faster processors and more intelligent algorithms. Munich is also the birthplace of

E [[**?**]], an award-winning modern equation prover, which we will become familiar with in the next example. On E's homepage one can read the following compact, ironic characterization, whose second part incidentally applies to all automated provers in existence today.
"E is a purely equational theorem prover for clausal logic. That means it is a program that you can stuff a mathematical specification (in clausal logic with equality) and a hypothesis into, and which will then run forever, using up all of your machines resources. Very occasionally it will find a proof for the hypothesis and tell you so ;-)." Finding proofs for true propositions is apparently so difficult that the search succeeds only extremely rarely, or only after a very long time-if at all. We will go into this in more detail in Chap. 4. Here it should be mentioned, though, that not only computers, but also most people have trouble finding strict formal proofs.

Though evidently computers by themselves are in many cases incapable of finding a proof, the next best thing is to build systems that work semi-automatically and allow close cooperation with the user. Thereby the human can better apply his knowledge of special application domains and perhaps limit the search for the proof. One of the most successful interactive provers for higher-order predicate logic is Isabelle [[**?**]], a common product of Cambridge University and the University of Technology, Munich.

Anyone searching for a high-performance prover should look at the current results of the CASC (CADE ATP System Competition) [[**?**]][2]. Here we find that the winner from 2001 to 2006 in the PL1 and clause normal form categories was Manchester's prover Vampire, which works with a resolution variant and a special approach to equality. The system Waldmeister of the Max Planck Institute in Saarbrücken has been leading for years in equality proving.

The many top positions of German systems at CASC show that German research groups in the area of automated theorem proving are playing a leading role, today as well as in the past.

0.7.8 Mathematical Examples

We now wish to demonstrate the application of an automated prover with the aforementioned prover E [Sch02]. E is a specialized equality prover which greatly shrinks the search space through an optimized treatment of equality.

We want to prove that left- and right-neutral elements in a semigroup are equal. First we formalize the claim step by step.

Definition 3.10 A structure ($M, \cdot$) consisting of a set M with a two-place inner operation "." is called a semigroup if the law of associativity

$$\forall x \forall y \forall z (x \cdot y) \cdot z = x \cdot (y \cdot z)$$

holds. An element $e \in M$ is called left-neutral (right-neutral) if $\forall x\, e \cdot x = x$ ($\forall x\, x \cdot e = x$).

[2]CADE is the annual "Conference on Automated Deduction" [[**?**]] and ATP stands for "Automated Theorem Prover".

It remains to be shown that

Theorem 3.7 If a semigroup has a left-neutral element e_l and a right-neutral element e_r, then $e_l = e_r$. First we prove the theorem semi-formally by intuitive mathematical reasoning. Clearly it holds for all $x \in M$ that

$$e_l \cdot x = x \tag{3.8}$$

and

$$x \cdot e_r = x \tag{3.9}$$

If we set $x = e_r$ in (3.8) and $x = e_l$ in (3.9), we obtain the two equations $e_l \cdot e_r = e_r$ and $e_l \cdot e_r = e_l$. Joining these two equations yields

$$e_l = e_l \cdot e_r = e_r$$

which we want to prove. In the last step, incidentally, we used the fact that equality is symmetric and transitive.

Before we apply the automated prover, we carry out the resolution proof manually. First we formalize the negated query and the knowledge base KB, consisting of the axioms as clauses in conjunctive normal form:

$(\neg e_l = e_r)_1$	negated query
$(m(m(x,y),z) = m(x,m(y,z)))_2$	
$(m\,(e_l,x) = x)_3$	
$(m\,(x,e_r) = x)_4$	
equality axioms:	
$(x = x)_5$	(reflexivity)
$(\neg x = y \vee y = x)_6$	(symmetry)
$(\neg x = y \vee \neg y = z \vee x = z)_7$	(transitivity)
$(\neg x = y \vee m(x,z) = m(y,z))_8$	substitution in m
$(\neg x = y \vee m(z,x) = m(z,y))_9$	substitution in m,

where multiplication is represented by the two-place function symbol m. The equality axioms were formulated analogously to (3.1) on page 45 and (3.2) on page 45. A simple resolution proof has the form

Res $(3, 6, x_6/m\,(e_l, x_3)\,, y_6/x_3)$:	$(x = m\,(e_l, x))_{10}$
Res $(7, 10, x_7/x_{10}, y_7/m\,(e_l, x_{10}))$:	$(\neg m\,(e_l, x) = z \vee x = z)_{11}$
Res $(4, 11, x_4/e_l, x_{11}/e_r, z_{11}/e_l)$:	$(e_r = e_l)_{12}$
Res$(1, 12, \emptyset)$:	$()$.

Here, for example, Res $(3, 6, x_6/m\,(e_l, x_3)\,, y_6/x_3)$ means that in the resolution of clause 3 with clause 6, the variable x from clause 6 is replaced by $m\,(e_l, x_3)$ with variable x from clause 3. Analogously, y from clause 6 is replaced by x from clause 3. Now we want to apply the prover E to the problem. The clauses are transformed into the clause normal form language LOP through the mapping

$$(\neg A_1 \vee \cdots \vee \neg A_m \vee B_1 \vee \cdots \vee B_n) \mapsto \mathrm{B}_1; \ldots; \mathrm{B}_n < -\mathrm{A}_1, \ldots, \mathrm{A}_m.$$

The syntax of LOP represents an extension of the PROLOG syntax (see Chap. 5) for non Horn clauses. Thus we obtain as an input file for E

```
<- eq(el,er). # query
eq( m(m(X,Y),Z), m(X,m(Y,Z)) ). # associativity of m
eq( m(el,X), X ). # left-neutral element of m
eq( m(X,er), X ). # right-neutral element of m
eq(X,X). # equality: reflexivity
eq(Y,X) <- eq(X,Y). # equality: symmetry
eq(X,Z) <- eq(X,Y), eq(Y,Z). # equality: transitivity
eq( m(X,Z), m(Y,Z) ) <- eq(X,Y). # equality: substitution in m
eq( m(Z,X), m(Z,Y) ) <- eq(X,Y). # equality: substitution in m
```

where equality is modeled by the predicate symbol eq. Calling the prover delivers

```
unixprompt> eproof halbgr1.lop
# Problem status determined, constructing proof object
# Evidence for problem status starts
   0 : [--eq(el,er)] : initial
   1 : [++eq(X1,X2),--eq(X2,X1)] : initial
   2 : [++eq(m(el,X1),X1)] : initial
   3 : [++eq(m(X1,er),X1)] : initial
   4 : [++eq(X1,X2),--eq(X1,X3),--eq(X3,X2)] : initial
   5 : [++eq(X1,m(X1,er))] : pm(3,1)
   6 : [++eq(X2,X1),--eq(X2,m(el,X1))] : pm(2,4)
   7 : [++eq(el,er)] : pm(5,6)
   8 : [] : sr(7,0)
   9 : [] : 8 : {proof}
# Evidence for problem status ends
```

Positive literals are identified by ++ and negative literals by –. In lines 0 to 4, marked with initial, the clauses from the input data are listed again. pm (a, b) stands for a resolution step between clause a and clause b. We see that the proof found by E is very similar to the manually created proof. Because we explicitly model the equality by the predicate eq, the particular strengths of E do not come into play. Now we omit the equality axioms and obtain

```
<- el = er. % query
m(m(X,Y),Z) = m(X,m(Y,Z)) . % associativity of m
m(el,X) = X . % left-neutral element of m
m(X,er) = X . % right-neutral element of m
```

as input for the prover.
The proof also becomes more compact. We see in the following output of the prover that the proof consists essentially of a single inference step on the two relevant clauses 1 and 2.

```
unixprompt> eproof halbgr1a.lop
# Problem status determined, constructing proof object
# Evidence for problem status starts
    0 : [--equal(el, er)] : initial
    1 : [++equal(m(el,X1), X1)] : initial
    2 : [++equal(m(X1,er), X1)] : initial
    3 : [++equal(el, er)] : pm(2,1)
    4 : [--equal(el, el)] : rw(0,3)
    5 : [] : cn(4)
    6 : [] : 5 : {proof}
# Evidence for problem status ends
```

The reader might now take a closer look at the capabilities of E (Exercise 3.9 on page 64).

0.7.9 Applications

In mathematics automated theorem provers are used for certain specialized tasks. For example, the important four color theorem of graph theory was first proved in 1976 with the help of a special prover. However, automated provers still play a minor role in mathematics.

On the other hand, in the beginning of AI, predicate logic was of great importance for the development of expert systems in practical applications. Due to its problems modeling uncertainty (see Sect. 4.4), expert systems today are most often developed using other formalisms. Today logic plays an ever more important role in verification tasks. Automatic program verification is currently an important research area between AI and software engineering. Increasingly complex software systems are now taking over tasks of more and more responsibility and security relevance. Here a proof of certain safety characteristics of a program is desirable. Such a proof cannot be brought about through testing of a finished program, for in general it is impossible to apply a program to all possible inputs. This is therefore an ideal domain for general or even specialized inference systems. Among other things, cryptographic protocols are in use today whose security characteristics have been automatically verified [[**?**], [**?**]]. A further challenge for the use of automated provers is the synthesis of software and hardware. To this end, for example, provers should support the software engineer in the generation of programs from specifications.

Software reuse is also of great importance for many programmers today. The programmer looks for a program that takes input data with certain properties and calculates a result with desired properties. A sorting algorithm accepts input data with entries of a certain data type and from these creates a permutation of these entries with the property that every element is less than or equal to the next element. The programmer first formulates a specification of the query in PL1 consisting of two parts. The first part PRE_Q comprises the preconditions, which must hold before the desired program is applied. The second part $POST_Q$ contains the postconditions, which must hold after the desired program is applied.

In the next step a software database must be searched for modules which fulfill these requirements. To check this formally, the database must store a formal description of the preconditions PRE_M and postconditions $POST_M$ for every module M. An assumption about the capabilities of the modules is that the preconditions of the module follow from the preconditions of the query. It must hold that

$$PRE_Q \Rightarrow PRE_M$$

All conditions that are required as a prerequisite for the application of module M must appear as preconditions in the query. If, for example, a module in the database only accepts lists of integers, then lists of integers as input must also appear as preconditions in the query. An additional requirement in the query that, for example, only even numbers appear, does not cause a problem.

Furthermore, it must hold for the postconditions that

$$\mathrm{POST}_M \Rightarrow \mathrm{POST}_Q$$

That is, after application of the module, all attributes that the query requires must be fulfilled. We now show the application of a theorem prover to this task in an example from [Sch01].

Example 3.8 VDM-SL, the Vienna Development Method Specification Language, is often used as a language for the specification of pre- and postconditions. Assume that in the software database the description of a module Rotate is available, which moves the first list element to the end of the list. We are looking for a module Shuffle, which creates an arbitrary permutation of the list. The two specifications read

$$\begin{array}{l}
\textsc{Rotate}(l : \mathit{List})\ l' : \mathit{List} \\
\textbf{pre}\ \mathit{true} \\
\textbf{post} \\
\quad (l = [\,] \Rightarrow l' = [\,]) \wedge \\
\quad (l \neq [\,] \Rightarrow l' = (\mathrm{tail}\, l)\hat{}[\mathrm{head}\, l])
\end{array}$$

$$\begin{array}{l}
\textsc{Shuffle}(x : \mathit{List})\ x' : \mathit{List} \\
\textbf{pre}\ \mathit{true} \\
\textbf{post}\ \forall i : \mathit{Item} \cdot \\
\quad (\exists x_1, x_2 : \mathit{List} \cdot x = x_1\hat{}[i]\hat{}x_2 \Leftrightarrow \\
\quad \exists y_1, y_2 : \mathit{List} \cdot x' = y_1\hat{}[i]\hat{}y_2)
\end{array}$$

Here "ˆ"stands for the concatenation of lists, and "·"separates quantifiers with their variables from the rest of the formula. The functions "head l"and "tail l"choose the first element and the rest from the list, respectively. The specification of shuffle indicates that every list element i that was in the list (x) before the application of shuffle must be in the result (x') after the application, and vice versa. It must now be shown that the formula $(PRE_Q \Rightarrow PRE_M) \wedge (POST_M \Rightarrow POST_Q)$ is a consequence of the knowledge base containing a description of the data type List. The two VDM-SL specifications yield the proof task

$$\begin{aligned}
&\forall l, l', x, x' : \text{List} \cdot (l = x \wedge l' = x' \wedge (w \Rightarrow w)) \wedge \\
&\quad (l = x \wedge l' = x' \wedge ((l = [] \Rightarrow l' = []) \wedge (l \neq [] \Rightarrow l' = (\text{tl}l)^\wedge[\text{hd}l]) \\
&\quad \Rightarrow \forall i : \text{Item} \cdot (\exists x_1, x_2 : \text{List} \cdot x = x_1 \uparrow [i]^\wedge x_2 \Leftrightarrow \exists y_1, y_2 : \text{List} \cdot x' = y_1 \uparrow [i]^\wedge y_2)))
\end{aligned}$$

which can then be proven with the prover SETHEO.

In the coming years the semantic web will likely represent an important application of PL1. The content of the World Wide Web is supposed to become interpretable not only for people, but for machines. To this end web sites are being furnished with a description of their semantics in a formal description language. The search for information in the web will thereby become significantly more effective than today, where essentially only text building blocks are searchable.

Decidable subsets of predicate logic are used as description languages. The development of efficient calculi for reasoning is very important and closely connected to the description languages. A query for a future semantically operating search engine could (informally) read: Where in Switzerland next Sunday at elevations under 2000 meters will there be good weather and optimally prepared ski slopes? To answer such a question, a calculus is required that is capable of working very quickly on large sets of facts and rules. Here, complex nested function terms are less important.

As a basic description framework, the World Wide Web Consortium developed the language RDF (Resource Description Framework). Building on RDF, the significantly more powerful language OWL (Web Ontology Language) allows the description of relations between objects and classes of objects, similarly to PL1 [**?**]. Ontologies are descriptions of relationships between possible objects. A difficulty when building a description of the innumerable websites is the expenditure of work and also checking the correctness of the semantic descriptions. Here machine learning systems for the automatic generation of descriptions can be very helpful. An interesting use of "automatic" generation of semantics in the web was introduced by Luis von Ahn of Carnegie Mellon University [[**?**]]. He developed computer games in which the players, distributed over the network, are supposed to collaboratively describe pictures with key words. Thus the pictures are assigned semantics in a fun way at no cost.

0.7.10 Summary

We have provided the most important foundations, terms, and procedures of predicate logic and we have shown that even one of the most difficult intellectual tasks, namely the proof of mathematical theorems, can be automated. Automated provers can be employed not only in mathematics, but rather, in particular, in verification tasks in computer science. For everyday reasoning, however, predicate logic in most cases is ill-suited. In the next and the following chapters we show its weak points and some interesting modern alternatives. Furthermore, we will show in Chap. 5 that one can program elegantly with logic and its procedural extensions.

Anyone interested in first-order logic, resolution and other calculi for automated provers will find good advanced instruction in [[**?**], [**?**], Bib82, Lov78, [**?**]]. References to Internet resources can be found on this book's web site.

0.7.11 Exercises

Exercise 3.1 Let the three-place predicate ”child” and the one-place predicate ”female” from Example 3.2 on page 43 be given. Define:
(a) A one-place predicate ”male”.
(b) A two-place predicate ”father” and ”mother”.
(c) A two-place predicate ”siblings”.
(d) A predicate ”parents (x, y, z) ”, which is true if and only if x is the father and y is the mother of z.
(e) A predicate ”uncle (x, y) ”, which is true if and only if x is the uncle of y (use the predicates that have already been defined).
(f) A two-place predicate ”ancestor” with the meaning: ancestors are parents, grandparents, etc. of arbitrarily many generations. Exercise 3.2 Formalize the following statements in predicate logic:
(a) Every person has a father and a mother.
(b) Some people have children.
(c) All birds fly.
(d) There is an animal that eats (some) grain-eating animals.
(e) Every animal eats plants or plant-eating animals which are much smaller than itself.

Exercise 3.3 Adapt Exercise 3.1 on page 63 by using one-place function symbols and equality instead of “father” and “mother”.

Exercise 3.4 Give predicate logic axioms for the two-place relation “$<$” as a total order. For a total order we must have (1) Any two elements are comparable. (2) It is symmetric. (3) It is transitive.

Exercise 3.5 Unify (if possible) the following terms and give the MGU and the resulting terms.
(a) $p(x, f(y)), p(f(z), u)$
(b) $p(x, f(x)), p(y, y)$
(c) $x = 4 - 7 \cdot x, \cos y = z$
(d) $x < 2 \cdot x, 3 < 6$
(e) $q(f(x, y, z), f(g(w, w), g(x, x), g(y, y))), q(u, u)$

0.8 Exercise 3.6

(a) Transform Russell's Paradox from Example 3.7 on page 54 into CNF.
(b) Show that the empty clause cannot be derived using resolution without factorization from (3.7) on page 54. Try to understand this intuitively.

0.9 Exercise 3.7

(a) Why is resolution with the set of support strategy incomplete?
(b) Justify (without proving) why the set of support strategy becomes complete if $(KB \wedge \neg Q) \backslash SOS$ is satisfiable.
(c) Why is resolution with the pure literal rule complete?

- Exercise 3.8 Formalize and prove with resolution that in a semigroup with at least two different elements a, b, a left-neutral element e, and a left null element n, these two elements have to be different, that is, that $n \neq e$. Use demodulation, which allows replacement of "like with like".

Exercise 3.9 Obtain the theorem prover E [Sch02] or another prover and prove the following statements. Compare these proofs with those in the text.
(a) The claim from Example 2.3 on page 33.
(b) Russell's paradox from Example 3.7 on page 54.
(c) The claim from Exercise 3.8.

0.9.1 The Search Space Problem

As already mentioned in several places, in the search for a proof there are almost always many (depending on the calculus, potentially infinitely many) possibilities for the application of inference rules at every step. The result is the aforementioned explosive growth of the search space (Fig. 4.1 on page 66). In the worst case, all of these possibilities must be tried in order to find the proof, which is usually not possible in a reasonable amount of time. If we compare automated provers or inference systems with mathematicians or human experts who have experience in special domains, we make interesting observations. For one thing, experienced mathematicians can prove theorems which are far out of reach for automated provers. On the other hand, automated provers perform tens of thousands of inferences per second. A human in contrast performs maybe one inference per second. Although human experts are much slower on the object level (that is, in carrying out inferences), they apparently solve difficult problems much faster.

There are several reasons for this. We humans use intuitive calculi that work on a higher level and often carry out many of the simple inferences of an automated prover in one step. Furthermore, we use lemmas, that is, derived true formulas that we already know and therefore do not need to re-prove them each time. Meanwhile there are also machine provers that work with such methods. But even they cannot yet compete with human experts.

Figure 11: Possible consequences of the explosion of a search space

A further, much more important advantage of us humans is intuition, without which we could not solve any difficult problems [[**?**]]. The attempt to formalize intuition causes problems. Experience in applied AI projects shows that in complex domains such as medicine (see Sect. 7.3) or mathematics, most experts are unable to formulate this intuitive meta-knowledge verbally, much less to formalize it. Therefore we cannot program this knowledge or integrate it into calculi in the form of heuristics. Heuristics are methods that in many cases can greatly simplify or shorten the way to the goal, but in some cases (usually rarely) can greatly lengthen the way to the goal.

Heuristic search is important not only to logic, but generally to problem solving in AI and will therefore be thoroughly handled in Chap. 6.

An interesting approach, which has been pursued since about 1990, is the application of machine learning techniques to the learning of heuristics for directing the search of inference systems, which we will briefly sketch now. A resolution prover has, during the search for a proof, hundreds or more possibilities for resolution steps at each step, but only a few lead to the goal. It would be ideal if the prover could ask an oracle which two clauses it should use in the next step to quickly find the proof. There are attempts to build such proof-directing modules, which evaluate the various alternatives for the next step and then choose the alternative with the best rating. In the case of resolution, the rating of the available clauses could be computed by a function that calculates a value based on the number of positive literals, the complexity of the terms, etc., for every pair of resolvable clauses.

How can this function be implemented? Because this knowledge is "intuitive", the programmer is not familiar with it. Instead, one tries to copy nature and uses machine learning algorithms to learn from successful proofs [[**?**], SE90].

The attributes of all clause pairs participating in successful resolution steps are stored as positive, and the attributes of all unsuccessful resolutions are stored as negative. Then, using this training data and a machine learning system, a program is generated which can rate clause pairs heuristically (see Sect. 9.5).

A different, more successful approach to improving mathematical reasoning is followed with interactive systems that operate under the control of the user. Here one could name computer algebra programs such as Mathematica, Maple, or Maxima, which can automatically carry out difficult symbolic mathematical manipulations. The search for the proof, however, is left fully to the human. The aforementioned interactive prover Isabelle [NPW02] provides distinctly more support during the proof search. There are at present several projects, such as Omega [[**?**]] and MKM, [1] for the development of systems for supporting mathematicians during proofs.

In summary, one can say that, because of the search space problem, automated provers today can only prove relatively simple theorems in special domains with few axioms.

0.9.2 Decidability and Incompleteness

First-order predicate logic provides a powerful tool for the representation of knowledge and reasoning. We know that there are correct and complete calculi and theorem provers. When proving a theorem, that is, a true statement, such a prover is very helpful because, due to completeness, one knows after finite time that the statement really is true. What if the statement is not true? The completeness theorem (Theorem 3.3 on page 50) does not answer this question. [2] Specifically, there is no process that can prove or refute any formula from PL1 in finite time, for it holds that

Theorem 4.1 The set of valid formulas in first-order predicate logic is semidecidable.

This theorem implies that there are programs (theorem provers) which, given a true (valid) formula as input, determine its truth in finite time. If the formula is not valid, however, it may happen that the prover never halts. (The reader may grapple with this question in Exercise 4.1 on page 73.) Propositional logic is decidable because the truth table method provides all models of a formula in finite time. Evidently predicate logic with quantifiers and nested function symbols is a language somewhat too powerful to be decidable.

On the other hand, predicate logic is not powerful enough for many purposes. One often wishes to make statements about sets of predicates or functions. This does not work in PL1 because it only knows quantifiers for variables, but not for predicates or functions. Kurt Gödel showed, shortly after his completeness theorem for PL1, that completeness is lost if we extend PL1 even minimally to construct a higher-order logic. A first-order logic can only quantify over variables. A second-order logic can also quantify over formulas of the first order, and a third-order logic can quantify over formulas of the second order. Even adding only the induction axiom for the natural numbers makes the logic incomplete. The statement "If a predicate $p(n)$ holds for n, then $p(n+1)$ also holds," or

$$\forall p \; p(n) \Rightarrow p(n+1)$$

is a second-order proposition because it quantifies over a predicate. Gödel proved the following theorem:

Theorem 4.2 (Gödel's incompleteness theorem) Every axiom system for the natural numbers with addition and multiplication (arithmetic) is incomplete. That is, there are true statements in arithmetic that are not provable.

Gödel's proof works with what is called Gödelization, in which every arithmetic formula is encoded as a number. It obtains a unique Gödel number. Gödelization is now used to formulate the proposition

$$F = \text{"I am not provable."}$$

in the language of arithmetic. This formula is true for the following reason. Assume F is false. Then we can prove F and therefore show that F is not provable. This is a contradiction. Thus F is true and therefore not provable.

The deeper background of this theorem is that mathematical theories (axiom systems) and, more generally, languages become incomplete if the language becomes too powerful. A similar example is set theory. This language is so powerful that one can formulate paradoxes with it. These are statements that contradict themselves, such as the statement we already know from Example 3.7 on page 54 about the barbers who all shave those who do not shave themselves (see Exercise 4.2 on page 73). The dilemma consists therein that with languages which are powerful enough to describe mathematics and interesting applications, we are smuggling contradictions and incompletenesses through the back door. This does not mean, however, that higher-order logics are wholly unsuited for formal methods. There are certainly formal systems as well as provers for higher-order logics.

[21] www.mathweb.org/mathweb/demo.html.

[2] Just this case is especially important in practice, because if I already know that a statement is true, I no longer need a prover.

0.9.3 The Flying Penguin

With a simple example we will demonstrate a fundamental problem of logic and possible solution approaches. Given the statements

1. Tweety is a penguin
2. Penguins are birds
3. Birds can fly

Formalized in PL1, the knowledge base KB results:

$$\begin{aligned}&\text{penguin(tweety)}\\&\text{penguin }(x) \Rightarrow \text{bird}(x)\\&\text{bird}(x) \Rightarrow \text{fly}(x)\end{aligned}$$

From there (for example with resolution) fly(tweety) can be derived (Fig. 4.2 on page 70). [3] Evidently the formalization of the flight attributes of penguins is insufficient. We try the additional statement Penguins cannot fly, that is

$$\text{penguin}(x) \Rightarrow \neg\,\text{fly}(x)$$

From there $\neg$ fly(tweety) can be derived. But fly(tweety) is still true. The knowledge base is therefore inconsistent. Here we notice an important characteristic of logic, namely monotony. Although we explicitly state that penguins cannot fly, the opposite can still be derived.

Definition 4.1 A logic is called monotonic if, for an arbitrary knowledge base KB and an arbitrary formula ϕ, the set of formulas derivable from KB is a subset of the formulas derivable from $KB \cup \phi$.

If a set of formulas is extended, then, after the extension, all previously derivable statements can still be proved, and additional statements can potentially also be proved. The set of provable statements thus grows monotonically when the set of formulas is extended. For our example this means that the extension of the knowledge base will never lead to our goal. We thus modify KB by replacing the obviously false statement "(all) birds can fly" with the more exact statement "(all) birds except penguins can fly" and obtain as KB_2 the following clauses:

$$\begin{aligned}&\text{penguin(tweety)}\\&\text{penguin }(x) \Rightarrow \text{bird}(x)\\&\text{bird}(x) \wedge \neg\,\text{penguin}(x) \Rightarrow \text{fly}(x)\\&\text{penguin}(x) \Rightarrow \neg\,\text{fly}(x)\end{aligned}$$

Now the world is apparently in order again. We can derive $\neg$ fly(tweety), but not fly(tweety), because for that we would need $\neg$ penguin(x), which, however,

[23] The formal execution of this and the following simple proof may be left to the reader (Exercise 4.3 on page 73).

Figure 12: The flying penguin Tweety

is not derivable. As long as there are only penguins in this world, peace reigns. Every normal bird, however, immediately causes problems. We wish to add the raven Abraxas (from the German book ”The Little Witch”) and obtain

$$
\begin{array}{l}
\text{raven(abraxas)}\\
\text{raven}(x) \Rightarrow \text{bird}(x)\\
\text{penguin (tweety)}\\
\text{penguin}(x) \Rightarrow \text{bird}(x)\\
\text{bird}(x) \wedge \neg\,\text{penguin}(x) \Rightarrow \text{fly}(x)\\
\text{penguin}(x) \Rightarrow \neg\,\text{fly}(x)
\end{array}
$$

We cannot say anything about the flight attributes of Abraxas because we forgot to formulate that ravens are not penguins. Thus we extend KB_3 to KB_4:

$$
\begin{array}{l}
\text{raven(abraxas)}\\
\text{raven}(x) \Rightarrow \text{bird}(x)\\
\text{raven}(x) \Rightarrow \neg\,\text{pinguin}(x)\\
\text{penguin(tweety)}\\
\text{penguin}(x) \Rightarrow \text{bird}(x)\\
\text{bird}(x) \wedge \neg\,\text{penguin}(x) \Rightarrow \text{fly}(x)\\
\text{penguin}(x) \Rightarrow \neg\,\text{fly}(x)
\end{array}
$$

The fact that ravens are not penguins, which is self-evident to humans, must be explicitly added here. For the construction of a knowledge base with all 9,800 or so types of birds worldwide, it must therefore be specified for every type of bird (except for penguins) that it is not a member of penguins. We must proceed analogously for all other exceptions such as the ostrich.

For every object in the knowledge base, in addition to its attributes, all of the attributes it does not have must be listed.

To solve this problem, various forms of non-monotonic logic have been developed, which allow knowledge (formulas) to be removed from the knowledge base. Under the name default logic, logics have been developed which allow objects to be assigned attributes which are valid as long as no other rules are available. In the Tweety example, the rule birds can fly would be such a default rule. Despite great effort, these logics have at present, due to semantic and practical problems, not succeeded.

Monotony can be especially inconvenient in complex planning problems in which the world can change. If for example a blue house is painted red, then afterwards it is red. A knowledge base such as

$$\text{color} (\text{ house }, \text{ blue })$$
$$\text{paint} (\text{ house }, \text{ red })$$
$$\text{paint } (x, y) \Rightarrow \text{color}(x, y)$$

leads to the conclusion that, after painting, the house is red and blue. The problem that comes up here in planning is known as the frame problem. A solution for this is the situation calculus presented in Sect. 5.6.

An interesting approach for modeling problems such as the Tweety example is probability theory. The statement ”all birds can fly” is false. A statement something like ”almost all birds can fly” is correct. This statement becomes more exact if we give a probability for ”birds can fly”. This leads to probabilistic logic, which today represents an important sub-area of AI and an important tool for modeling uncertainty (see Chap. 7).

0.9.4 Modeling Uncertainty

Two-valued logic can and should only model circumstances in which there is true, false, and no other truth values. For many tasks in everyday reasoning, two-valued logic is therefore not expressive enough. The rule

$$\text{bird}(x) \Rightarrow \text{fly}(x)$$

is true for almost all birds, but for some it is false. As was already mentioned, working with probabilities allows exact formulation of uncertainty. The statement ”99% of all birds can fly” can be formalized by the expression

$$P(\text{bird}(x) \Rightarrow \text{fly}(x)) = 0.99$$

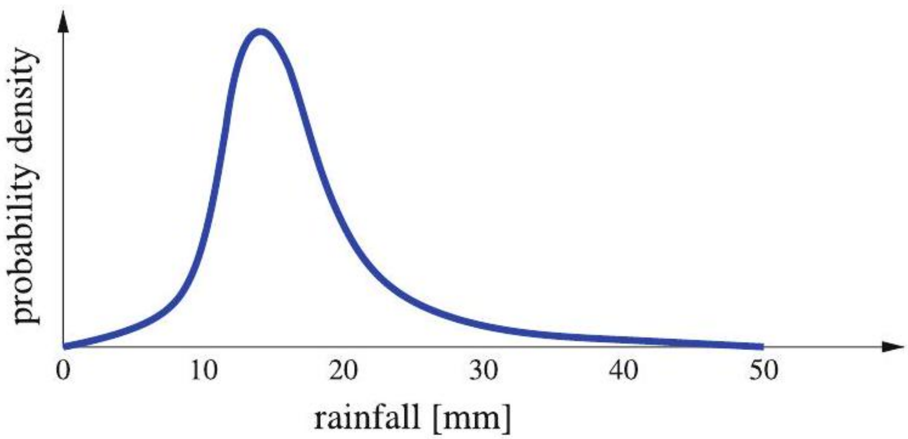


Figure 13: Probability density of the continuous variable rainfall

In Chap. 7 we will see that here it is better to work with conditional probabilities such as

$$P(\text{fly} \mid \text{bird}) = 0.99.$$

With the help of Bayesian networks, complex applications with many variables can also be modeled.

A different model is needed for the statement "The weather is nice". Here it often makes no sense to speak in terms of true and false. The variable weather_is_nice should not be modeled as binary, rather continuously with values, for example, in the interval $[0, 1]$. weather_is_nice $= 0.7$ then means "The weather is fairly nice". Fuzzy logic was developed for this type of continuous (fuzzy) variable.

Probability theory also offers the possibility of making statements about the probability of continuous variables. A statement in the weather report such as "There is a high probability that there will be some rain" could for example be exactly formulated as a probability density of the form

$$P(\text{rainfall} = X) = Y$$

and represented graphically something like in Fig. 4.3.
This very general and even visualizable representation of both types of uncertainty we have discussed, together with inductive statistics and the theory of Bayesian networks, makes it possible, in principle, to answer arbitrary probabilistic queries.

Probability theory as well as fuzzy logic are not directly comparable to predicate logic because they do not allow variables or quantifiers. They can thus be seen as extensions of propositional logic as shown in the following table.

Formalism	Number of truth values	Probabilities expressible
Propositional logic	2	-
Fuzzy logic	∞	-
Discrete probabilistic logic	n	yes
Continuous probabilistic logic	∞	yes

0.10 Exercise

(a) With the following (false) argument, one could claim that PL1 is decidable: We take a complete proof calculus for PL1. With it we can find a prooffor any true formula in finite time. For every other formula ϕ I proceed as follows: I apply the calculus to $\neg\phi$ and show that $\neg\phi$ is true. Thus ϕ is false. Thus I can prove or refute every formula in PL1. Find the mistake in the argument and change it so it becomes correct.
(b) Construct a decision process for the set of true and unsatisfiable formulas in PL1.

0.11 Exercise

(a) Given the statement "There is a barber who shaves every person who does not shave himself." Consider whether this barber shaves himself.
(b) Let $M = \{x \mid x \notin x\}$. Describe this set and consider whether M contains itself.

0.12 Exercise

Use an automated theorem prover (for example E [Sch02]) and apply it to all five different axiomatizations of the Tweety example from Sect. 4.3. Validate the example's statements.

0.12.1 Logic Programming with PROLOG

Compared to classical programming languages such as C or Pascal, Logic makes it possible to express relationships elegantly, compactly, and declaratively. Automated theorem provers are even capable of deciding whether a knowledge base logically entails a query. Proof calculus and knowledge stored in the knowledge base are strictly separated. A formula described in clause normal form can be used as input data for any theorem prover, independent of the proof calculus used. This is of great value for reasoning and the representation of knowledge.

If one wishes to implement algorithms, which inevitably have procedural components, a purely declarative description is often insufficient. Robert Kowalski, one of the pioneers of logic programming, made this point with the formula

$$\text{Algorithm} = \text{Logic} + \text{Control}.$$

This idea was brought to fruition in the language PROLOG. PROLOG is used in many projects, primarily in AI and computational linguistics. We will now give a short introduction to this language, present the most important concepts, show its strengths, and compare it with other programming languages and theorem provers. Those looking for a complete programming course are directed to textbooks such as [[**?**], [**?**]] and the handbooks [[**?**], [**?**]].

The syntax of the language PROLOG only allows Horn clauses. Logical notation and PROLOG's syntax are juxtaposed in the following table:

PL1 / clause normal form	PROLOG	Description
$(\neg A_1 \vee \ldots \vee \neg A_m \vee B)$	$\mathrm{B} :- \mathrm{A}_1, \ldots, \mathrm{A}_\mathrm{m}.$	Rule
$(A_1 \wedge \ldots \wedge A_m) \Rightarrow B$	$\mathrm{B} :- \mathrm{A}_1, \ldots, \mathrm{A}_\mathrm{m}.$	Rule
A	A.	Fact
$(\neg A_1 \vee \ldots \vee \neg A_m)$	$?- \mathrm{A}_1, \ldots, \mathrm{A}_\mathrm{m}.$	Query
$\neg(A_1 \wedge \ldots \wedge A_m)$	$?- \mathrm{A}_1, \ldots, \mathrm{A}_\mathrm{m}.$	Query

Here $A_1, \ldots, A_m, A, B$ are literals. The literals are, as in PL1, constructed from predicate symbols with terms as arguments. As we can see in the above table, in PROLOG there are no negations in the strict logical sense because the sign of a literal is determined by its position in the clause.

0.12.2 PROLOG Systems and Implementations

An overview of current PROLOG systems is available in the collection of links on this book's home page. To the reader we recommend the very powerful and freely available (under GNU public licenses) systems GNU-PROLOG [Dia04] and SWI-PROLOG. For the following examples, SWI-PROLOG [Wie04] was used.

Most modern PROLOG systems work with an interpreter based on the Warren abstract machine (WAM). PROLOG source code is compiled into so-called WAM code, which is then interpreted by the WAM. The fastest implementations of a WAM manage up to 10 million logical inferences per second (LIPS) on a 1 GHz PC .

0.12.3 Simple Examples

We begin with the family relationships from Example 3.2 on page 43. The small knowledge base KB is coded-without the facts for the predicate female-as a PROLOG program named rel.pl in Fig. 5.1.

The program can be loaded and compiled in the PROLOG interpreter with the command

```
?- [rel].
```

```
child(oscar,karen,frank).
child(mary,karen,frank).
child(eve,anne,oscar).
child(henry,anne,oscar).
child(isolde,anne,oscar).
child(clyde,mary,oscarb).
child(X,Z,Y) :- child(X,Y,Z).
descendant(X,Y) :- child(X,Y,Z).
descendant(X,Y) :- child(X,U,V), descendant(U,Y).
```

Fig. 5.1 PROLOG program with family relationships

An initial query returns the dialog

?- child(eve,oscar, anne).

Yes

with the correct answer Yes. How does this answer come about? For the query "? - child (eve, oscar, anne) ." there are six facts and one rule with the same predicate in its clause head. Now unification is attempted between the query and each of the complementary literals in the input data in order of occurrence. If one of the alternatives fails, this results in backtracking to the last branching point, and the next alternative is tested. Because unification fails with every fact, the query is unified with the recursive rule in line 8 . Now the system attempts to solve the subgoal child (eve, anne, oscar), which succeeds with the third alternative. The query

?- descendant (X,Y).

X = oscar

Y = karen

Yes

is answered with the first solution found, as is

?- descendant (clyde,Y).

Y = mary

Yes

The query

?- descendant (clyde, karen).

is not answered, however. The reason for this is the clause in line 8 , which specifies symmetry of the child predicate. This clause calls itself recursively without the possibility of termination. This problem can be solved with the following new program (facts have been omitted here).

```
descendant(X,Y) :- child(X,Y,Z).
descendant(X,Y) :- child(X,Z,Y).
descendant(X,Y) :- child(X,U,V), descendant(U,Y).
```

But now the query

?- child(eve,oscar, anne).

is no longer correctly answered because the symmetry of child in the last two variables is no longer given. A solution to both problems is found in the program

```
child_fact(oscar,karen,franz).
child_fact(mary, karen,franz).
child_fact(eva,anne,oscar).
child_fact(henry,anne,oscar).
child_fact(isolde,anne,oscar).
child_fact(clyde,mary,oscarb).
child(X,Z,Y) :- child_fact(X,Y,Z).
child(X,Z,Y) :- child_fact(X,Z,Y).
descendant(X,Y) :- child(X,Y,Z).
descendant(X,Y) :- child(X,U,V), descendant(U,Y).
```

By introducing the new predicate child_fact for the facts, the predicate child is no longer recursive. However, the program is no longer as elegant and simple as the-logically correct-first variant in Fig. 5.1 on page 76, which leads to the infinite loop. The PROLOG programmer must, just as in other languages, pay attention to processing and avoid infinite loops. PROLOG is just a programming language and not a theorem prover.

We must distinguish here between declarative and procedural semantics of PROLOG programs. The declarative semantics is given by the logical interpretation of the horn clauses. The procedural semantics, in contrast, is defined by the execution of the PROLOG program, which we wish to observe in more detail now. The execution of the program from Fig. 5.1 on page 76 with the query child (eve, oscar, anne) is represented in Fig. 5.2 as a search tree. [1] Execution begins at the top left with the query. Each edge represents a possible SLD resolution step with a complementary unifiable literal. While the search tree becomes infinitely deep by the recursive rule, the PROLOG execution terminates because the facts occur before the rule in the input data.

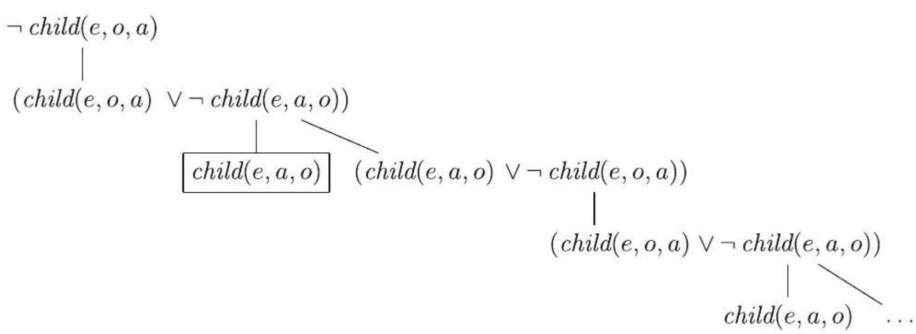


Figure 14: PROLOG search tree for child (eve, oscar, anne)

With the query descendant (clyde, karen), in contrast, the PROLOG execution does not terminate. We can see this clearly in the and-or tree presented in Fig. 5.3. In this representation the branches, represented by , lead from the

[21] The constants have been abbreviated to save space.

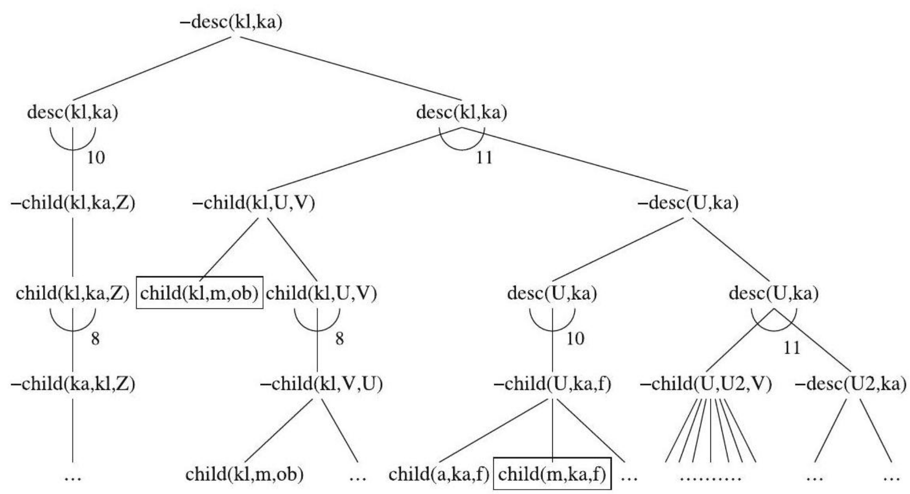


Figure 15: And-or tree for desc (clyde, karen)

head of a clause to the subgoals. Because all subgoals of a clause must be solved, these are and branches. All other branches are or branches, of which at least one must be unifiable with its parent nodes. The two outlined facts represent the solution to the query. The PROLOG interpreter does not terminate here, however, because it works by using a depth-first search with backtracking (see Sect. 6.2.2) and thus first chooses the infinitely deep path to the far left.

0.12.4 Execution Control and Procedural Elements

As we have seen in the family relationship example, it is important to control the execution of PROLOG. Avoiding unnecessary backtracking especially can lead to large increases in efficiency. One means to this end is the cut. By inserting an exclamation mark into a clause, we can prevent backtracking over this point. In the following program, the predicate max (X, Y, Max) computes the maximum of the two numbers X and Y.

```
1 max(X,Y,X) :- X >= Y.
2 max(X,Y,Y) :- X < Y.
```

If the first case (first clause) applies, then the second will not be reached. On the other hand, if the first case does not apply, then the condition of the second case is true, which means that it does not need to be checked. For example, in the query
?- max(3,2,Z), Z > 10.
backtracking is employed because $Z = 3$, and the second clause is tested for max, which is doomed to failure. Thus backtracking over this spot is unnecessary. We can optimize this with a cut:

```
max(X,Y,X) :- X >= Y, !.
max(X,Y,Y).
```

Thus the second clause is only called if it is really necessary, that is, if the first clause fails. However, this optimization makes the program harder to understand.

Another possibility for execution control is the built-in predicate fail, which is never true. In the family relationship example we can quite simply print out all children and their parents with the query

```
?- child_fact(X,Y,Z), write(X), write(' is a child of '),
write(Y), write(' and '), write(Z), write('.'), nl, fail.
```

The corresponding output is

```
oscar is a child of karen and frank.
mary is a child of karen and frank.
eve is a child of anne and oscar.
No.
```

where the predicate nl causes a line break in the output. What would be the output in the end without use of the fail predicate?

With the same knowledge base, the query "?- child_fact (ulla, X, Y)." would result in the answer No because there are no facts about ulla. This answer is not logically correct. Specifically, it is not possible to prove that there is no object with the name ulla. Here the prover E would correctly answer "No proof found." Thus if PROLOG answers No, this only means that the query Q cannot be proved. For this, however, $\neg Q$ must not necessarily be proved. This behavior is called negation as failure.

Restricting ourselves to Horn clauses does not cause a big problem in most cases. However, it is important for procedural execution using SLD-resolution (Sect. 2.5). Through the singly determined positive literal per clause, SLD resolution, and therefore the execution of PROLOG programs, have a unique entry point into the clause. This is the only way it is possible to have reproducible execution of logic programs and, therefore, well-defined procedural semantics.

Indeed, there are certainly problem statements which cannot be described by Horn clauses. An example is Russell's paradox from Example 3.7 on page 54, which contains the non-Horn clause (shaves(barber, X) $\vee$ shaves (X, X)).

0.12.5 Lists

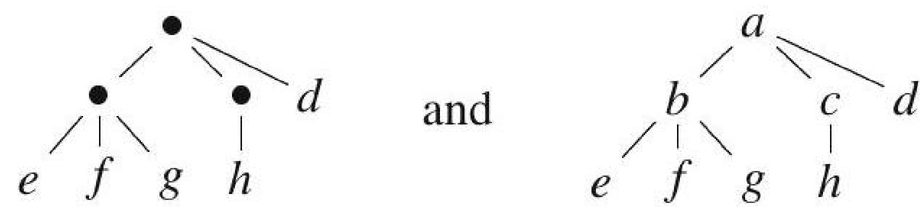
As a high-level language, PROLOG has, like the language LISP, the convenient generic list data type. A list with the elements $A, 2, 2, B, 3, 4, 5$ has the form $[A, 2, 2, B, 3, 4, 5]$

The construct $[Head|Tail]$ separates the first element (Head) from the rest (Tail) of the list. With the knowledge base list([A,2,2,B,3,4,5]).

PROLOG displays the dialog ?- list([H|T]).
$H = A$
$T = [2, 2, B, 3, 4, 5]$
Yes

By using nested lists, we can create arbitrary tree structures. For example, the two trees

can be represented by the lists $[b, c]$ and $[a, b, c]$, respectively, and the two trees



by the lists $[[e, f, g], [h], d]$ and $[a, [b, e, f, g], [c, h], d]$, respectively. In the trees where the inner nodes contain symbols, the symbol is the head of the list and the child nodes are the tail.

A nice, elegant example of list processing is the definition of the predicate append (X, Y, Z) for appending list Y to the list X. The result is saved in Z. The corresponding PROLOG program reads

```
append([],L,L).
append([X|L1],L2,[X|L3]) :- append(L1,L2,L3).
```

This is a declarative (recursive) logical description of the fact that L3 results from appending L2 to L1. At the same time, however, this program also does the work when it is called. The call ?- append([a,b,c],[d,1,2],Z). returns the substitution $Z = [a, b, c, d, 1, 2]$, just as the call ?- append(X, [1,2,3], [4,5,6,1,2,3]). yields the substitution $X = [4, 5, 6]$. Here we observe that append is not a two-place function, but a three-place relationship. Actually, we can also input the "output parameter" Z and ask whether it can be created.

Reversing the order of a list's elements can also be elegantly described and simultaneously programmed by the recursive predicate

```
nrev([],[]).
nrev([H|T],R) :- nrev(T,RT), append(RT,[H],R).
```

which reduces the reversal of a list down to the reversal of a list that is one element smaller. Indeed, this predicate is very inefficient due to calling append. This program is known as naive reverse and is often used as a PROLOG benchmark (see Exercise 5.6 on page 89). Things go better when one proceeds using a temporary store, known as the accumulator, as follows:

List	Accumulator
[a, b, c, d]	[]
[b, c, d]	[a]
[c, d]	[b, a]
[d]	[c, b, a]
[]	[d, c, b, a]

The corresponding PROLOG program reads

```
accrev([],A,A).
accrev([H|T],A,R) :- accrev(T,[H|A],R).
```

0.12.6 Self-modifying Programs

PROLOG programs are not fully compiled, rather, they are interpreted by the WAM. Therefore it is possible to modify programs at runtime. A program can even modify itself. With commands such as assert and retract, facts and rules can be added to the knowledge base or taken out of it.

A simple application of the variant asserta is the addition of derived facts to the beginning of the knowledge base with the goal of avoiding a repeated, potentially time-expensive derivation (see Exercise 5.8 on page 89). If in our family relationship example we replace the two rules for the predicate descendant with

```
:- dynamic descendant/2.
descendant(X,Y) :- child(X,Y,Z), asserta(descendant(X,Y)).
descendant(X,Y) :- child(X,U,V), descendant(U,Y),
    asserta(descendant(X,Y)).
```

then all derived facts for this predicate are saved in the knowledge base and thus in the future are not re-derived. The query

```
?- descendant(clyde, karen).
```

leads to the addition of the two facts

```
descendant(clyde, karen).
descendant(mary, karen).
```

By manipulating rules with assert and retract, even programs that change themselves completely can be written. This idea became known under the term genetic programming. It allows the construction of arbitrarily flexible learning programs. In practice, however, it turns out that, due to the huge number of senseless possible changes, changing the code by trial and error rarely leads to a performance increase. Systematic changing of rules, on the other hand, makes programming so much more complex that, so far, such programs that extensively modify their own code have not been successful. In Chap. 8 we will show how machine learning has been quite successful. However, only very limited modifications of the program code are being conducted here.

0.12.7 A Planning Example

Example 5.1 The following riddle serves as a problem statement for a typical PROLOG program.

A farmer wants to bring a cabbage, a goat, and a wolf across a river, but his boat is so small that he can only take them across one at a time. The farmer thought it over and then said to himself: "If I first bring the wolf to the other side, then the goat will eat the cabbage. If I transport the cabbage first, then the goat will be eaten by the wolf. What should I do?"

This is a planning task which we can quickly solve with a bit of thought. The PROLOG program given in Fig. 5.4 on page 84 is not created quite as fast.

The program works on terms of the form state (Farmer, Wolf, Goat, Cabbage), which describe the current state of the world. The four variables with possible values left, right give the location of the objects. The central

```
start :- action(state(left,left,left,left),
            state(right,right,right,right)).
action(Start,Goal):-
        plan(Start,Goal,[Start],Path),
        nl,write('Solution:'),nl,
        write_path(Path).
            write_path(Path), fail. % all solutions output
plan(Start,Goal,Visited,Path):-
    go(Start,Next),
    safe(Next),
    \+ member(Next,Visited), % not(member(...))
    plan(Next,Goal,[Next|Visited],Path).
plan(Goal,Goal,Path,Path).
go(state(X,X,Z,K),state(Y,Y,Z,K)):-across(X,Y). % farmer, wolf
go(state(X,W,X,K),state(Y,W,Y,K)):-across(X,Y). % farmer, goat
go(state(X,W,Z,X),state(Y,W,Z,Y)):-across(X,Y). % farmer, cabbage
go(state(X,W,Z,K),state(Y,W,Z,K)):-across(X,Y). % farmer
across(left,right).
across(right,left).
safe(state(B,W,Z,K)) :- across(W,Z), across(Z,K).
safe(state(B,B,B,K)).
safe(state(B,W,B,B)).
```

Fig. 5.4 PROLOG program for the farmer-wolf-goat-cabbage problem
The recursive predicate plan first creates a successor state Next using go, tests its safety with safe, and repeats this recursively until the start and goal states are the same (in program line 15). The states which have already been visited are stored in the third argument of plan. With the built-in predicate member it is tested whether the state Next has already been visited. If yes, it is not attempted.

The definition of the predicate write_path for the task of outputting the plan found is missing here. It is suggested as an exercise for the reader (Exercise 5.2 on page 88). For initial program tests the literal write_path (Path) can be replaced with write (Path). For the query "? - start." we get the answer

```
Solution:
Farmer and goat from left to right
Farmer from right to left
Farmer and wolf from left to right
Farmer and goat from right to left
Farmer and cabbage from left to right
Farmer from right to left
Farmer and goat from left to right
```

Yes
For better understanding we describe the definition of plan in logic:

$$\forall z\, \text{plan}(z, z) \wedge \forall s \forall z \forall n [go(s, n) \wedge \text{safe}(n) \wedge \text{plan}(n, z) \Rightarrow \text{plan}(s, z)]$$

This definition comes out significantly more concise than in PROLOG. There are two reasons for this. For one thing, the output of the discovered plan is unimportant for logic. Furthermore, it is not really necessary to check whether the next state was already visited if unnecessary trips do not bother the farmer. If, however, $\backslash+$ member (...) is left out of the PROLOG program, then there is an infinite loop and PROLOG might not find a schedule even if there is one. The cause of this is PROLOG's backward chaining search strategy, which, according to the depth-first search (Sect. 6.2.2) principle, always works on subgoals one at a time without restricting recursion depth, and is therefore incomplete. This would not happen to a theorem prover with a complete calculus.

As in all planning tasks, the state of the world changes as actions are carried out from one step to the next. This suggests sending the state as a variable to all predicates that depend on the state of the world, such as in the predicate safe. The state transitions occur in the predicate go. This approach is called situation calculus [RN10]. We will become familiar with an interesting extension to learning action sequences in partially observable, non-deterministic worlds in Chap. 10.

Predicate logic and simpler sub-languages for the description and solution of planning tasks play an increasingly important role for intelligent robots. To solve complex tasks, robots often work on symbolic relational descriptions of the world. The classic application for a service robot is recognizing an audible command linguistically and then converting it into a logical formula. Then a planner has the task of finding a plan which eventually reaches a state in which the formula becomes true. Even more interesting is when the robot learns to meet the goals of its trainer by observing them and then use the symbolic planner to achieve the goals [[**?**]]. One very exciting thing about this is that the robot is able to take numerical sensor values such as pixel images and turn them into a symbolic description of situations. This generally very difficult task is known in AI as symbol grounding. Robotics developers have found interesting solutions to this problem for certain tasks [[**?**]].

0.12.8 Constraint Logic Programming

The programming of scheduling systems, in which many (sometimes complex) logical and numerical conditions must be fulfilled, can be very expensive and difficult with conventional programming languages. This is precisely where logic could be useful. One simply writes all logical conditions in PL1 and then enters a query. Usually this approach fails miserably. The reason is the penguin problem discussed in Sect. 4.3. The fact penguin (tweety) does ensure that penguin (tweety) is true. However, it does not rule out that raven (tweety) is also true. To rule this out with additional axioms is very inconvenient (Sect. 4.3).

Constraint Logic Programming (CLP), which allows the explicit formulation of constraints for variables, offers an elegant and very efficient mechanism for solving this problem. The interpreter constantly monitors the execution of the program for adherence to all of its constraints. The programmer is fully relieved of the task of controlling the constraints, which in many cases can greatly simplify programming. This is expressed in the following quotation by Eugene C. Freuder from [[**?**]]:

> Constraint programming represents one of the closest approaches
> computer science has yet made to the Holy Grail of programming:
> the user states the problem, the computer solves it.

Without going into the theory of the constraint satisfaction problem (CSP), we will apply the CLP mechanism of GNU-PROLOG to the following example.

Example 5.2 The secretary of Albert Einstein High School has to come up with a plan for allocating rooms for final exams. He has the following information: the four teachers Mayer, Hoover, Miller and Smith give tests for the subjects German, English, Math, and Physics in the ascendingly numbered rooms 1, 2, 3 and 4. Every teacher gives a test for exactly one subject in exactly one room. Besides that, he knows the following about the teachers and their subjects.

1. Mr. Mayer never tests in room 4.
2. Mr. Miller always tests German.
3. Mr. Smith and Mr. Miller do not give tests in neighboring rooms.

4. Mrs. Hoover tests Mathematics.
5. Physics is always tested in room number 4.
6. German and English are not tested in room 1.

Who gives a test in which room?
A GNU-PROLOG program for solving this problem is given in Fig. 5.5. This program works with the variables Mayer, Hoover, Miller, Smith as well as

```
start :-
    fd_domain([Mayer, Hoover, Miller, Smith],1,4),
    fd_all_different([Mayer, Miller, Hoover, Smith]),
    fd_domain([German, English, Math, Physics],1,4),
    fd_all_different([German, English, Math, Physics]),
    fd_labeling([Mayer, Hoover, Miller, Smith]),
    Mayer #\=4, % Mayer not in room 4
    Miller #= German, % Miller tests German
    dist(Miller,Smith) #>= 2, % Distance Miller/Smith >= 2
    Hoover #= Math, % Hoover tests mathematics
    Physics #= 4, % Physics in room 4
    German #\=1, % German not in room 1
    English #\=1, % English not in room 1
    nl,
    write([Mayer, Hoover, Miller, Smith]), nl,
    write([German, English, Math, Physics]), nl.
```

Fig. 5.5 CLP program for the room scheduling problem

German, English, Math, Physics, which can each take on an integer value from 1 to 4 as the room number (program lines 2 and 5). A binding Mayer = 1 and German $= 1$ means that Mr. Mayer gives the German test in room 1. Lines 3 and 6 ensure that the four particular variables take on different values. Line 8 ensures that all variables are assigned a concrete value in the case of a solution. This line is not absolutely necessary here. If there were multiple solutions, however, only intervals would be output. In lines 10 to 16 the constraints are given, and the remaining lines output the room numbers for all teachers and all subjects in a simple format.

The program is loaded into GNU-PROLOG with "['raumplan.pl'].", and with "start." we obtain the output

```
[3,1,2,4]
[2,3,1,4]
```

Represented somewhat more conveniently, we have the following room schedule:

Room num.	1	2	3	4
Teacher	Hoover	Miller	Mayer	Smith
Subject	Math	German	English	Physics

GNU-PROLOG has, like most other CLP languages, a so-called finite domain constraint solver, with which variables can be assigned a finite range of integers. This need not necessarily be an interval as in the example. We can also input a list of values. As an exercise the user is invited, in Exercise 5.9 on page 89, to create a CLP program, for example with GNU-PROLOG, for a not-so-simple logic puzzle. This puzzle, supposedly created by Einstein, can very easily be solved with a CLP system. If we tried using PROLOG without constraints, on the other hand, we could easily grind our teeth out. Anyone who finds an elegant solution with PROLOG or a prover, please let it find its way to the author.

0.12.9 Summary

Unification, lists, declarative programming, and the relational view of procedures, in which an argument of a predicate can act as both input and output, allow the development of short, elegant programs for many problems. Many programs would be significantly longer and thus more difficult to understand if written in a procedural language. Furthermore, these language features save the programmer time. Therefore PROLOG is also an interesting tool for rapid prototyping, particularly for AI applications. The CLP extension of PROLOG is helpful not only for logic puzzles, but also for many optimization and scheduling tasks.

Since its invention in 1972, in Europe PROLOG has developed into one of Europe's leading programming languages in AI, along with procedural languages. In the U.S., on the other hand, the natively invented language LISP dominates the AI market.

PROLOG is not a theorem prover. This is intentional, because a programmer must be able to easily and flexibly control processing, and would not get very far with a theorem prover. On the other hand, PROLOG is not very helpful on its own for proving mathematical theorems. However, there are certainly interesting theorem provers which are programmed in PROLOG.

Recommended as advanced literature are [Bra11] and [CM94], as well as the handbooks [Wie04, Dia04] and, on the topic of CLP, [[**?**]].

0.12.10 Exercises

Exercise 5.1 Try to prove the theorem from Sect. 3.7 about the equality of left- and right-neutral elements of semi-groups with PROLOG. Which problems come up? What is the cause of this?

0.13 Exercise 5.2

(a) Write a predicate write_move(+State1, +State2), that outputs a sentence like "Farmer and wolf cross from left to right" for each boat crossing. Statel and State2 are terms of the form state (Farmer, Wolf, Goat, Cabbage).

(b) Write a recursive predicate write_path(+Path), which calls the predicate write_move(+State1, +State2) and outputs all of the farmer's actions.

0.14 Exercise 5.3

(a) At first glance the variable Path in the predicate plan of the PROLOG program from Example 5.1 on page 83 is unnecessary because it is apparently not changed anywhere. What is it needed for?
(b) If we add a fail to the end of action in the example, then all solutions will be given as output. Why is every solution now printed twice? How can you prevent this?

0.15 Exercise 5.4

(a) Show by testing out that the theorem prover E (in contrast to PROLOG), given the knowledge base from Fig. 5.1 on page 76, answers the query "?- descendant (clyde, karen) ." correctly. Why is that?
(b) Compare the answers of PROLOG and E for the query "?- descendant (X, Y) .".

Exercise 5.5 Write as short a PROLOG program as possible that outputs 1024 ones.

- Exercise 5.6 Investigate the runtime behavior of the naive reverse predicate.

- (a) Run PROLOG with the trace option and observe the recursive calls of nrev, append, and accrev.
- (b) Compute the asymptotic time complexity of append ($\mathrm{L}1,\ \mathrm{L}2,\ \mathrm{L}3$), that is, the dependency of the running time on the length of the list for large lists. Assume that access to the head of an arbitrary list takes constant time.
- (c) Compute the time complexity of nrev(L, R).
- (d) Compute the time complexity of accrev (L, R).
- (e) Experimentally determine the time complexity of the predicates nrev, append, and accrev, for example by carrying out time measurements (time(+Goal) gives inferences and CPU time.).

Exercise 5.7 Use function symbols instead of lists to represent the trees given in Sect. 5.4 on page 81.

- Exercise 5.8 The Fibonacci sequence is defined recursively by $fib(0) = 1, fib(1) = 1$ and $fib(n) = fib(n-1) + fib(n-2)$.

- (a) Define a recursive PROLOG predicate fib (N, R) which calculates $fib(N)$ and returns it in R.
- (b) Determine the runtime complexity of the predicate fib theoretically and by measurement.
- (c) Change your program by using asserta such that unnecessary inferences are no longer carried out.
- (d) Determine the runtime complexity of the modified predicate theoretically and by measurement (notice that this depends on whether fib was previously called).
- (e) Why is fib with asserta also faster when it is started for the first time right after PROLOG is started?

• Exercise 5.9 The following typical logic puzzle was supposedly written by Albert Einstein. Furthermore, he supposedly claimed that only 2% of the world's population is capable of solving it. The following statements are given.

• There are five houses, each painted a different color.

• Every house is occupied by a person with a different nationality.

• Every resident prefers a specific drink, smokes a specific brand of cigarette, and has a specific pet.

• None of the five people drinks the same thing, smokes the same thing, or has the same pet.

• Hints:

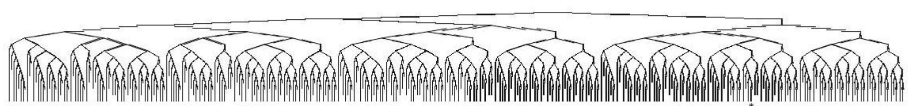
  - The Briton lives in the red house.
  - The Swede has a dog.
  - The Dane likes to drink tea.
  - The green house is to the left of the white house.
  - The owner of the green house drinks coffee.
  - The person who smokes Pall Mall has a bird.
  - The man who lives in the middle house drinks milk.
  - The owner of the yellow house smokes Dunhill.
  - The Norwegian lives in the first house.
  - The Marlboro smoker lives next to the one who has a cat.
  - The man with the horse lives next to the one who smokes Dunhill.
  - The Winfield smoker likes to drink beer.
  - The Norwegian lives next to the blue house.
  - The German smokes Rothmanns.

– The Marlboro smoker has a neighbor who drinks water.

Question: To whom does the fish belong?
(a) First solve the puzzle manually.
(b) Write a CLP program (for example with GNU-PROLOG) to solve the puzzle. Orient yourself with the room scheduling problem in Fig. 5.5 on page 86.

0.15.1 Introduction

The search for a solution in an extremely large search tree presents a problem for nearly all inference systems. From the starting state there are many possibilities for the first inference step. For each of these possibilities there are again many possibilities in the next step, and so on. Even in the proof of a very simple formula from [[**?**]] with three Horn clauses, each with at most three literals, the search tree for SLD resolution has the following shape:



The tree was cut off at a depth of 14 and has a solution in the leaf node marked by $*$. It is only possible to represent it at all because of the small branching factor of at most two and a cutoff at depth 14. For realistic problems, the branching factor and depth of the first solution may become significantly bigger.

Assume the branching factor is a constant equal to 30 and the first solution is at depth 50. The search tree has $30^{50} \approx 7.2 \times 10^{73}$ leaf nodes. But the number of inference steps is even bigger because not only every leaf node, but also every inner node of the tree corresponds to an inference step. Therefore we must add up the nodes over all levels and obtain the total number of nodes of the search tree

$$\sum_{d=0}^{50} 30^d = \frac{1 - 30^{51}}{1 - 30} = 7.4 \times 10^{73}$$

which does not change the node count by much. Evidently, nearly all of the nodes of this search tree are on the last level. As we will see, this is generally the case. But now back to the search tree with the 7.4×10^{73} nodes. Assume we had 10,000
computers which can each perform a billion inferences per second, and that we could distribute the work over all of the computers with no cost. The total computation time for all 7.4×10^{73} inferences would be approximately equal to

$$\frac{7.4 \times 10^{73} \text{ inferences}}{10000 \times 10^9 \text{ inferences /sec}} = 7.4 \times 10^{60}\text{sec} \approx 2.3 \times 10^{53} \text{ years}$$

which is about 10^{43} times as much time as the age of our universe. By this simple thought exercise, we can quickly recognize that there is no realistic chance of searching this kind of search space completely with the means available to us in this world. Moreover, the assumptions related to the size of the search space were completely realistic. In chess for example, there are over 30 possible moves for a typical situation, and a game lasting 50 half-turns is relatively short.

How can it be then, that there are good chess players-and these days also good chess computers? How can it be that mathematicians find proofs for theorems in which the search space is even much bigger? Evidently we humans use intelligent strategies which dramatically reduce the search space. The experienced chess player, just like the experienced mathematician, will, by mere observation of the situation, immediately rule out many actions as senseless. Through his experience, he has the ability to evaluate various actions for their utility in reaching the goal. Often a person will go by feel. If one asks a mathematician how he found a proof, he may answer that the intuition came to him in a dream. In difficult cases, many doctors find a diagnosis purely by feel, based on all known symptoms. Especially in difficult situations, there is often no formal theory for solution-finding that guarantees an optimal solution. In everyday problems, such as the search for a runaway cat in Fig. 6.1 on page 93, intuition plays a big role. We will deal with this kind of heuristic search method in Sect. 6.3 and additionally describe processes with which computers can, similarly to humans, improve their heuristic search strategies by learning.

First, however, we must understand how uninformed search, that is, blindly trying out all possibilities, works. We begin with a few examples.

Example 6.1 With the 8-puzzle, a classic example for search algorithms [[**?**], RN10], the various algorithms can be very visibly illustrated. Squares with the numbers 1 to 8 are distributed in a 3×3 matrix like the one in Fig. 6.2 on page 93. The goal is to reach a certain ordering of the squares, for example in ascending order by rows as represented in Fig. 6.2 on page 93. In each step a square can be moved left, right, up, or down into the empty space. The empty space therefore moves in the corresponding opposite direction. For analysis of the search space, it is convenient to always look at the possible movements of the empty field.

The search tree for a starting state is represented in Fig. 6.3 on page 94. We can see that the branching factor alternates between two, three, and four. Averaged over two levels at a time, we obtain an average branching factor [1] of $\sqrt{8} \approx 2.83$. We see

Fig. 6.3 Search tree for the 8 -puzzle. Bottom right a goal state in depth 3 is represented. To save space the other nodes at this level have been omitted

that each state is repeated multiple times two levels deeper because in a simple uninformed search, every action can be reversed in the next step.

If we disallow cycles of length 2, then for the same starting state we obtain

[21] The average branching factor of a tree is the branching factor that a tree with a constant branching factor, equal depth, and an equal amount of leaf nodes would have.

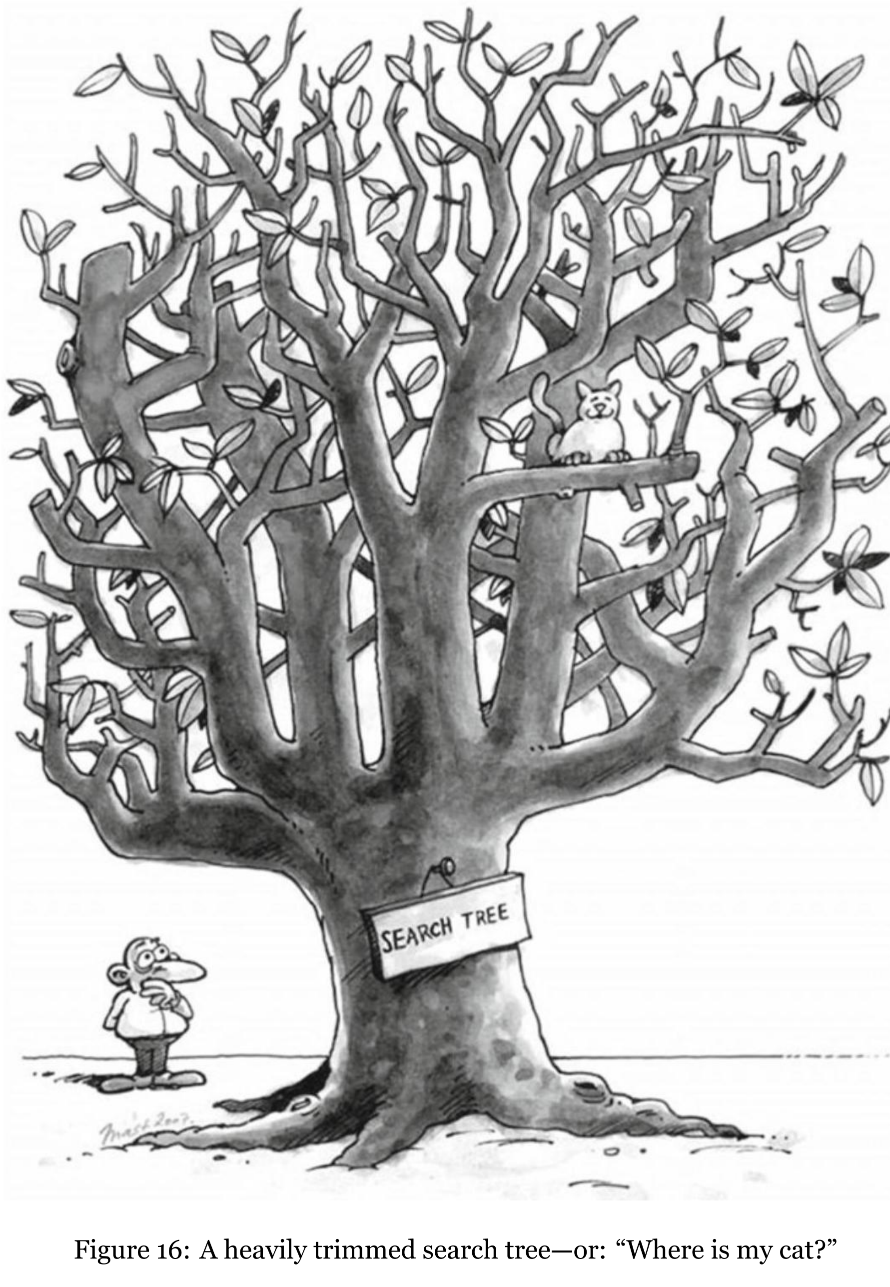


Figure 16: A heavily trimmed search tree—or: “Where is my cat?”

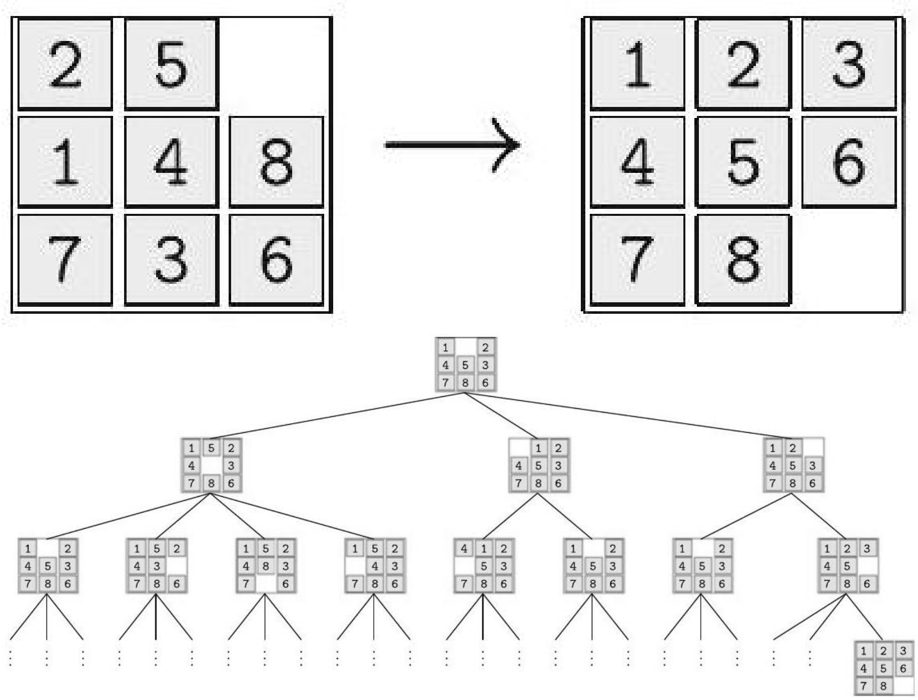


Figure 17: Possible starting and goal states of the 8-puzzle

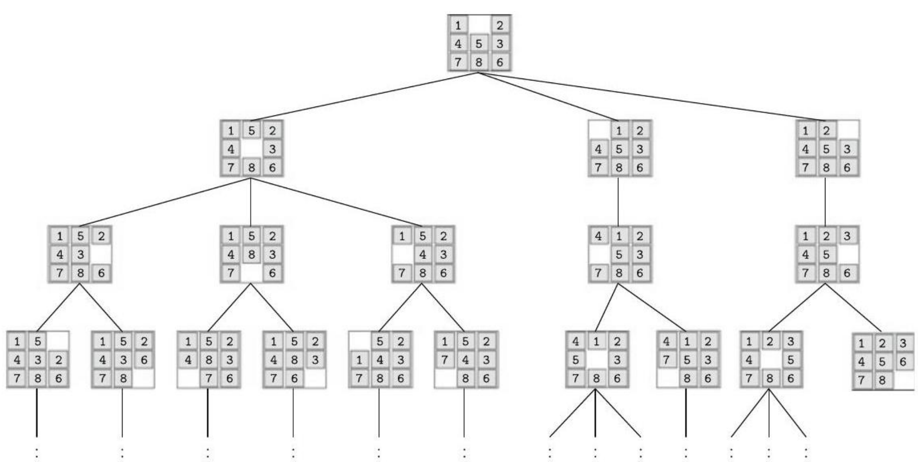


Figure 18: Search tree for an 8-puzzle without cycles of length 2

the search tree represented in Fig. 6.4. The average branching factor is reduced by about 1 and becomes 1.8.[2]

Before we begin describing the search algorithms, a few new terms are needed. We are dealing with discrete search problems here. Being in state s, an action a_1 leads to a new state s'. Thus $s' = a_1(s)$. A different action may lead to state s'', in other words: $s'' = a_2(s)$. Recursive application of all possible actions to all states, beginning with the starting state, yields the search tree.

Definition 6.1 A search problem is defined by the following values
State: Description of the state of the world in which the search agent finds itself.
Starting state: The initial state in which the search agent is started.
Goal state: If the agent reaches a goal state, then it terminates and outputs a solution (if desired).
Actions: All of the agents allowed actions.
Solution: The path in the search tree from the starting state to the goal state.
Cost function: Assigns a cost value to every action. Necessary for finding a cost-optimal solution.
State space: Set of all states.

Applied to the 8-puzzle, we get
State: 3×3 matrix S with the values $1, 2, 3, 4, 5, 6, 7, 8$ (once each) and one empty square.
Starting state: An arbitrary state.
Goal state: An arbitrary state, e.g. the state given to the right in Fig. 6.2 on page 93.
Actions: Movement of the empty square S_{ij} to the left (if $j \neq 1$), right (if $j \neq 3$), up (if $i \neq 1$), down (if $i \neq 3$).
Cost function: The constant function 1, since all actions have equal cost.
State space: The state space is degenerate in domains that are mutually unreachable (Exercise 6.4 on page 122). Thus there are unsolvable 8-puzzle problems.
For analysis of the search algorithms, the following terms are needed:

0.16 Definition

- The number of successor states of a state s is called the branching factor $b(s)$, or b if the branching factor is constant.
- The effective branching factor of a tree of depth d with n total nodes is defined as the branching factor that a tree with constant branching factor, equal depth, and equal n would have (see Exercise 6.3 on page 122).
- A search algorithm is called complete if it finds a solution for every solvable problem. If a complete search algorithm terminates without finding a solution, then the problem is unsolvable.

For a given depth d and node count n, the effective branching factor can be calculated by solving the equation

$$n = \frac{b^{d+1} - 1}{b - 1} \tag{6.1}$$

for b because a tree with constant branching factor and depth d has a total of

$$n = \sum_{i=0}^{d} b^i = \frac{b^{d+1} - 1}{b - 1} \tag{6.2}$$

nodes.

For the practical application of search algorithms for finite search trees, the last level is especially important because

Theorem 6.1 For heavily branching finite search trees with a large constant branching factor, almost all nodes are on the last level.

The simple proof of this theorem is recommended to the reader as an exercise (Exercise 6.1 on page 122).

Example 6.2 We are given a map, such as the one represented in Fig. 6.5, as a graph with cities as nodes and highway connections between the cities as weighted edges with distances. We are looking for an optimal route from city A to city B. The description of the corresponding schema reads

State: A city as the current location of the traveler.
Starting state: An arbitrary city.
Goal state: An arbitrary city.
Actions: Travel from the current city to a neighboring city.
Cost function: The distance between the cities. Each action corresponds to an edge in the graph with the distance as the weight.

State space: All cities, that is, nodes of the graph.

To find the route with minimal length, the costs must be taken into account because they are not constant as they were in the 8-puzzle.

Definition 6.3 A search algorithm is called optimal if it, if a solution exists, always finds the solution with the lowest cost.

The 8-puzzle problem is deterministic, which means that every action leads from a state to a unique successor state. It is furthermore observable, that is, the agent always knows which state it is in. In route planning in real applications both characteristics are not always given. The action "Drive from Munich to Ulm" may—for example because of an accident—lead to the successor state "Munich". It can also occur that the traveler no longer knows where he is because he got lost. We want to ignore these kinds of complications at first. Therefore in this chapter we will only look at problems that are deterministic and observable.

Problems like the 8-puzzle, which are deterministic and observable, make action planning relatively simple because, due to having an abstract model, it

[22] For an 8-puzzle the average branching factor depends on the starting state (see Exercise 6.2 on page 122).

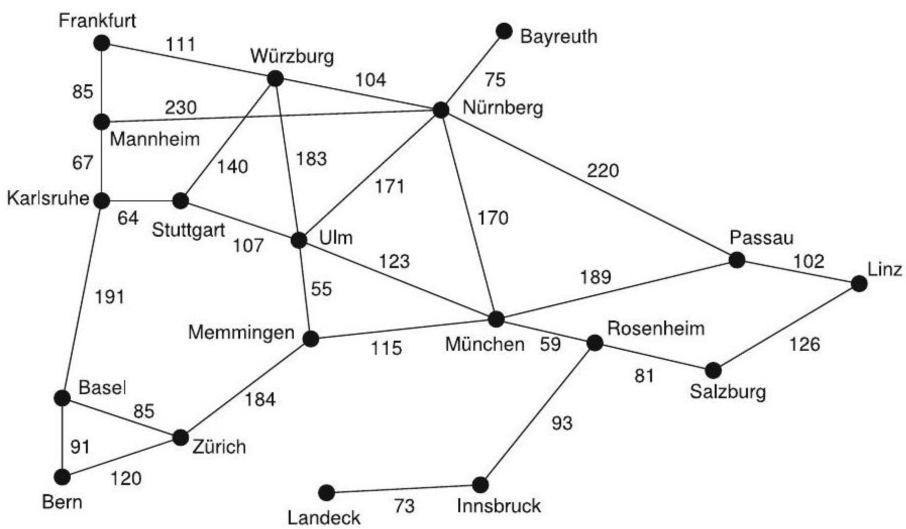


Figure 19: The graph of southern Germany as an example of a search task with a cost function

is possible to find action sequences for the solution of the problem without actually carrying out the actions in the real world. In the case of the 8-puzzle, it is not necessary to actually move the squares in the real world to find the solution. We can find optimal solutions with so-called offline algorithms. One faces much different challenges when, for example, building robots that are supposed to play soccer. Here there will never be an exact abstract model of the actions. For example, a robot that kicks the ball in a specific direction cannot predict with certainty where the ball will move because, among other things, it does not know whether an opponent will catch or deflect the ball. Here online algorithms are then needed, which make decisions based on sensor signals in every situation. Reinforcement learning, described in Chap. 10, works toward optimization of these decisions based on experience.

0.16.1 Breadth-First Search

In breadth-first search, the search tree is explored from top to bottom according to the algorithm given in Fig. 6.6 on page 98 until a solution is found. First every node in the node list is tested for whether it is a goal node, and in the case of success, the program is stopped. Otherwise all successors of the node are generated. The search is then continued recursively on the list of all newly generated nodes. The whole thing repeats until no more successors are generated.

This algorithm is generic. That is, it works for arbitrary applications if the two application-specific functions "GoalReached" and "Successors" are provided.

```
BreadthFirstSearch(NodeList, Goal)
```

```
NewNodes = ∅
For all Node \in NodeList
    If GoalReached(Node, Goal)
        Return("Solution found", Node)
    NewNodes = Append(NewNodes, Successors(Node))
If NewNodes != ∅
    Return(BreadthFirstSearch(NewNodes, Goal))
Else
    Return("No solution")
```

Fig. 6.6 The algorithm for breadth-first search

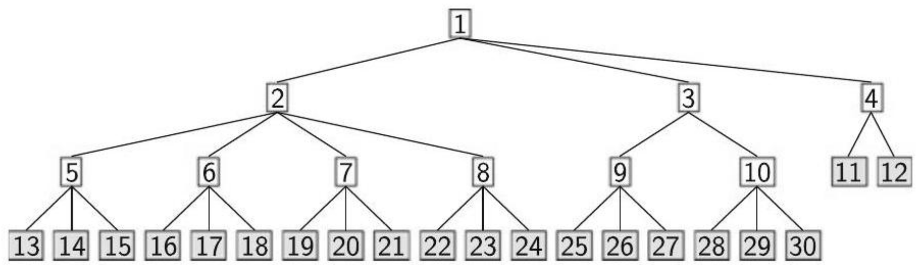


Figure 20: Breadth-first search during the expansion of the third-level nodes. The nodes are numbered according to the order they were generated. The successors of nodes 11 and 12 have not yet been generated

“GoalReached”calculates whether the argument is a goal node, and “Successors”calculates the list of all successor nodes of its argument. Figure 6.7 shows a snapshot of breadth-first search. Analysis Since breadth-first search completely searches through every depth and reaches every depth in finite time, it is complete if the branching factor b is finite. The optimal (that is, the shortest) solution is found if the costs of all actions are the same (see Exercise 6.7 on page 123). Computation time and memory space grow exponentially with the depth of the tree. For a tree with constant branching factor b and depth d, the total compute time is thus given by

$$c \cdot \sum_{i=0}^{d} b^i = \frac{b^{d+1}-1}{b-1} = O\left(b^d\right).$$

Although only the last level is saved in memory, the memory space requirement is also $O\left(b^d\right)$.

With the speed of today’s computers, which can generate billions of nodes within minutes, main memory quickly fills up and the search ends. The problem of the shortest solution not always being found can be solved by the so-called Uniform Cost Search, in which the node with the lowest cost from the list of nodes (which is sorted ascendingly by cost) is always expanded, and the new nodes sorted in. Thus we find the optimal solution. The memory problem is not yet solved, however. A solution for this problem is provided by depth-first search.

0.16.2 Depth-First Search

In depth-first search only a few nodes are stored in memory at one time. After the expansion of a node only its successors are saved, and the first successor node is immediately expanded. Thus the search quickly becomes very deep. Only when a node has no successors and the search fails at that depth is the next open node expanded via backtracking to the last branch, and so on. We can best perceive this in the elegant recursive algorithm in Fig. 6.8 and in the search tree in Fig. 6.9 on page 100.
Analysis Depth-first search requires much less memory than breadth-first search because at most b nodes are saved at each depth. Thus we need at most $b \cdot d$ memory cells.

However, depth-first search is not complete for infinitely deep trees because depth-first search runs into an infinite loop when there is no solution in the far left branch. Therefore the question of finding the optimal solution is obsolete. Because of the infinite loop, no bound on the computation time can be given. In the case of a finitely deep search tree with depth d, a total of about b^d nodes are generated. Thus the computation time grows, just as in breadth-first search, exponentially with depth.

We can make the search tree finite by setting a depth limit. Now if no solution is found in the pruned search tree, there can nonetheless be solutions outside the limit.

```
DEPTHFIRSTSEARCH(Node, Goal)
If GoalReached(Node, Goal) Return("Solution found")
NewNodes = Successors(Node)
While NewNodes # }=
    Result = DEPTH-FIRST-SEARCH(First(NewNodes), Goal)
    If Result = "Solution found" Return("Solution found")
    NewNodes = Rest(NewNodes)
Return("No solution")
```

Fig. 6.8 The algorithm for depth-first search. The function ”First” returns the first element of a list, and ”Rest” the rest of the list

Thus the search becomes incomplete. There are obvious ideas, however, for getting the search to completeness.

0.16.3 Iterative Deepening

We begin the depth-first search with a depth limit of 1. If no solution is found, we raise the limit by 1 and start searching from the beginning, and so on, as

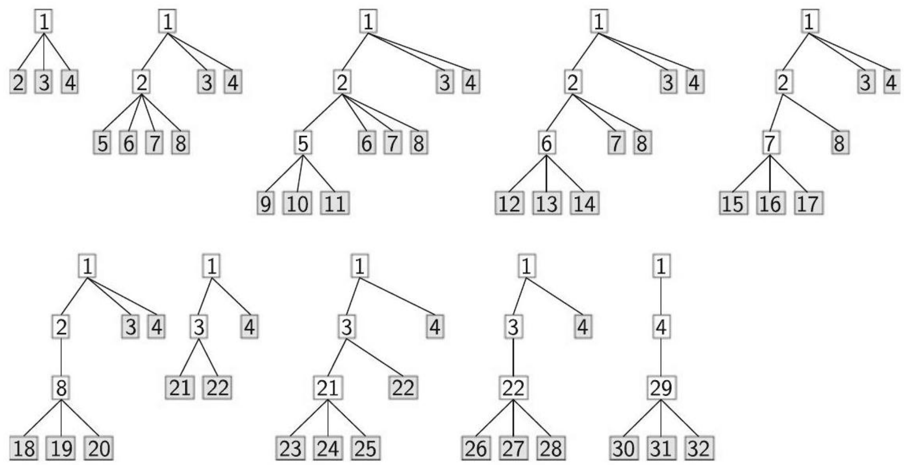


Figure 21: Execution of depth-first search. All nodes at depth three are unsuccessful and cause backtracking. The nodes are numbered in the order they were generated

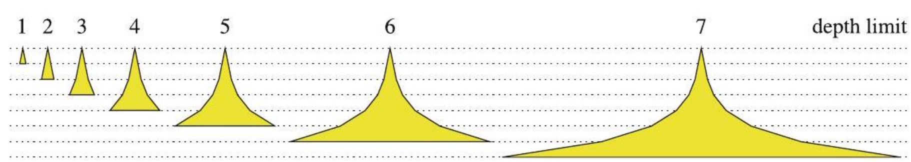


Figure 22: Schematic representation of the development of the search tree in iterative deepening with limits from 1 to 7. The breadth of the tree corresponds to a branching factor of 2

shown in Fig. 6.10. This iterative raising of the depth limit is called iterative deepening.

We must augment the depth-first search program given in Fig. 6.8 on page 99 with the two additional parameters ”Depth” and ”Limit”. ”Depth” is raised by one at the recursive call, and the head line of the while loop is replaced by ”While NewNodes $\neq \emptyset$ **And** Depth $<$ Limit”. The modified algorithm is represented in Fig. 6.11 on page 101.

Analysis The memory requirement is the same as in depth-first search. One could argue that repeatedly re-starting depth-first search at depth zero causes a lot of redundant work. For large branching factors this is not the case. We now show that

IterativeDeepening(Node, Goal)
DepthLimit $= 0$
Repeat
Result = DepthFirstSearch-B(Node, Goal, 0, DepthLimit)
DepthLimit = DepthLimit + 1
Until Result = ”Solution found”
DepthFirstSearch-B(Node, Goal, Depth, Limit)
If GoalReached(Node, Goal) **Return**(”Solution found”)
NewNodes = Successors(Node)
While NewNodes $\neq \emptyset$ **And** Depth $<$ Limit
Result =
DepthFirstSearch-B(First(NewNodes), Goal, Depth + 1, Limit)
If Result = ”Solution found” **Return**(”Solution found”)
NewNodes = Rest(NewNodes)
Return(”No solution”)

Fig. 6.11 The algorithm for iterative deepening, which calls the slightly modified depth-first search with a depth limit (Tiefensuche-B)

the sum of the number of nodes of all depths up to the one before last $d_{\max} - 1$ in all trees searched is much smaller than the number of nodes in the last tree searched. Let $N_b(d)$ be the number of nodes of a search tree with branching factor b and depth d and $d_{\max}$ be the last depth searched. The last tree searched contains

$$N_b\left(d_{\max}\right) = \sum_{i=0}^{d_{\max}} b^i = \frac{b^{d_{\max}+1} - 1}{b - 1}$$

nodes. All trees searched beforehand together have

$$\begin{aligned}\sum_{d=1}^{d_{\max}-1} N_b(d) &= \sum_{d=1}^{d_{\max}-1} \frac{b^{d+1}-1}{b-1} = \frac{1}{b-1}\left(\left(\sum_{d=1}^{d_{\max}-1} b^{d+1}\right) - d_{\max} + 1\right) \\ &= \frac{1}{b-1}\left(\left(\sum_{d=2}^{d_{\max}} b^{d}\right) - d_{\max} + 1\right) \\ &= \frac{1}{b-1}\left(\frac{b^{d_{\max}+1}-1}{b-1} - 1 - b - d_{\max} + 1\right) \\ &\approx \frac{1}{b-1}\left(\frac{b^{d_{\max}+1}-1}{b-1}\right) = \frac{1}{b-1} N_b\left(d_{\max}\right)\end{aligned}$$

nodes. For $b > 2$ this is less than the number $N_b\left(d_{\max}\right)$ of nodes in the last tree. For $b = 20$ the first $d_{\max} - 1$ trees together contain only about $\frac{1}{b-1} = 1/19$ of the number of nodes in the last tree. The computation time for all iterations besides the last can be ignored.

Just like breadth-first search, this method is complete, and given a constant cost for all actions, it finds the shortest solution.

0.16.4 Comparison

The described search algorithms have been put side-by-side in Table 6.1. We can clearly see that iterative deepening is the winner of this test because it gets the best grade in all categories. In fact, of all four algorithms presented it is the only practically usable one.

We do indeed have a winner for this test, although for realistic applications it is usually not successful. Even for the 15-puzzle, the 8-puzzle's big brother (see Exercise 6.4 on page 122), there are about 2×10^{13} different states. For non-trivial inference systems the state space is many orders of magnitude bigger. As shown in Sect. 6.1, all the computing power in the world will not help much more. Instead what is needed is an intelligent search that only explores a tiny fraction of the search space and finds a solution there.

0.16.5 Cycle Check

As shown in Sect. 6.1, nodes may be repeatedly visited during a search. In the 8-puzzle, for example, every move can be immediately undone, which leads to unnecessary cycles of length two. Such cycles can be prevented by recording within each node all of its predecessors and, when expanding a node, comparing the newly created successor nodes with the predecessor nodes. All of the duplicates found can be removed from the list of successor nodes. This simple check costs only a small constant factor of additional memory space and increases the constant computation time c by an additional constant δ for the check itself for a total of $c + \delta$. This overhead for the cycle check is (hopefully) offset by a reduction in the cost of the

Table 6.1 Comparison of the uninformed search algorithms. (*) means that the statement is only true given a constant action cost. d_s is the maximal depth for a finite search tree

	Breadth-first search	Uniform cost search	Depth-first search	Iterative deepening
Completeness	Yes	Yes	No	Yes
Optimal solution	Yes (*)	Yes	No	Yes (*)
Computation time	b^d	b^d	∞ or b^d	b^d
Memory use	b^d	b^d	bd	bd

search. The reduction depends, of course, on the particular application and therefore cannot be given in general terms. For the 8 -puzzle we obtain the result as follows. If, for example, during breadth-first search with effective branching factor b on a finite tree of depth d, the computation time without the cycle check is $c \cdot b^d$, the required time with the cycle check becomes

$$(c + \delta) \cdot (b - 1)^d$$

The check thus practically always results in a clear gain because reducing the branching factor by one has an exponentially growing effect as the depth increases, whereas the additional computation time δ only somewhat increases the constant factor.

Now the question arises as to how a check on cycles of arbitrary length would affect the search performance. The list of all predecessors must now be stored for each node, which can be done very efficiently (see Exercise 6.8 on page 123). During the search, each newly created node must now be compared with all its predecessors. The computation time of depth-first search or breadth-first search is given by

$$c_1 \cdot \sum_{i=0}^{d} b^i + c_2 \cdot \sum_{i=0}^{d} i \cdot b^i$$

Here, the first term is the already-known cost of generating the nodes, and the second term is the cost of the cycle check. We can show that for large values of b and d,

$$\sum_{i=0}^{d} i \cdot b^i \approx d \cdot b^d.$$

The complexity of the search with the full cycle check therefore only increases by a factor of d faster than for the search without a cycle check. In search trees that are not very deep, this extra complexity is not important. For search tasks with very deep, weakly branching trees, it may be advantageous to use a hash table [[**?**]] to store the list of predecessors. Lookups in the table can be done in constant time such that the computation time of the search algorithm only grows by a small constant factor.

In summary, we can conclude that the cycle check implies hardly any additional overhead and is therefore worthwhile for applications with repeatedly occurring nodes.

0.16.6 Heuristic Search

Heuristics are problem-solving strategies which in many cases find a solution faster than uninformed search. However, this is not guaranteed. Heuristic search could require a lot more time and can even result in the solution not being found.

We humans successfully use heuristic processes for all kinds of things. When buying vegetables at the supermarket, for example, we judge the various options for a pound of strawberries using only a few simple criteria like price, appearance, source of production, and trust in the seller, and then we decide on the best option by gut feeling. It might theoretically be better to subject the strawberries to a basic chemical analysis before deciding whether to buy them. For example, the strawberries might be poisoned. If that were the case the analysis would have been worth the trouble. However, we do not carry out this kind of analysis because there is a very high probability that our heuristic selection will succeed and will quickly get us to our goal of eating tasty strawberries. Heuristic decisions are closely linked with the need to make real-time decisions with limited resources. In practice a good solution found quickly is preferred over a solution that is optimal, but very expensive to derive.

A heuristic evaluation function $f(s)$ for states is used to mathematically model a heuristic. The goal is to find, with little effort, a solution to the stated search problem with minimal total cost. Please note that there is a subtle difference between the effort to find a solution and the total cost of this solution. For example it may take Google Maps half a second's worth of effort to find a route from the City Hall in San Francisco to Tuolumne Meadows in Yosemite National Park, but the ride from San Francisco to Tuolumne Meadows by car may take four hours and some money for gasoline etc. (total cost).

Next we will modify the breadth-first search algorithm by adding the evaluation function to it. The currently open nodes are no longer expanded left to right by row, but rather according to their heuristic rating. From the set of open nodes, the node with the minimal rating is always expanded first. This is achieved by immediately evaluating nodes as they are expanded and sorting them into the list of open nodes. The list may then contain nodes from different depths in the tree.

Because heuristic evaluation of states is very important for the search, we will differentiate from now on between states and their associated nodes. The node contains the state and further information relevant to the search, such as its depth in the search tree and the heuristic rating of the state. As a result, the function "Successors", which generates the successors (children) of a node, must also immediately calculate for these successor nodes their heuristic ratings as a component of each node. We define the general search algorithm HeuristicSearch in Fig. 6.12 on page 105.

The node list is initialized with the starting nodes. Then, in the loop, the first node from the list is removed and tested for whether it is a solution node. If not, it will be expanded with the function "Successors" and its successors added to the list with the function "SortIn". "SortIn(X,Y)" inserts the elements from the unsorted list X into the ascendingly sorted list Y. The heuristic rating is used as the sorting key. Thus it is guaranteed that the best node (that is, the one with the lowest heuristic value) is always at the beginning of the list. [3]

```
HeuristicSearch(Start, Goal)
NodeList = [Start]
While True
    If NodeList = ∅ Return("No solution")
    Node = First(NodeList)
    NodeList = Rest(NodeList)
    If GoalReached(Node, Goal) Return("Solution found", Node)
    NodeList = SortIn(Successors(Node),NodeList)
```

Fig. 6.12 The algorithm for heuristic search

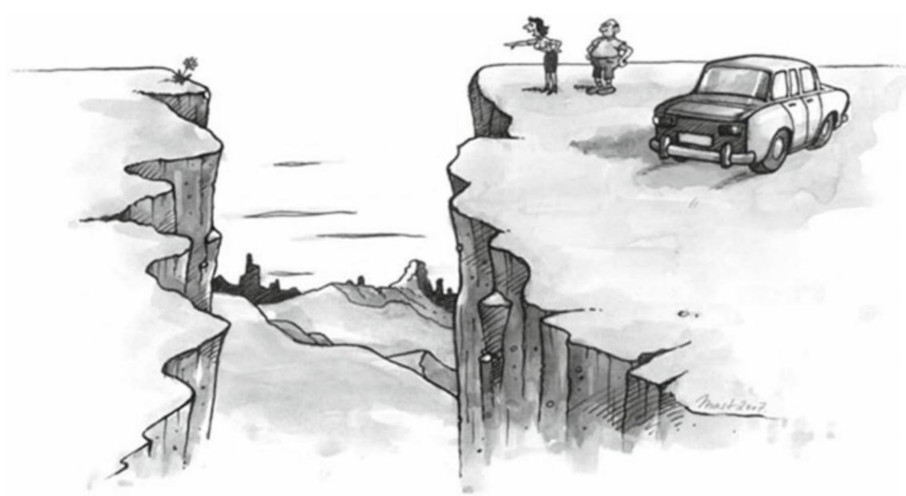


Figure 23: He: "Dear, think of the fuel cost! I'll pluck one for you somewhere else." She: "No, I want that one over there!"

Depth-first and breadth-first search also happen to be special cases of the function HeuristicSearch. We can easily generate them by plugging in the appropriate evaluation function (Exercise 6.11 on page 123).

The best heuristic would be a function that calculates the actual costs from each node to the goal. To do that, however, would require a traversal of the

[23] When sorting in a new node from the node list, it may be advantageous to check whether the node is already available and, if so, to delete the duplicate.

entire search space, which is exactly what the heuristic is supposed to prevent. Therefore we need a heuristic that is fast and simple to compute. How do we find such a heuristic?

An interesting idea for finding a heuristic is simplification of the problem. The original task is simplified enough that it can be solved with little computational cost. The costs from a state to the goal in the simplified problem then serve as an estimate for the actual problem (see Fig. 6.13). This cost estimate function we denote h.

0.16.7 Greedy Search

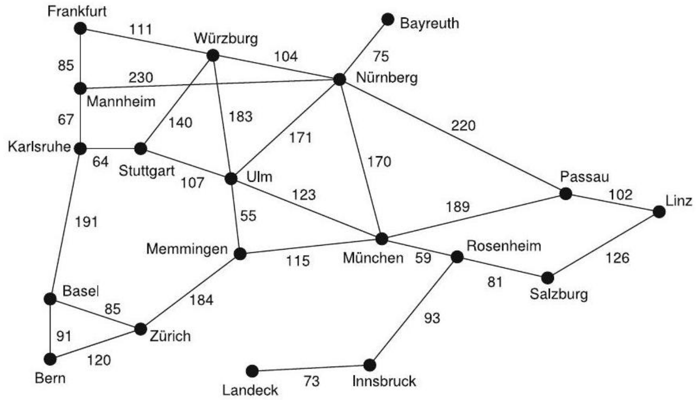
It seems sensible to choose the state with the lowest estimated h value (that is, the one with the lowest estimated cost) from the list of currently available states. The cost estimate then can be used directly as the evaluation function. For the evaluation in the function HeuristicSearch we set $f(s) = h(s)$. This can be seen clearly in the trip planning example (Example 6.2 on page 96). We set up the task of finding the straight line path from city to city (that is, the flying distance) as a simplification of the problem. Instead of searching the optimal route, we first determine from every node a route with minimal flying distance to the goal. We choose Ulm as the destination. Thus the cost estimate function becomes

$$h(s) = \text{flying distance from city } s \text{ to Ulm.}$$

The flying distances from all cities to Ulm are given in Fig. 6.14 next to the graph.
The search tree for starting in Linz is represented in Fig. 6.15 on page 107 left. We can see that the tree is very slender. The search thus finishes quickly. Unfortunately, this search does not always find the optimal solution. For example, this algorithm fails to find the optimal solution when starting in Mannheim (Fig. 6.15 on page 107 right). The Mannheim-Nürnberg-Ulm path has a length of 401 km . The route Mannheim-Karlsruhe-Stuttgart-Ulm would be significantly shorter at 238 km . As we observe the graph, the cause of this problem becomes clear. Nürnberg is in fact somewhat closer than Karlsruhe to Ulm, but the distance from Mannheim to Nürnberg is significantly greater than that from Mannheim to Karlsruhe. The heuristic only looks ahead "greedily" to the goal instead of also taking into account the stretch that has already been laid down to the current node. This is why we give it the name greedy search.

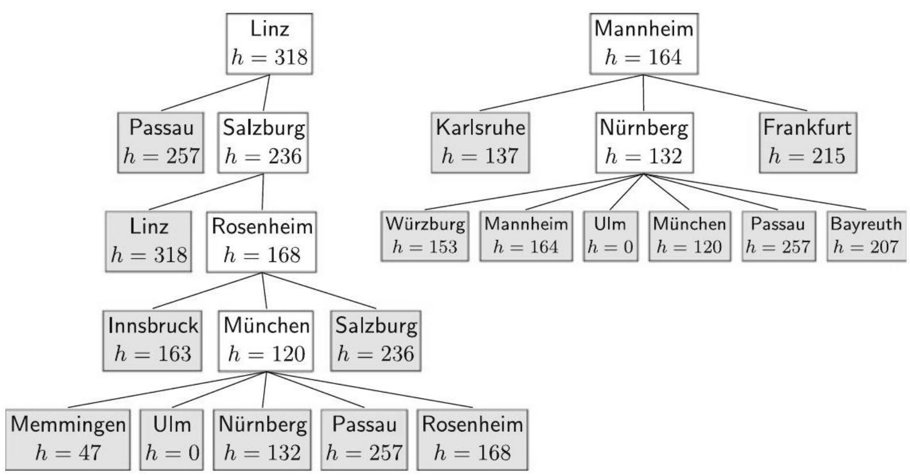
Node	München	Innsbruck	Salzburg	Passau	Linz
heuristic rating	120	163	236	257	318

Fig. 6.15 Greedy search: from Linz to Ulm (left) and from Mannheim to Ulm (right). The node list data structure for the left search tree, sorted by the node rating before the expansion of the node München is given



Basel	204
Bayreuth	207
Bern	247
Frankfurt	215
Innsbruck	163
Karlsruhe	137
Landeck	143
Linz	318
München	120
Mannheim	164
Memmingen	47
Nürnberg	132
Passau	257
Rosenheim	168
Stuttgart	75
Salzburg	236
Würzburg	153
Zürich	157

Figure 24: City graph with flying distances from all cities to Ulm



0.16.8 A *-Search

We now want to take into account the costs that have accrued during the search up to the current node s. First we define the cost function

$$g(s) = \text{Sum of accrued costs from the start to the current node,}$$

then add to that the estimated cost to the goal and obtain as the heuristic evaluation function

$$f(s) = g(s) + h(s)$$

Now we add yet another small, but important requirement.

Definition 6.4 A heuristic cost estimate function $h(s)$ that never overestimates the actual cost from state s to the goal is called admissible.

The function HeuristicSearch together with an evaluation function $f(s) = g(s) + h(s)$ and an admissible heuristic function h is called A*-algorithm. This famous algorithm is complete and optimal. $A^\star$ thus always finds the shortest solution for every solvable search problem. We will explain and prove this in the following discussion.

First we apply the $A^\star$-algorithm to the example. We are looking for the shortest path from Frankfurt to Ulm. In the top part of Fig. 6.16 we see that the successors of Mannheim are generated before the successors of Würzburg. The optimal solution Frankfurt-Würzburg-Ulm is generated shortly thereafter in the eighth step, but it is not yet recognized as such. Thus the algorithm does not terminate yet because the node Karlsruhe (3) has a better (lower) f value and thus is ahead of the node Ulm (8) in line. Only when all f values are greater than or equal to that of the solution node Ulm (8) have we ensured that we have an optimal solution. Otherwise there could potentially be another solution with lower costs. We will now show that this is true generally.

Theorem 6.2 The $A^\star$ algorithm is optimal. That is, it always finds the solution with the lowest total cost if the heuristic h is admissible.

Proof In the HeuristicSearch algorithm, every newly generated node s is sorted in by the function "SortIn" according to its heuristic rating $f(s)$. The node with the

smallest rating value thus is at the beginning of the list. If the node l at the beginning of the list is a solution node, then no other node has a better heuristic rating. For all other nodes s it is true then that $f(l) \leq f(s)$. Because the heuristic is admissible, no better solution l' can be found, even after expansion of all other nodes (see Fig. 6.17). Written formally:

$$g(l) = g(l) + h(l) = f(l) \leq f(s) = g(s) + h(s) \leq g\left(l'\right)$$

The first equality holds because l is a solution node with $h(l) = 0$. The second is the definition of f. The third (in)equality holds because the list of open nodes is sorted in ascending order. The fourth equality is again the definition of f. Finally, the last (in)equality is the admissibility of the heuristic, which never overestimates the cost from node s to an arbitrary solution. Thus it has been shown that $g(l) \leq g\left(l'\right)$, that is, that the discovered solution l is optimal.

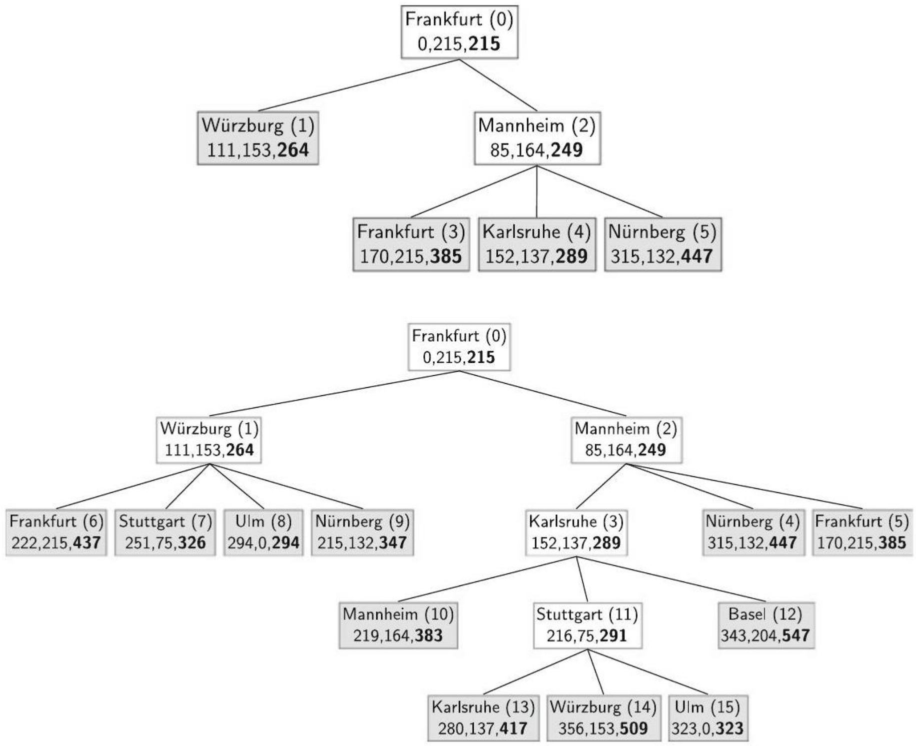


Figure 25: Two snapshots of the $A^\star$ search tree for the optimal route from Frankfurt to Ulm. In the boxes below the name of the city s we show $g(s), h(s), f(s)$. Numbers in parentheses after the city names show the order in which the nodes have been generated by the "Successor" function.

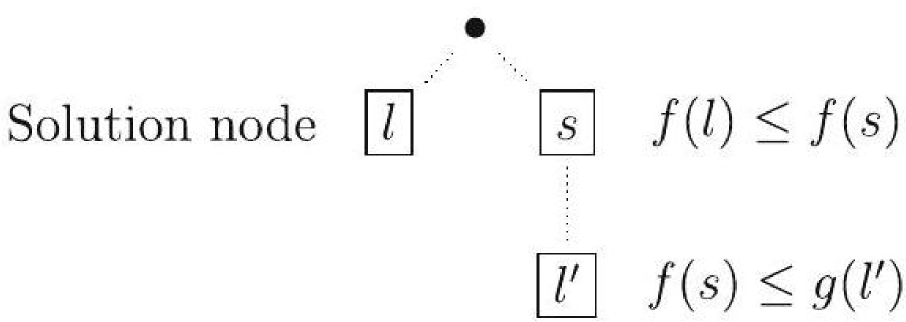


Figure 26: The first solution node l found by $A^\star$ never has a higher cost than another arbitrary node l'

0.16.9 Route Planning with the A* Search Algorithm

Many current car navigation systems use the $A^\star$ algorithm. The simplest, but very good heuristic for computing A* is the straight-line distance from the current node to the destination. The use of 5 to 60 so-called landmarks is somewhat better. For these randomly chosen points the shortest paths to and from all nodes on the map are calculated in a precomputation step. Let l be such a landmark, s the current node, and z the destination node. Also let $c^\star(x, y)$ be the cost of the shortest path from x to y. Then we obtain for the shortest path from s to l the triangle inequality (see Exercise 6.11 on page 123)

$$c^\star(s, l) \leq c^\star(s, z) + c^\star(z, l).$$

Solving for $c^\star(s, z)$ results in the admissible heuristic

$$h(s) = c^\star(s, l) - c^\star(z, l) \leq c^\star(s, z).$$

In [[**?**]], it was shown that this heuristic is better than the straight-line distance for route planning. On one hand, it can be calculated faster than the straight-line distance. Due to precomputation, distances to the landmarks can be quickly retrieved from an array, whereas the Euclidean distances must be computed individually. It turns out that the landmark heuristic shrinks the search space even more. This can be seen in the left image of Fig. 6.18, which illustrates the search tree of $A^\star$ search for planning a route from Ravensburg to Biberach (two towns in southern Germany). [4] The edges without any heuristic (i.e. with $h(s) = 0$) are plotted in red colour, dark green lines show the search tree using the straight-line distance heuristic, and the edges of the landmark heuristic with twenty landmarks are plotted in blue.

The right image shows the same route using bidirectional search, where a route from Ravensburg to Biberach and one in the opposite direction are planned effectively in parallel. If the routes meet, given certain conditions of the heuristic, an optimal route has been found [Bat16]. A quantitative analysis of the search tree sizes and the computation time on a PC can be found in Table 6.2.

Table 2: Comparison of search tree size and computation time for route planning with and without each of the two heuristics. The landmark heuristic is the clear winner

	Unidirectional		Bidirectional	
	Tree Size [nodes]	Comp. time [msec.]	Tree Size [nodes]	Comp. time [m
No heuristic	62000	192	41850	122
Straight-line distance	9380	86	12193	84
Landmark heuristic	5260	16	7290	16

[24] Both graphs in Fig. 6.18 were generated by A. Batzill using the system described in [Bat16].

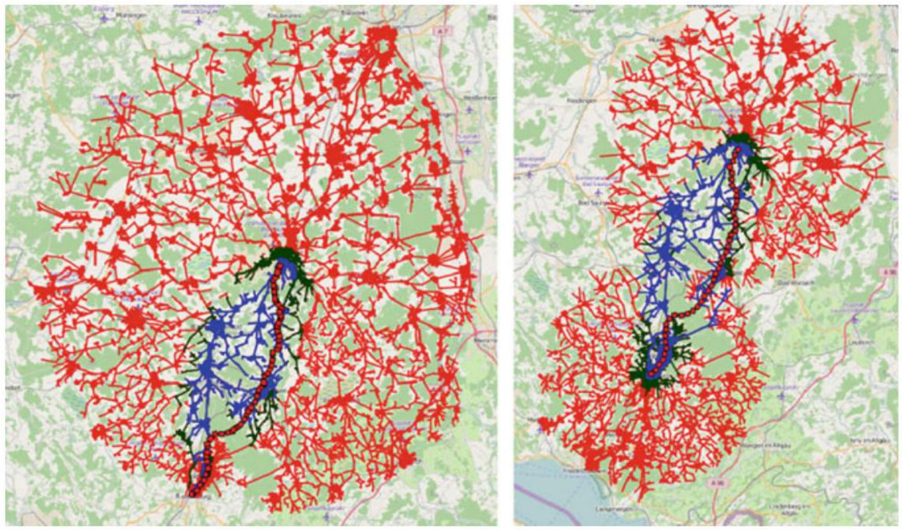


Figure 27: A* search tree without heuristic (red), with straight-line distance (dark green) and with landmarks (blue). The left image shows unidirectional search and the right shows bidirectional search. Note that the green edges are covered by blue and the red edges are covered by green and blue

Observing unidirectional search, we see that both heuristics clearly reduce the search space. The computation times are truly interesting. In the case of the landmark heuristic, we see the computation time and the size of the search space reduced by a factor of about 12. The cost of computing the heuristic is thus insignificant. The straight-line distance, however, results in a search space reduction of a factor of 6.6, but only an improvement of a factor of 2.2 in run time due to the overhead of computing the euclidean distance. In the case of bidirectional search, in contrast to unidirectional search, we see a significant reduction of the search space even without heuristic. On the other hand, the search space is larger than the unidirectional case for both heuristics. However, because the nodes are partitioned into two sorted lists in bidirectional search (see HeuristicSearch function in Fig. 6.12 on page 105), the lists are handled faster and the resulting computation times are roughly the same [Bat16].

When planning a route, usually the driver cares more about driving time than the distance driven. We should thus adjust the heuristic accordingly and replace straight-line distance $d(s, z)$ with time $t(s, z) = d(s, z)/v_{\max}$. Here we have to divide by the maximum average velocity, which degrades the heuristic because it causes the heuristically estimated times to be much too small. The landmark heuristic, in contrast, builds on precomputed optimal routes and therefore does not degrade. Thus, as shown in [Bat16], the search for a time-optimized route using landmark heuristic is significantly faster than with the modified straight-line distance.

The contraction hierarchies algorithm performs even better than $A^\star$ with landmark heuristic. It is based on the idea of combining, in a precompution step, several edges into so-called shortcuts, which are then used to reduce the search space [GSSD08, Bat16].

0.16.10 IDA *-Search

The $A^\star$ search inherits a quirk from breadth-first search. It has to save many nodes in memory, which can lead to very high memory use. Furthermore, the list of open nodes must be sorted. Thus insertion of nodes into the list and removal of nodes from the list can no longer run in constant time, which increases the algorithm's complexity slightly. Based on the heapsort algorithm, we can structure the node list as a heap with logarithmic time complexity for insertion and removal of nodes (see [CLR90]).

Both problems can be solved—similarly to breadth-first search—by iterative deepening. We work with depth-first search and successively raise the limit. However, rather than working with a depth limit, here we use a limit for the heuristic evaluation $f(s)$. This process is called the IDA *-algorithm.

0.16.11 Empirical Comparison of the Search Algorithms

In $A^\star$, or (alternatively) IDA *, we have a search algorithm with many good properties. It is complete and optimal. It can thus be used without risk. The most important thing, however, is that it works with heuristics, and therefore can significantly reduce the computation time needed to find a solution. We would like to explore this empirically in the 8-puzzle example. For the 8 -puzzle there are two simple admissible heuristics. The heuristic h_1 simply counts the number of squares that are not in the right place. Clearly this heuristic is admissible. Heuristic h_2 measures the Manhattan distance. For every square the horizontal and vertical distances to that square's location in the goal state are added together. This value is then summed over all squares. For example, the Manhattan distance of the two states

2	5	
1	4	8
7	3	6

and

1	2	3
4	5	6
7	8	

is calculated as

$$h_2(s) = 1 + 1 + 1 + 1 + 2 + 0 + 3 + 1 = 10.$$

The admissibility of the Manhattan distance is also obvious (see Exercise 6.13 on page 123).

The described algorithms were implemented in Mathematica. For a comparison with uninformed search, the A* algorithm with the two heuristics h_1 and h_2 and iterative deepening was applied to 132 randomly generated 8 -puzzle problems. The average values for the number of steps and computation time are given in Table 6.3. We see that the heuristics significantly reduce the search cost compared to uninformed search.

If we compare iterative deepening to A* with h_1 at depth 12 , for example, it becomes evident that h_1 reduces the number of steps by a factor of about 3,000 , but

Table 3: Comparison of the computation cost of uninformed search and heuristic search for solvable 8-puzzle problems with various depths. Measurements are in steps and seconds. All values are averages over multiple runs (see last column)

Depth	Iterative deepening		A* algorithm				Num. runs
	Steps	Time [sec]	Heuristic h_1		Heuristic h_2		
			Steps	Time [sec]	Steps	Time [sec]	
2	20	0.003	3.0	0.0010	3.0	0.0010	10
4	81	0.013	5.2	0.0015	5.0	0.0022	24
6	806	0.13	10.2	0.0034	8.3	0.0039	19
8	6455	1.0	17.3	0.0060	12.2	0.0063	14
10	50512	7.9	48.1	0.018	22.1	0.011	15
12	486751	75.7	162.2	0.074	56.0	0.031	12
			IDA*				
14	-	-	10079.2	2.6	855.6	0.25	16
16	-	-	69386.6	19.0	3806.5	1.3	13
18	-	-	708780.0	161.6	53941.5	14.1	4

the computation time by only a factor of 1,023 . This is due to the higher cost per step for the computation of the heuristic. Closer examination reveals a jump in the number of steps between depth 12 and depth 14 in the column for h_1. This jump cannot be explained solely by the repeated work done by IDA*. It comes about because the implementation of the A^* algorithm deletes duplicates of identical nodes and thereby shrinks the search space. This is not possible with IDA* because it saves almost no nodes. Despite this, A^* can no longer compete with IDA* beyond depth 14 because the cost of sorting in new nodes pushes up the time per step so much.

A computation of the effective branching factor according to (6.1) on page 96 yields values of about 2.8 for uninformed search. This number is consistent with the value from Sect. 6.1. Heuristic h_1 reduces the branching factor to values of about 1.5 and h_2 to about 1.3. We can see in the table that a small reduction of the branching factor from 1.5 to 1.3 gives us a big advantage in computation time.

Heuristic search thus has an important practical significance because it can solve problems which are far out of reach for uninformed search.

0.16.12 Summary

Of the various search algorithms for uninformed search, iterative deepening is the only practical one because it is complete and can get by with very little memory. However, for difficult combinatorial search problems, even iterative deepening usually fails due to the size of the search space. Heuristic search helps here through its reduction of the effective branching factor. The IDA*-algorithm, like iterative deepening, is complete and requires very little memory.

Heuristics naturally only give a significant advantage if the heuristic is "good". When solving difficult search problems, the developer's actual task consists of designing heuristics which greatly reduce the effective branching factor. In Sect. 6.5 we will deal with this problem and also show how machine learning techniques can be used to automatically generate heuristics.

In closing, it remains to note that heuristics have no performance advantage for unsolvable problems because the unsolvability of a problem can only be established when the complete search tree has been searched through. For decidable problems such as the 8-puzzle this means that the whole search tree must be traversed up to a maximal depth whether a heuristic is being used or not. The heuristic is always a disadvantage in this case, attributable to the computational cost of evaluating the heuristic. This disadvantage can usually be estimated by a constant factor independent of the size of the problem. For undecidable problems such as the proof of PL1 formulas, the search tree can be infinitely deep. This means that, in the unsolvable case, the search potentially never ends. In summary we can say the following: for solvable problems, heuristics often reduce computation time dramatically, but for unsolvable problems the cost can even be higher with heuristics.

0.16.13 Games with Opponents

Games for two players, such as chess, checkers, Othello, and Go are deterministic because every action (a move) results in the same child state given the same parent state. In contrast, backgammon is non-deterministic because its child state depends on the result of a dice roll. These games are all observable because every player always knows the complete game state. Many card games, such as poker, for example, are only partially observable because the player does not know the other players' cards, or only has partial knowledge about them. The problems discussed so far in this chapter were deterministic and observable. In the following we will look at games which, too, are deterministic and observable. Furthermore, we will limit ourselves to zero-sum games. These are games in which every gain one player makes means a loss of the same value for the opponent. The sum of the gain and loss is always equal to zero. This is true of the games chess, checkers, Othello, and Go, mentioned above.

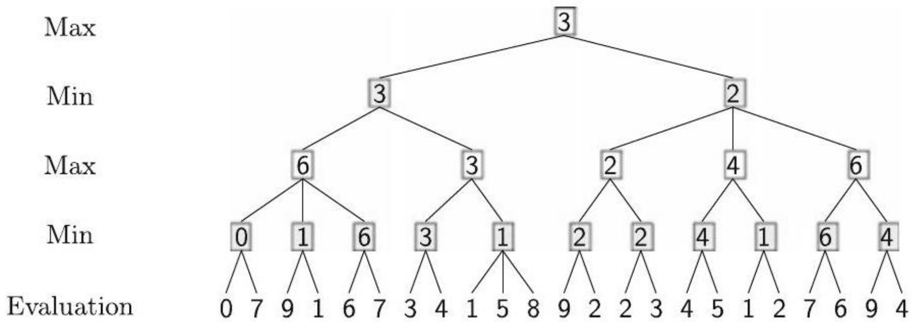


Figure 28: A minimax game tree with look-ahead of four half-moves

0.16.14 Minimax Search

The goal of each player is to make optimal moves that result in victory. In principle it is possible to construct a search tree and completely search through it (like with the 8 -puzzle) for a series of moves that will result in victory. However, there are several peculiarities to watch out for:

1. The effective branching factor in chess is around 30 to 35 . In a typical game with 50 moves per player, the search tree has more than $30^{100} \approx 10^{148}$ leaf nodes. Thus there is no chance to fully explore the search tree. Additionally, chess is often played with a time limit. Because of this real-time requirement, the search must be limited to an appropriate depth in the tree, for example eight half-moves. Since among the leaf nodes of this depth-limited tree there are normally no solution nodes (that is, nodes which terminate the game) a heuristic evaluation function B for board positions is used. The level of play of the program strongly depends on the quality of this evaluation function. Therefore we will further treat this subject in Sect. 6.5.

2. In the following we will call the player whose game we wish to optimize Max, and his opponent Min. The opponent's (Min's) moves are not known in advance, and thus neither is the actual search tree. This problem can be elegantly solved by assuming that the opponent always makes the best move he can. The higher the evaluation $B(s)$ for position s, the better position s is for the player Max and the worse it is for his opponent Min. Max tries to maximize the evaluation of his moves, whereas Min makes moves that result in as low an evaluation as possible. A search tree with four half-moves and evaluations of all leaves is given in Fig. 6.19 on page 115. The evaluation of an inner node is derived recursively as the maximum or minimum of its child nodes, depending on the node's level.

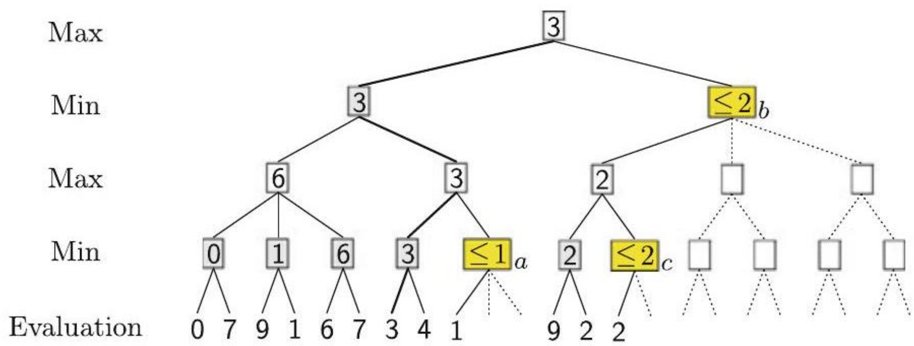


Figure 29: An alpha-beta game tree with look-ahead of four half-moves. The dotted portions of the tree are not traversed because they have no effect on the end result

0.16.15 Alpha-Beta-Pruning

By switching between maximization and minimization, we can save ourselves a lot of work in some circumstances. Alpha-beta pruning works with depth-first search up to a preset depth limit. In this way the search tree is searched through from left to right. Like in minimax search, in the minimum nodes the minimum is generated from the minimum value of the successor nodes and in the maximum nodes likewise the maximum. In Fig. 6.20 this process is depicted for the tree from Fig. 6.19. At the node marked a, all other successors can be ignored after the first child is evaluated as the value 1 because the minimum is sure to be ≤ 1. It could even become smaller still, but that is irrelevant since the maximum is already ≥ 3 one level above. Regardless of how the evaluation of the remaining successors turns out, the maximum will keep the value 3. Analogously the tree will be trimmed at node b. Since the first child of b has the value 2, the minimum to be generated for b can only be less than or equal to 2. But the maximum at the root node is already sure to be ≥ 3. This cannot be changed by values ≤ 2. Thus the remaining subtrees of b can be pruned. The same reasoning applies for the node c. However, the relevant maximum node is not the direct parent, but the root node. This can be generalized.

AlphaBetaMax(Node, α, β)
If DepthLimitReached(Node) **Return** (Rating(Node))
NewNodes = Successors(Node)
While NewNodes $\neq \varnothing$
  $\alpha =$ Maximum$\big(\alpha,$ AlphaBetaMin(First$(NewNodes), \alpha, \beta)\big)$
  If $\alpha \geq \beta$ **Return** (β)
  NewNodes = Rest(NewNodes)
Return (α)

AlphaBetaMin(Node, α, β)

If DepthLimitReached(Node) **Return** (Rating(Node))
NewNodes = Successors(Node)
While NewNodes $\neq \varnothing$
  $\beta =$ Minimum$\big(\beta,$ AlphaBetaMax(First$(NewNodes), \alpha, \beta)\big)$
  If $\beta \leq \alpha$ **Return** (α)
  NewNodes = Rest(NewNodes)
Return (β)

Fig. 6.21 The algorithm for alpha-beta search with the two functions AlphaBetaMin and AlphaBetaMax

- At every leaf node the evaluation is calculated.
- For every maximum node the current largest child value is saved in α.
- For every minimum node the current smallest child value is saved in β.
- If at a minimum node k the current value $\beta \leq \alpha$, then the search under k can end. Here α is the largest value of a maximum node in the path from the root to k.
- If at a maximum node l the current value $\alpha \geq \beta$, then the search under l can end. Here β is the smallest value of a minimum node in the path from the root to l.

The algorithm given in Fig. 6.21 is an extension of depth-first search with two functions which are called in alternation. It uses the values defined above for α and β.

The initial alpha-beta pruning call is done with the command AlphaBetaMax(RootNode, $-\infty, \infty$).

Complexity The computation time saved by alpha-beta pruning heavily depends on the order in which child nodes are traversed. In the worst case, alphabeta pruning does not offer any advantage. For a constant branching factor b the number n_d of leaf nodes to evaluate at depth d is equal to

$$n_d = b^d$$

In the best case, when the successors of maximum nodes are descendingly sorted and the successors of minimum nodes are ascendingly sorted, the effective branching factor is reduced to $\sqrt{b}$. In chess this means a substantial reduction of the effective branching factor from 35 to about 6. Then only

$$n_d = (\sqrt{b})^d = b^{d/2}$$

leaf nodes would be created. This means that the depth limit and thus also the search horizon are doubled with alpha-beta pruning. However, this is only true in the case of optimally sorted successors because the child nodes' ratings are unknown at the time when they are created. If the child nodes are randomly sorted, then the branching factor is reduced to $b^{3/4}$ and the number of leaf nodes to

$$n_d = b^{\frac{3}{4}d}.$$

With the same computing power a chess computer using alpha-beta pruning can, for example, compute eight half-moves ahead instead of six, with an effective branching factor of about 14. A thorough analysis with a derivation of these parameters can be found in [**?**].

To double the search depth as mentioned above, we would need the child nodes to be optimally ordered, which is not the case in practice. Otherwise the search would be unnecessary. With a simple trick we can get a relatively good node ordering. We connect alpha-beta pruning with iterative deepening over the depth limit. Thus at every new depth limit we can access the ratings of all nodes of previous levels and order the successors at every branch. Thereby we reach an effective branching factor of roughly 7 to 8, which is not far from the theoretical optimum of $\sqrt{35}$ [**?**].

0.16.16 Non-deterministic Games

Minimax search can be generalized to all games with non-deterministic actions, such as backgammon. Each player rolls before his move, which is influenced by the result of the dice roll. In the game tree there are now therefore three types of levels in the sequence

Max, dice, Min, dice, ...,

where each dice roll node branches six ways. Because we cannot predict the value of the die, we average the values of all rolls and conduct the search as described with the average values from [**?**].

0.16.17 Heuristic Evaluation Functions

How do we find a good heuristic evaluation function for the task of searching? Here there are fundamentally two approaches. The classical way uses the knowledge of human experts. The knowledge engineer is given the usually difficult task of formalizing the expert's implicit knowledge in the form of a computer program. We now want to show how this process can be simplified in the chess program example. In the first step, experts are questioned about the most important factors in the selection of a move. Then it is attempted to quantify these factors. We obtain a list of relevant features or attributes. These are then (in the simplest case) combined into a linear evaluation function $B(s)$ for positions, which could look like:

$$\begin{aligned} B(s) = a_1 \cdot \text{material} + a_2 \cdot \text{pawn\_structure} + a_3 \cdot \text{king\_safety} \\ + a_4 \cdot \text{knight\_in\_center} + a_5 \cdot \text{bishop\_diagonal\_coverage} + \cdots, \end{aligned} \tag{6.3}$$

where "material" is by far the most important feature and is calculated by

$$\text{material} = \text{material(own\_team)} - \text{material(opponent)}$$

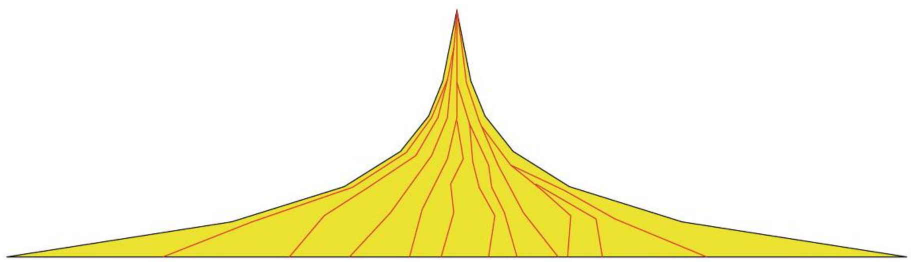


Figure 30: In this sketch of a search tree, several MCTS paths to leaf nodes are shown in red. Notice that only a small part of the tree is searched.

with

$$\begin{aligned}\text{material}(\text{team}) = &\ \text{num\_pawns}(\text{team}) \cdot 100 + \text{num\_knights}(\text{team}) \cdot 300 \\ &+ \text{num\_bishops}(\text{team}) \cdot 300 + \text{num\_rooks}(\text{team}) \cdot 500 \\ &+ \text{num\_queens}(\text{team}) \cdot 900\end{aligned}$$

Nearly all chess programs make a similar evaluation for material. However, there are big differences for all other features, which we will not go into here [[**?**], [**?**]].

In the next step the weights a_i of all features must be determined. These are set intuitively after discussion with experts, then changed after each game based on positive and negative experience. The fact that this optimization process is very expensive and furthermore that the linear combination of features is very limited suggests the use of machine learning.

0.16.18 Learning of Heuristics

We now want to automatically optimize the weights a_i of the evaluation function $B(\text{ s})$ from (6.3). In this approach the expert is only asked about the relevant features $f_1(s), \ldots, f_n(s)$ for game state s. Then a machine learning process is used with the goal of finding an evaluation function that is as close to optimal as possible. We start with an initial pre-set evaluation function (determined by the learning process), and then let the chess program play. At the end of the game a rating is derived from the result (victory, defeat, or draw). Based on this rating, the evaluation function is

changed with the goal of making fewer mistakes next time. In principle, the same thing that is done by the developer is now being taken care of automatically by the learning process.

As easy as this sounds, it is very difficult in practice. A central problem with improving the position rating based on won or lost matches is known today as the credit assignment problem. We do in fact have a rating at the end of the game, but no ratings for the individual moves. Thus the agent carries out many actions but does not receive any positive or negative feedback until the very end. How should it then assign this feedback to the many actions taken in the past? And how should it improve its actions in that case? The exciting field of reinforcement learning deals with these questions (see Chap. 10).

Monte Carlo tree search (MCTS) [[**?**]] works quite similarly. To improve the heuristic rating of a game state s, a random number of search tree branches starting from this state are either explored to the end and evaluated, or stopped at a certain depth and then the leaf nodes are evaluated heuristically. The evaluation $B(s)$ of state s is given as the mean of all leaf node scores. The use of MCTS paths requires only a small part of the entire exponentially exploding tree to be searched. This is illustrated in Fig. 6.22. For many computer-simulated games, such as chess, this algorithm can be used to achieve better play for the same computational effort or to reduce computational effort for the same difficulty level [KS06]. This method was used together with machine learning algorithms in 2016 by the program AlphaGo, described in Sect. 10.10, which was the first Go program to defeat world-class human players [SHM$^+$16].

0.16.19 State of the Art

For evaluation of the quality of the heuristic search processes, I would like to repeat Elaine Rich's definition [Ric83]:

Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.

There is hardly a better suited test for deciding whether a computer program is intelligent as the direct comparison of computer and human in a game like chess, checkers, backgammon or Go.

In 1950, Claude Shannon, Konrad Zuse, and John von Neumann introduced the first chess programs, which, however, could either not be implemented or would take a great deal of time to implement. Just a few years later, in 1955, Arthur Samuel wrote a program that played checkers and could improve its own parameters through a simple learning process. To do this he used the first programmable logic computer, the IBM 701. Compared to the chess computers of today, however, it had access to a large number of archived games, for which every individual move had been rated by experts. Thus the program improved its evaluation function. To achieve further improvements, Samuel had his program play against itself. He solved the credit assignment problem in a simple manner. For each individual position during a game it compares the evaluation by the function $B(s)$ with the one calculated by alpha-beta pruning and changes $B(s)$ accordingly. In 1961 his checkers program beat the fourth-best checkers player in the USA. With this ground-breaking work, Samuel was surely nearly 30 years ahead of his time.

Only at the beginning of the nineties, as reinforcement learning emerged, did Gerald Tersauro build a learning backgammon program named TD-Gammon, which played at the world champion level (see Chap. 10).

0.16.20 Chess

Today many chess programs exist that play above grandmaster level. The breakthrough came in 1997, as IBM's Deep Blue defeated the chess world champion Gary Kasparov with a score of 3.5 games to 2.5. Deep Blue could on average compute 12 half-moves ahead with alpha-beta pruning and heuristic position evaluation.

Around the year 2005 one of the most powerful chess computers was Hydra, a parallel computer owned by a company in the United Arab Emirates. The software was developed by the scientists Christian Donninger (Austria) and Ulf Lorenz (Germany), as well as the German chess grand champion Christopher Lutz. Hydra uses 64 parallel Xeon processors with about 3 GHz computing power and 1 GByte memory each. For the position evaluation function each processor has an FPGA (field programmable gate array) co-processor. Thereby it becomes possible to evaluate 200 million positions per second even with an expensive evaluation function.

With this technology Hydra can on average compute about 18 moves ahead. In special, critical situations the search horizon can even be stretched out to 40 half-moves. Clearly this kind of horizon is beyond what even grand champions can do, for Hydra often makes moves which grand champions cannot comprehend, but which in the end lead to victory. In 2005 Hydra defeated seventh ranked grandmaster Michael Adams with 5.5-0.5 games.

Hydra uses little special textbook knowledge about chess, rather alpha-beta search with relatively general, well-known heuristics and a good hand-coded position evaluation. In particular, Hydra is not capable of learning. Improvements are carried out between games by the developers. As a consequence, Hydra was soon outperformed by machines that used smart learning algorithms rather than expensive hardware. In 2009 the system Pocket Fritz 4, running on a PDA, won the Copa Mercosur chess tournament in Buenos Aires with nine wins and one draw against 10 excellent human chess players, three of them grandmasters. Even though not much information about the internal structure of the software is available, this chess machine represents a trend away from raw computing power toward more intelligence. This machine plays at grandmaster level, and is comparable to, if not better than Hydra. According to Pocket Fritz developer Stanislav Tsukrov [[**?**]], Pocket Fritz with its chess search engine HIARCS 13 searches less than 20,000 positions per second, which is slower than Hydra by a factor of about 10,000. This leads to the conclusion that HIARCS 13 definitely uses better heuristics to decrease the effective branching factor than Hydra and can thus well be called more intelligent than Hydra. By the way, HIARCS is a short hand for Higher Intelligence Auto Response Chess System.

0.16.21 Go

Even though today no human stands a chance against the best chess computers, there are still many challenges for AI. For example Go. In this ancient Japanese game, played on a square board of 361 spaces with 181 white and 180 black stones, the effective branching factor is about 250. After 8 half-moves there are already $1.5 \cdot 10^{19}$ possible positions. Given this complexity, none of the classic, well-known game tree search algorithms have a chance against a good human Go player. Yet in the most recent previous edition of this book, it was stated that:

> The experts agree that "truly intelligent" algorithms are needed here. Combinatoric enumeration of all possibilities is the wrong approach. Rather, procedures are needed that recognize patterns on the board, track gradual developments, and make rapid "intuitive" decisions. Similar to object recognition in complex images, we humans are still far superior to today's computer programs. We process the image as a whole in a highly parallel manner, whereas the computer processes the millions of pixels successively and has great difficulty recognizing the essentials in the abundance of pixels. The program "The Many Faces of Go" recognizes 1100 different patterns and knows 200 different playing strategies. All Go programs, however, still have great difficulty recognizing whether a group of stones is dead or alive, or where in between to classify them.

This statement is now obsolete. In January of 2016, Google [SHM+16] and Facebook [[**?**]] published the breakthrough concurrently. That same month, the program AlphaGo, developed and presented in [SHM+16] by Google DeepMind, defeated European Go champion Fan Hui 5:0. Two months later, Korean player Lee Sedol, one of the best in the world, was defeated 4:1. Deep Learning for pattern recognition (see Sect. 9.7), reinforcement learning (see Chap. 10) and Monte Carlo tree search (MCTS, see Sect. 6.5.1) lead to this successful result. The program plays hundreds of thousands of games against itself and uses the results (win, loss, draw) to learn the best possible heuristic score for a given position. Monte Carlo tree search is used as a replacement for Minimax search, which is not suitable for Go. In Sect. 10.10, after we have gained familiarity with the necessary learning algorithms, we will introduce AlphaGo.

0.17 Exercise

(a) Prove Theorem 6.1 on page 96, in other words, prove that for a tree with large constant branching factor b, almost all nodes are on the last level at depth d.
(b) Show that this is not always true when the effective branching factor is large and not constant.

0.18 Exercise

(a) Calculate the average branching factor for the 8-puzzle without a check for cycles. The average branching factor is the branching factor that a tree with an equal number of nodes on the last level, constant branching factor, and equal depth would have.
(b) Calculate the average branching factor for the 8-puzzle for uninformed search while avoiding cycles of length 2.

0.19 Exercise

(a) What is the difference between the average and the effective branching factor (Definition 6.2 on page 95)?
(b) Why is the effective branching factor better suited to analysis and comparison of the computation time of search algorithms than the average branching factor?
(c) Show that for a heavily branching tree with n nodes and depth d the effective branching factor $\bar{b}$ is approximately equal to the average branching factor and thus equal to $\sqrt[d]{n}$.

0.20 Exercise

(a) Calculate the size of the state space for the 8-puzzle, for the analogous 3-puzzle (2×2-matrix), as well as for the 15-puzzle (4×4-matrix).
(b) Prove that the state graph consisting of the states (nodes) and the actions (edges) for the 3-puzzle falls into two connected sub-graphs, between which there are no connections.

Exercise 6.5 With breadth-first search for the 8-puzzle, find a path (manually) from the starting node

1		3
4	2	6
7	5	8

to the goal node

1	2	3
4	5	6
7	8	

0.21 Exercise

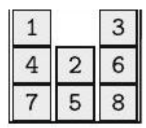
(a) Program breadth-first search, depth-first search, and iterative deepening in the language of your choice and test them on the 8-puzzle example.
(b) Why does it make little sense to use depth-first search on the 8-puzzle?

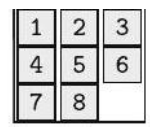
0.22 Exercise

(a) Show that breadth-first search given constant cost for all actions is guaranteed to find the shortest solution.
(b) Show that this is not the case for varying costs.

Exercise 6.8 The predecessors of all nodes must be stored to check for cycles during depth-first search.
(a) For depth first search develop a data structure (not a hash table) that is as efficient as possible for storing all nodes in the search path of a search tree.
(b) For constant branching factor b and depth d, give a formula for the storage space needed by depth-first search with and without storing predecessors.
(c) Show that for large b and d, we have $\sum_{k=0}^{d} k \cdot b^k \approx d \cdot b^d$.

Exercise 6.9 Using $A^\star$ search for the 8-puzzle, search (manually) for a path from

the starting node 

to the goal node 

(a) using the heuristic h_1 (Sect. 6.3.4).
(b) using the heuristic h_2 (Sect. 6.3.4).

Exercise 6.10 Construct the $A^\star$ search tree for the city graph from Fig. 6.14 on page 106 and use the flying distance to Ulm as the heuristic. Start in Bern with Ulm as the destination. Take care that each city only appears once per path.

0.23 Exercise

(a) Show that the triangle inequality is valid for shortest distances on maps.
(b) Using an example, show that it is not always the case that the triangle inequality holds for direct neighbor nodes x and y, where the distance is $d(x, y)$. That is, it is not the case that $d(x, y) \leq d(x, z) + d(z, y)$.
$\rightarrow$ Exercise 6.12 Program $A^\star$ search in the programming language of your choice using the heuristics h_1 and h_2 and test these on the 8 -puzzle example.

- Exercise 6.13 Give a heuristic evaluation function for states with which HeuristicSearch can be implemented as depth-first search, and one for a breadth-first search implementation.

Exercise 6.14 What is the relationship between the picture of the couple at the canyon from Fig. 6.13 on page 105 and admissible heuristics? Exercise 6.15 Show that the heuristics h_1 and h_2 for the 8-puzzle from Sect. 6.3.4 are admissible.

0.24 Exercise

(a) The search tree for a two-player game is given in Fig. 6.23 with the ratings of all leaf nodes. Use minimax search with $\alpha - \beta$ pruning from left to right. Cross out all nodes that are not visited and give the optimal resulting rating for each inner node. Mark the chosen path.

(b) Test yourself using P. Winston's applet [[**?**]].

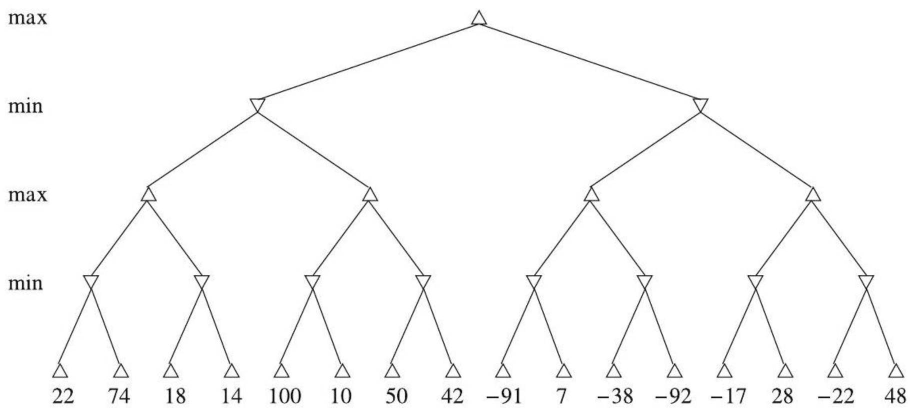


Figure 31: Minimax search tree

0.25 Reasoning with Uncertainty

We have already shown in Chap. 4 with the Tweety problem that two-value logic leads to problems in everyday reasoning. In this example, the statements Tweety is a penguin, Penguins are birds, and All birds can fly lead to the (semantically incorrect) inference Tweety can fly. Probability theory provides a language in which we can formalize the statement Nearly all birds can fly and carry out inferences on it. Probability theory is a proven method we can use here because the uncertainty about whether birds can fly can be modeled well by a probability value. We will show, that statements such as 99% of all birds can fly, together with probabilistic logic, lead to correct inferences. Reasoning under uncertainty with limited resources plays a big role in everyday situations and also in many technical applications of AI. In these areas heuristic processes are very important, as we have already discussed in Chap. 6. For example, we use heuristic techniques when looking for a parking space in city traffic. Heuristics alone are often not enough, especially when a quick decision is needed given incomplete knowledge, as shown in the following example. A pedestrian crosses the street and an auto quickly approaches. To prevent a serious accident, the pedestrian must react quickly. He is not capable of worrying about complete information about the state of the world, which he would need for the search algorithms discussed in Chap. 6. He must therefore come to an optimal decision under the given constraints (little time and little, potentially uncertain knowledge). If he thinks too long, it will be dangerous. In this and many similar situations (see Fig. 7.1 on page 126), a method for reasoning with uncertain and incomplete knowledge is needed.

We want to investigate the various possibilities of reasoning under uncertainty in a simple medical diagnosis example. If a patient experiences pain in the right lower abdomen and a raised white blood cell (leukocyte) count, this raises the suspicion that it might be appendicitis. We model this relationship using propositional logic with the formula

Stomach pain right lower $\wedge$ Leukocytes $> 10000 \rightarrow$ Appendicitis

If we then know that

$$\text{Stomach pain right lower } \wedge \text{ Leukocytes } > 10000$$

is true, then we can use modus ponens to derive Appendicitis. This model is clearly too coarse. In 1976, Shortliffe and Buchanan recognized this when building their medical expert system MYCIN [[**?**]]. They developed a calculus using so-called certainty factors, which allowed the certainty of facts and rules to be represented. A rule $A \rightarrow B$ is assigned a certainty factor β. The semantic of a rule $A \rightarrow_{\beta} B$ is defined via the conditional probability $P(B \mid A) = \beta$. In the above example, the rule could then read

$$\text{Stomach pain right lower } \wedge \text{ Leukocytes } > 10000 \rightarrow_{0.6} \text{ Appendicitis.}$$

For reasoning with this kind of formulas, they used a calculus for connecting the factors of rules. It turned out, however, that with this calculus inconsistent results could be derived.

As discussed in Chap. 4, there were also attempts to solve this problem by using non-monotonic logic and default logic, which, however, were unsuccessful in the end. The Dempster-Schäfer theory assigns a belief function Bel(A) to a logical proposition A, whose value gives the degree of evidence for the truth of A. But even this formalism has weaknesses, which is shown in [Pea88] using a variant of the Tweety example. Even fuzzy logic, which above all is successful in control theory, demonstrates considerable weaknesses when reasoning under uncertainty in more complex applications [[**?**]]. Since about the mid-1980s, probability theory has had more and more influence in AI [**?**, **?**, **?**, **?**]. In the field of reasoning with Bayesian networks, or subjective probability, it has secured itself a firm place among successful AI techniques. Rather than implication as it is known in logic (material implication), conditional probability is used here, which models everyday causal reasoning significantly better. Reasoning with probability profits heavily from the fact that probability theory is a hundreds of years old, well-established branch of mathematics.

In this chapter we will select an elegant, but for an instruction book somewhat unusual, entry point into this field. After a short introduction to the most

Figure 32: “Let’s just sit back and think about what to do!”

important foundations needed here for reasoning with probability, we will begin with a simple, but important example for reasoning with uncertain and incomplete knowledge. In a quite natural, almost compelling way, we will be led to the method of maximum entropy (MaxEnt). Then we will show the usefulness of this method in practice using the medical expert system lexmed. Finally we will introduce the now widespread reasoning with Bayesian networks, and show the relationship between the two methods.

0.25.1 Computing with Probabilities

The reader who is familiar with probability theory can skip this section. For everyone else we will give a quick ramp-up and recommend a few appropriate textbooks such as [**?**, **?**].

Probability is especially well-suited for modeling reasoning under uncertainty. One reason for this is that probabilities are intuitively easy to interpret, which can be seen in the following elementary example.

Example 7.1 For a single roll of a gaming die (experiment), the probability of the event "rolling a six" equals $1/6$, whereas the probability of the occurrence "rolling an odd number" is equal to $1/2$.

Definition 7.1 Let Ω be the finite set of events for an experiment. Each event $\omega \in \Omega$ represents a possible outcome of the experiment. If these events $w_i \in \Omega$ mutually exclude each other, but cover all possible outcomes of the attempt, then they are called elementary events.

Example 7.2 For a single roll of one gaming die

$$\Omega = \{1, 2, 3, 4, 5, 6\}$$

because no two of these events can happen simultaneously. Rolling an even number ($\{2, 4, 6\}$) is therefore not an elementary event, nor is rolling a number smaller than five ($\{1, 2, 3, 4\}$) because $\{2, 4, 6\} \cap \{1, 2, 3, 4\} = \{2, 4\} \neq \emptyset$.

Given two events A and B, $A \cup B$ is also an event. Ω itself is denoted the certain event, and the empty set $\emptyset$ the impossible event.

In the following we will use the propositional logic notation for set operations. That is, for the set $A \cap B$ we write $A \wedge B$. This is not only a syntactic transformation, rather it is also semantically correct because the intersection of two sets is defined as

$$x \in A \cap B \Leftrightarrow x \in A \wedge x \in B.$$

Because this is the semantic of $A \wedge B$, we can and will use this notation. This is also true for the other set operations union and complement, and we will, as shown in the following table, use the propositional logic notation for them as well.

Set notation	Propositional logic	Description
$A \cap B$	$A \wedge B$	intersection / and
$A \cup B$	$A \vee B$	union / or
$\bar{A}$	$\neg A$	complement / negation
Ω	t	certain event / true
$\emptyset$	f	impossible event / false

The variables used here (for example A, B, etc.) are called random variables in probability theory. We will only use discrete chance variables with finite domains here. The variable face_number for a dice roll is discrete with the values $1, 2, 3, 4,$ 5,6 . The probability of rolling a five or a six is equal to $1/3$. This can be described by

$$P(\text{face\_number} \in \{5,6\}) = P(\text{face\_number} = 5 \vee \text{face\_number} = 6) = 1/3.$$

The concept of probability is supposed to give us a description as objective as possible of our "belief" or "conviction" about the outcome of an experiment. All numbers in the interval $[0, 1]$ should be possible, where 0 is the probability of the impossible event and 1 the probability of the certain event. We come to this from the following definition.

Definition 7.2 Let $\Omega = \{\omega_1, \omega_2, \ldots, \omega_n\}$ be finite. There is no preferred elementary event, which means that we assume a symmetry related to the frequency of how often each elementary event appears. The probability $P(A)$ of the event A is then

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{Number of favorable cases for } A}{\text{Number of possible cases}}.$$

It follows immediately that every elementary event has the probability $1/|\Omega|$. The requirement that elementary events have equal probability is called the Laplace assumption and the probabilities calculated thereby are called Laplace probabilities. This definition hits its limit when the number of elementary events becomes infinite. Because we are only looking at finite event spaces here, though, this does not present a problem. To describe events we use variables with the appropriate number of values. For example, a variable eye_color can take on the values green, blue, brown. eye_color = blue then describes an event because we are dealing with a proposition with the truth values t or f. For binary (boolean) variables, the variable itself is already a proposition. Here it is enough, for example, to write $P(\text{JohnCalls})$ instead of $P(\text{JohnCalls} = t)$.

Example 7.3 By this definition, the probability of rolling an even number is

$$P(\text{face\_number} \in \{2,4,6\}) = \frac{|\{2,4,6\}|}{|\{1,2,3,4,5,6\}|} = \frac{3}{6} = \frac{1}{2}.$$

The following important rules follow directly from the definition.

Theorem 7.1

1. $P(\Omega) = 1$.
2. $P(\emptyset) = 0$, which means that the impossible event has a probability of 0 .
3. For pairwise exclusive events A and B it is true that $P(A \vee B) = P(A) + P(B)$.
4. For two complementary events A and $\neg A$ it is true that $P(A)+P(\neg A) = 1$.
5. For arbitrary events A and B it is true that $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$.
6. For $A \subseteq B$ it is true that $P(A) \leq P(B)$.
7. If $A_1, \ldots, A_n$ are the elementary events, then $\sum_{i=1}^{n} P(A_i) = 1$ (normalization condition).

The expression $P(A \wedge B)$ or equivalently $P(A, B)$ stands for the probability of the events $A \wedge B$. We are often interested in the probabilities of all elementary events, that is, of all combinations of all values of the variables A and B. For the binary variables A and B these are $P(A, B), P(A, \neg B), P(\neg A, B), P(\neg A, \neg B)$. We call the vector

$$(P(A, B), P(A, \neg B), P(\neg A, B), P(\neg A, \neg B))$$

consisting of these four values a distribution or joint probability distribution of the variables A and B. A shorthand for this is $\boldsymbol{P}(A, B)$. The distribution in the case of two variables can be nicely visualized in the form of a table (matrix), represented as follows:

$\boldsymbol{P}(A, B)$	$B = w$	$B = f$
$A = w$	$P(A, B)$	$P(A, \neg B)$
$A = f$	$P(\neg A, B)$	$P(\neg A, \neg B)$

For the d variables $X_1, \ldots, X_d$ with n values each, the distribution has the values $P(X_1 = x_1, \ldots, X_d = x_d)$ and $x_1, \ldots, x_d$, each of which take on n different values. The distribution can therefore be represented as a d-dimensional matrix with a total of n^d elements. Due to the normalization condition from Theorem 7.1 on page 129, however, one of these n^d values is redundant and the distribution is characterized by $n^d - 1$ unique values.

0.25.2 Conditional Probability

Example 7.4 On Landsdowne street in Boston, the speed of 100 vehicles is measured. For each measurement it is also noted whether the driver is a student. The results are

Event	Frequency	Relative frequency
Vehicle observed	100	1
Driver is a student (S)	30	0.3
Speed too high (G)	10	0.1
Driver is a student and speeding ($S \wedge G$)	5	0.05

We pose the question: Do students speed more frequently than the average person, or than non-students? [1]

The answer is given by the probability

$$P(G \mid S) = \frac{\mid \text{Driver is a student and speeding} \mid}{\mid \text{Driver is a student} \mid} = \frac{5}{30} = \frac{1}{6} \approx 0.17$$

for speeding under the condition that the driver is a student. This is obviously different from the a priori probability $P(G) = 0.1$ for speeding. For the a priori probability, the event space is not limited by additional conditions.

Definition 7.3 For two events A and B, the probability $P(A \mid B)$ for A under the condition B (conditional probability) is defined by

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)}.$$

In Example 7.4 we see that in the case of a finite event space, the conditional probability $P(A \mid B)$ can be understood as the probability of $A \wedge B$ when we only look at the event B, that is, as

$$P(A \mid B) = \frac{|A \wedge B|}{|B|}.$$

This formula can be easily derived using Definition 7.2 on page 129

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)} = \frac{\frac{|A \wedge B|}{|\Omega|}}{\frac{|B|}{|\Omega|}} = \frac{|A \wedge B|}{|B|}.$$

Definition 7.4 If, for two events A and B,

$$P(A \mid B) = P(A),$$

then these events are called independent.

Thus A and B are independent if the probability of the event A is not influenced by the event B.

Theorem 7.2 For independent events A and B, it follows from the definition that

$$P(A \wedge B) = P(A) \cdot P(B).$$

[21] [1] The computed probabilities can only be used for continued propositions if the measured sample (100 vehicles) is representative. Otherwise only propositions about the observed 100 vehicles can be made.

Example 7.5 For a roll of two dice, the probability of rolling two sixes is $1/36$ if the two dice are independent because

$$P(D_1 = 6 \wedge D_2 = 6) = P(D_1 = 6) \cdot P(D_2 = 6) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$$

where the first equation is only true when the two dice are independent. If for example by some magic power die 2 is always the same as die 1, then

$$P(D_1 = 6 \wedge D_2 = 6) = \frac{1}{6}.$$

0.26 Chain Rule

Solving the definition of conditional probability for $P(A \wedge B)$ results in the so-called product rule

$$P(A \wedge B) = P(A \mid B)\, P(B),$$

which we immediately generalize for the case of n variables. By repeated application of the above rule we obtain the chain rule

$$\begin{aligned}
\boldsymbol{P}(X_1, \ldots, X_n) &= \boldsymbol{P}(X_n \mid X_1, \ldots, X_{n-1}) \cdot \boldsymbol{P}(X_1, \ldots, X_{n-1}) \\
&= \boldsymbol{P}(X_n \mid X_1, \ldots, X_{n-1}) \cdot \boldsymbol{P}(X_{n-1} \mid X_1, \ldots, X_{n-2}) \cdot \boldsymbol{P}(X_1, \ldots, X_{n-2}) \\
&= \boldsymbol{P}(X_n \mid X_1, \ldots, X_{n-1}) \cdot \boldsymbol{P}(X_{n-1} \mid X_1, \ldots, X_{n-2}) \cdot \ldots \cdot \boldsymbol{P}(X_n \mid X_1) \cdot \boldsymbol{P}(X_1) \\
&= \prod_{i=1}^{n} \boldsymbol{P}(X_n \mid X_1, \ldots, X_{i-1}) \qquad (7.1)
\end{aligned}$$

with which we can represent a distribution as a product of conditional probabilities. Because the chain rule holds for all values of the variables $X_1, \ldots, X_n$, it has been formulated for the distribution using the symbol $\boldsymbol{P}$.

0.27 Marginalization

Because $A \Leftrightarrow (A \wedge B) \vee (A \wedge \neg B)$ is true for binary variables A and B

$$P(A) = P((A \wedge B) \vee (A \wedge \neg B)) = P(A \wedge B) + P(A \wedge \neg B).$$

By summation over the two values of B, the variable B is eliminated. Analogously, for arbitrary variables $X_1, \ldots, X_d$, a variable, for example X_d, can be eliminated by summation over all of their variables and we get

$$P(X_1 = x_1, \ldots, X_{d-1} = x_{d-1}) = \sum_{x_d} P(X_1 = x_1, \ldots, X_{d-1} = x_{d-1}, X_d = x_d).$$

The application of this formula is called marginalization. This summation can continue with the variables $X_1, \ldots, X_{d-1}$ until just one variable is left. Marginalization can also be applied to the distribution $\boldsymbol{P}(X_1, \ldots, X_d)$. The resulting distribution $\boldsymbol{P}(X_1, \ldots, X_{d-1})$ is called the marginal distribution. It is comparable to the projection of a rectangular cuboid on a flat surface. Here the three-dimensional object is drawn on the edge or "margin"of the cuboid, i.e. on a two-dimensional set. In both cases the dimensionality is reduced by one.

Example 7.6 We observe the set of all patients who come to the doctor with acute stomach pain. For each patient the leukocyte value is measured, which is a metric for the relative abundance of white blood cells in the blood. We define the variable Leuko, which is true if and only if the leukocyte value is greater than 10,000. This indicates an infection in the body. Otherwise we define the variable App, which tells us whether the patient has appendicitis, that is, an infected appendix. The distribution $\boldsymbol{P}(\text{App}, \text{Leuko})$ of these two variables is given in the following table:

$\boldsymbol{P}(\text{App}, \text{Leuko})$	App	¬App	Total
Leuko	0.23	0.31	0.54
¬ Leuko	0.05	0.41	0.46
Total	0.28	0.72	1

In the last row the sum over the rows is given, and in the last column the sum of the columns is given. These sums are arrived at by marginalization. For example, we read off

$$P(\text{Leuko}) = P(\text{App}, \text{Leuko}) + P(\neg\text{App}, \text{Leuko}) = 0.54.$$

The given distribution $\boldsymbol{P}(\text{App}, \text{Leuko})$ could come from a survey of German doctors, for example. From it we can then calculate the conditional probability

$$P(\text{Leuko} \mid \text{App}) = \frac{P(\text{Leuko}, \text{App})}{P(\text{App})} = 0.82$$

which tells us that about 82% of all appendicitis cases lead to a high leukocyte value. Values like this are published in medical literature. However, the conditional
probability $P(\text{App} \mid \text{Leuko})$, which would actually be much more helpful for diagnosing appendicitis, is not published. To understand this, we will first derive a simple, but very important formula.

0.28 Bayes' Theorem

Swapping A and B in Definition 7.3 yields

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)} \quad \text{and} \quad P(B \mid A) = \frac{P(A \wedge B)}{P(A)}$$

By solving both equations for $P(A \wedge B)$ and equating them we obtain Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} \tag{7.2}$$

whose relevance to many applications we will illustrate using three examples. First we apply it to the appendicitis example and obtain
Example 7.7

$$P(\text{App} \mid \text{Leuko}) = \frac{P(\text{Leuko} \mid \text{App}) \cdot P(\text{App})}{P(\text{Leuko})} = \frac{0.82 \cdot 0.28}{0.54} = 0.43. \qquad (7.3)$$

Why then is $P(\text{Leuko} \mid \text{App})$ published, but not $P(\text{App} \mid \text{Leuko})$?
Assuming that appendicitis affects the biology of all humans the same, regardless of ethnicity, $P(\text{Leuko} \mid \text{App})$ is a universal value that is valid worldwide. In Equation 7.3 we see that $P(\text{App} \mid \text{Leuko})$ is not universal, for this value is influenced by the a priori probabilities $P(\text{App})$ and $P(\text{Leuko})$. Each of these can vary according to one's life circumstances. For example, $P(\text{Leuko})$ is dependent on whether a population has a high or low rate of exposure to infectious diseases. In the tropics, this value can differ significantly from that of cold regions. Bayes' theorem, however, makes it easy for us to take the universally valid value $P(\text{Leuko} \mid \text{App})$, and compute $P(\text{App} \mid \text{Leuko})$ which is useful for diagnosis.

Before we dive deeper into this example and build a medical expert system for appendicitis in Sect. 7.3 let us first apply Bayes' theorem to another interesting medical example.

Example 7.8 In cancer diagnosis, so-called tumor markers are often measured. One example of this is the use of the tumor marker PSA (prostate specific antigen) for the diagnosis of prostate cancer (PCa $=$ prostate cancer) in men. Assuming that no further tests for PCa have been conducted, the test is considered positive, that is, there is suspected PCa, if the concentration of PSA reaches a level at or above $4\,\text{ng/ml}$. If this occurs, the probability $P(C \mid \text{pos})$ of PCa is of interest to the patient.

The binary variable C is true if the patient has PCa, and pos represents a PSA value $\geq 4\,\text{ng/ml}$. Let us now compute the $P(C \mid \text{pos})$. For reasons similar to those mentioned for appendicitis diagnosis, this value is not reported. Instead, researchers publish the sensitivity $P(\text{pos} \mid C)$ and the specificity $P(\text{neg} \mid \neg C)$ of the test.[2] According to [**?**], for a sensitivity of 0.95, the specificity can be at most 0.25, which is why we proceed from $P(\text{pos} \mid C) = 0.95$ and $P(\text{neg} \mid \neg C) = 0.25$ below. We apply Bayes' theorem and obtain

$$\begin{aligned} P(C \mid \text{pos}) &= \frac{P(\text{pos} \mid C) \cdot P(C)}{P(\text{pos})} = \frac{P(\text{pos} \mid C) \cdot P(C)}{P(\text{pos} \mid C) \cdot P(C) + P(\text{pos} \mid \neg C) \cdot P(\neg C)} \\ &= \frac{0.95 \cdot 0.0021}{0.95 \cdot 0.0021 + 0.75 \cdot 0.99679} = \frac{0.95 \cdot 0.0021}{0.75} = 0.0027 \end{aligned}$$

Here we use $P(\text{pos} \mid \neg C) = 1 - P(\text{neg} \mid \neg C) = 1 - 0.25 = 0.75$ and $P(C) = 0.0021 = 0.21\%$ as the a priori probability of PCa during one year.[3] It makes sense to assume that the PSA test is done once per year. This result is somewhat surprising from the patient's perspective because the probability of PCa after a positive test is, at 0.27%, only marginally higher than the probability of 0.21% for PCa for a 55-year-old man. Thus, a PSA value of just over $4\,\text{ng/ml}$

is definitively no reason for the patient to panic. At most it is used as a basis for further examinations, such as biopsy or MRI, leading if necessary to radiation and surgery. The situation is similar for many other tumor markers such as those for colorectal cancer or breast cancer diagnosis by mammography.

The cause of this problem is the very low specificity $P(\text{neg} \mid \neg C) = 0.25$, which leads to 75% of healthy patients (without PCa) getting a false-positive test result and consequently undergoing unnecessary examinations. Because of this, PSA testing has been a controversial discussion topic for years.[4]

Assume we had a better test with a specificity of 99%, which would only deliver a false-positive result for one percent of healthy men. Then, in the above calculation, we would assign $P(\text{pos} \mid \neg C)$ the value 0.01 and obtain the result $P(C \mid \text{pos}) = 0, 17$. Plainly, this test would be much more specific.

Example 7.9 A sales representative who wants to sell an alarm system could make the following argument:

> If you buy this very reliable alarm system, it will alert you to any break-in with 99% certainty. Our competitor's system only offers a certainty of 85%.

Hearing this, if the buyer concludes that from an alert A he can infer a break-in B with high certainty, he is wrong. Bayes' theorem shows the reason. What the

representative told us is that $P(A \mid B) = 0.99$. What he doesn't say, however, is what it means when we hear the alarm go off. To find out, we use Bayes' theorem to compute $P(B \mid A)$ and assume that the buyer lives in a relatively safe area in which break-ins are rare, with $P(B) = 0.001$. Additionally, we assume that the alarm system is triggered not only by burglars, but also by animals, such as birds or cats in the yard, which results in $P(A) = 0.1$. Thus we obtain

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)} = \frac{0.99 \cdot 0.001}{0.1} = 0.01,$$

which means that whoever buys this system will not be happy because they will be startled by too many false alarms. When we examine the denominator

$$P(A) = P(A \mid B)P(B) + P(A \mid \neg B)P(\neg B) = 0.00099 + P(A \mid \neg B) \cdot 0.999 = 0.1$$

of Bayes' theorem more closely, we see that $P(A \mid \neg B) \approx 0.1$, which means that the alarm will be triggered roughly every tenth day that there is not a break-in. From this example we learn, among other things, that it is important to consider which probabilities we are really interested in as a buyer, expecially when it comes to security. When the arguments of a conditional probability are interchanged, the value can change dramatically when the prior probabilities differ significantly.

[2]2 For definitions of sensitivity and specificity see Eqs. 7.16 and 7.17.

[3] See http://www.prostata.de/pca_haeufigkeit.html for a 55-year-old man.

[4] The author is not a medical doctor. Therefore these computations should not be used as a basis for personal medical decisions by potentially afflicted individuals. If necessary, please consult a specialist physician or the relevant specialist literature.

0.28.1 The Principle of Maximum Entropy

We will now show, using an inference example, that a calculus for reasoning under uncertainty can be realized using probability theory. However, we will soon see that the well-worn probabilistic paths quickly come to an end. Specifically, when too little knowledge is available to solve the necessary equations, new ideas are needed. The American physicist E.T. Jaynes did pioneering work in this area in the 1950s. He claimed that given missing knowledge, one can maximize the entropy of the desired probability distribution, and applied this principle to many examples in [[**?**], [**?**]]. This principle was then further developed [[**?**], [**?**], [**?**], [**?**]] and is now mature and can be applied technologically, which we will show in the example of the lexmed project in Sect. 7.3.

0.28.2 An Inference Rule for Probabilities

We want to derive an inference rule for uncertain knowledge that is analogous to the modus ponens. From the knowledge of a proposition A and a rule $A \Rightarrow B$, the conclusion B shall be reached. Formulated succinctly, this reads

$$\frac{A, A \to B}{B}.$$

The generalization for probability rules yields

$$\frac{P(A) = \alpha, \quad P(B \mid A) = \beta}{P(B) = ?}.$$

Let the two probability rules α, β be given and the value $P(B)$ desired. By marginalization we obtain the desired marginal distribution

$$P(B) = P(A, B) + P(\neg A, B) = P(B \mid A) \cdot P(A) + P(B \mid \neg A) \cdot P(\neg A).$$

The three values $P(A), P(\neg A), P(B \mid A)$ on the right side are known, but the value $P(B \mid \neg A)$ is unknown. We cannot make an exact statement about $P(B)$ with classical probability theory, but at the most we can estimate $P(B) \geq P(B \mid A) \cdot P(A)$.

We now consider the distribution

$$\boldsymbol{P}(A, B) = (P(A, B), P(A, \neg B), P(\neg A, B), P(\neg A, \neg B))$$

and introduce for shorthand the four unknowns

$$\begin{aligned} p_1 &= P(A, B), \\ p_2 &= P(A, \neg B), \\ p_3 &= P(\neg A, B), \\ p_4 &= P(\neg A, \neg B). \end{aligned}$$

These four parameters determine the distribution. If they are all known, then every probability for the two variables A and B can be calculated. To calculate the four unknowns, four equations are needed. One equation is already known in the form of the normalization condition

$$p_1 + p_2 + p_3 + p_4 = 1$$

Therefore, three more equations are needed. In our example, however, only two equations are known.

From the given values $P(A) = \alpha$ and $P(B \mid A) = \beta$ we calculate

$$P(A, B) = P(B \mid A) \cdot P(A) = \alpha\beta$$

and

$$P(A) = P(A, B) + P(A, \neg B).$$

From this we can set up the following system of equations and solve it as far as is possible:

$$p_1 = \alpha\beta, \tag{7.4}$$
$$p_1 + p_2 = \alpha, \tag{7.5}$$
$$p_1 + p_2 + p_3 + p_4 = 1, \tag{7.6}$$

$$(7.4) \text{ in } (7.5): \quad p_2 = \alpha - \alpha\beta = \alpha(1 - \beta), \tag{7.7}$$
$$(7.5) \text{ in } (7.6): \quad p_3 + p_4 = 1 - \alpha. \tag{7.8}$$

The probabilities p_1, p_2 for the interpretations (A, B) and $(A, \neg B)$ are thus known, but for the values p_3, p_4 only one equation still remains. To come to a definite solution despite this missing knowledge, we change our point of view. We use the given equation as a constraint for the solution of an optimization problem.

We are looking for a distribution $\boldsymbol{p}$ (for the variables p_3, p_4) which maximizes the entropy

$$H(\boldsymbol{p}) = -\sum_{i=1}^{n} p_i \ln p_i = -p_3 \ln p_3 - p_4 \ln p_4 \tag{7.9}$$

under the constraint $p_3 + p_4 = 1 - \alpha$ (7.8). Why exactly should the entropy function be maximized? Because we are missing information about the distribution, it must somehow be added in. We could fix an ad hoc value, for example $p_3 = 0.1$. Yet it is better to determine the values p_3 and p_4 such that the information added is minimal. We can show (Sect. 8.4.2 and [**?**]) that entropy measures the uncertainty of a distribution up to a constant factor. Negative entropy is then a measure of the amount of information a distribution contains. Maximization of entropy minimizes the information content of the distribution. To visualize this, the entropy function for the two-dimensional case is represented graphically in Fig. 7.2 on page 139.

To determine the maximum of the entropy under the constraint $p_3+p_4-1+\alpha = 0$ we use the method of Lagrange multipliers [**?**]. The Lagrange function reads

$$L = -p_3 \ln p_3 - p_4 \ln p_4 + \lambda\,(p_3 + p_4 - 1 + \alpha)$$

Taking the partial derivatives with respect to p_3 and p_4 we obtain

$$\frac{\partial L}{\partial p_3} = -\ln p_3 - 1 + \lambda = 0$$
$$\frac{\partial L}{\partial p_4} = -\ln p_4 - 1 + \lambda = 0$$

and calculate

$$p_3 = p_4 = \frac{1-\alpha}{2}.$$

Now we can calculate the desired value

$$P(B) = P(A,B) + P(\neg A, B) = p_1 + p_3 = \alpha\beta + \frac{1-\alpha}{2} = \alpha\left(\beta - \frac{1}{2}\right) + \frac{1}{2}.$$

Substituting in α and β yields

$$P(B) = P(A)\left(P(B \mid A) - \frac{1}{2}\right) + \frac{1}{2}$$

$P(B)$ is shown in Fig. 7.3 on page 140 for various values of $P(B \mid A)$. We see that in the two-value edge case, that is, when $P(B)$ and $P(B \mid A)$ take on the values 0 or 1, probabilistic inference returns the same value for $P(B)$ as the modus ponens. When A and $B \mid A$ are both true, B is also true. An interesting case is $P(A) = 0$, in which $\neg A$ is true. Modus ponens cannot be applied here, but our formula results in the value $1/2$ for $P(B)$ irrespective of $P(B \mid A)$. When A is false, we know nothing about B, which reflects our intuition exactly. The case where $P(A) = 1$ and $P(B \mid A) = 0$ is also covered by propositional logic. Here A is true and $A \Rightarrow B$ false, and thus $A \wedge \neg B$ true. Then B is false. The horizontal line in the figure means that we cannot make a prediction about B in the case of $P(B \mid A) = 1/2$. Between these points, $P(B)$ changes linearly for changes to $P(A)$ or $P(B \mid A)$.

Theorem 7.3 Let there be a consistent [5] set of linear probabilistic equations. Then there exists a unique maximum for the entropy function with the given equations as constraints. The MaxEnt distribution thereby defined has minimum information content under the constraints.

It follows from this theorem that there is no distribution which satisfies the constraints and has higher entropy than the MaxEnt distribution. A calculus

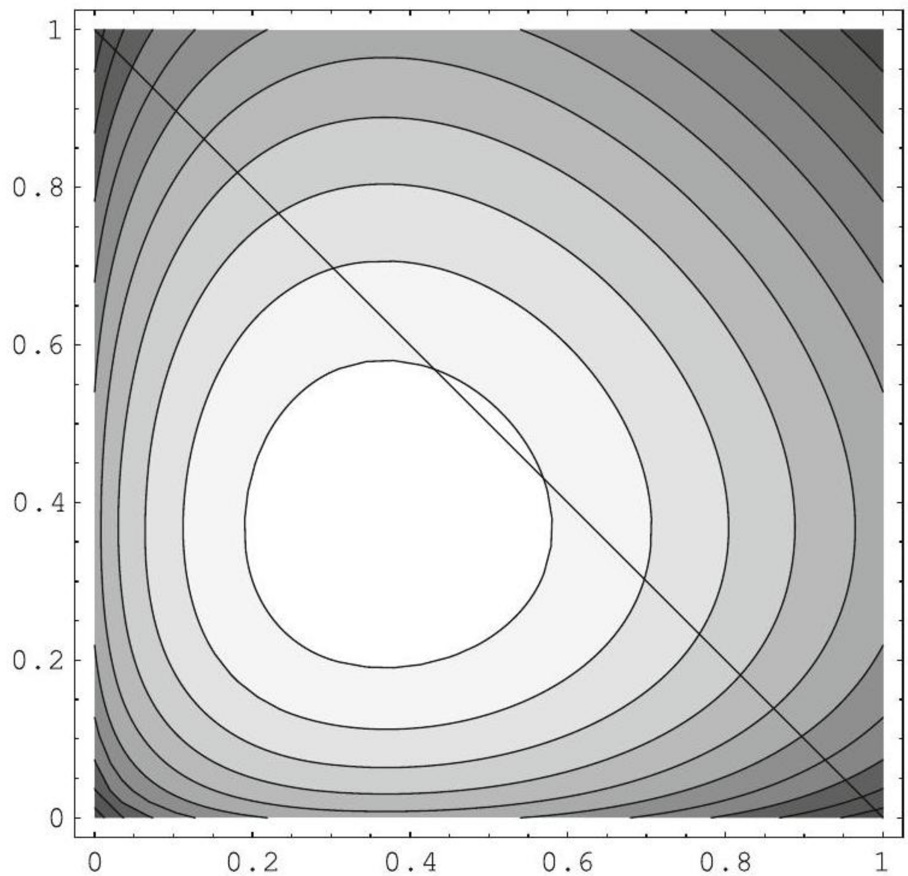


Figure 33: Contour line diagram of the two-dimensional entropy function. We see that it is strictly convex in the whole unit square and that it has an isolated global maximum. Also marked is the constraint $p_3 + p_4 = 1$ as a special case of the condition $p_3 + p_4 - 1 + \alpha = 0$ for $\alpha = 0$ which is relevant here.

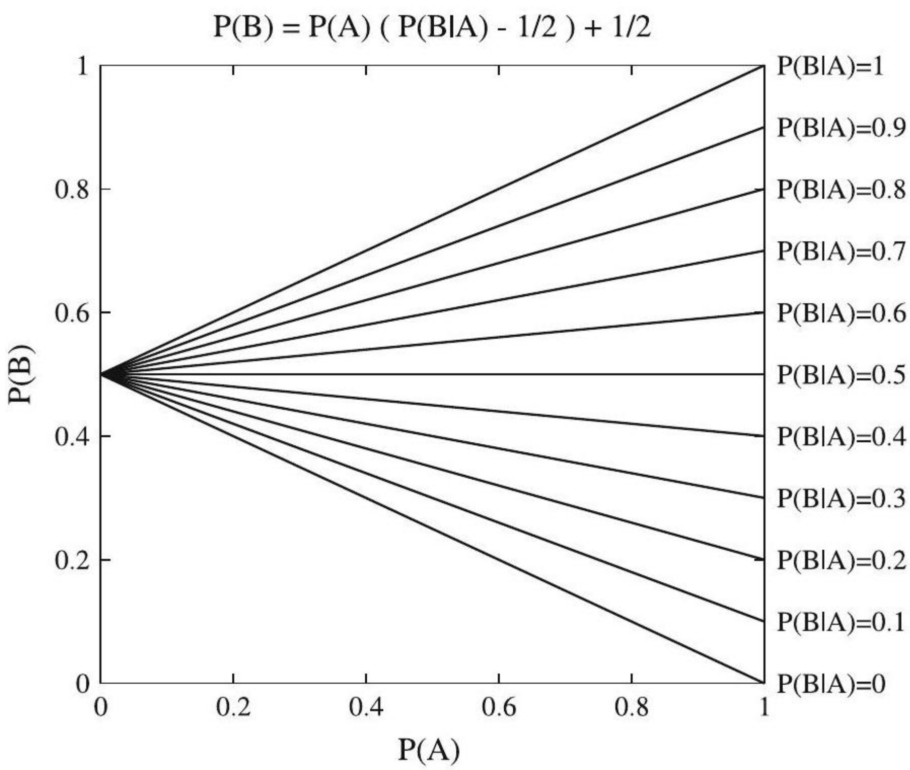


Figure 34: Curve array for $P(B)$ as a function of $P(A)$ for different values of $P(B \mid A)$

that leads to lower entropy puts in additional ad hoc information, which is not justified.

Looking more closely at the above calculation of $P(B)$, we see that the two values p_3 and p_4 always occur symmetrically. This means that swapping the two variables does not change the result. Thus the end result is $p_3 = p_4$. The so-called indifference of these two variables leads to them being set equal by MaxEnt. This relationship is valid generally:

Definition 7.5 If an arbitrary exchange of two or more variables in the Lagrange equations results in equivalent equations, these variables are called indifferent.

Theorem 7.4 If a set of variables $\{p_{i_1}, \ldots, p_{i_k}\}$ is indifferent, then the maximum of the entropy under the given constraints is at the point where $p_{i_1} = p_{i_2} = \cdots = p_{i_k}$.

With this knowledge we could have immediately set the two variables p_3 and p_4 equal (without solving the Lagrange equations).

0.28.3 Maximum Entropy Without Explicit Constraints

We now look at the case in which no knowledge is given. This means that, other than the normalization condition

$$p_1 + p_2 + \cdots + p_n = 1,$$

there are no constraints. All variables are therefore indifferent. Therefore we can set them equal and it follows that $p_1 = p_2 = \cdots = p_n = 1/n$.[6] For reasoning under uncertainty, this means that given a complete lack of knowledge, all worlds are equally probable. That is, the distribution is uniform. For example, in the case of two variables A and B it would be the case that

$$P(A, B) = P(A, \neg B) = P(\neg A, B) = P(\neg A, \neg B) = 1/4$$

from which $P(A) = P(B) = 1/2$ and $P(B \mid A) = 1/2$ follow. The result for the two-dimensional case can be seen in Fig. 7.2 on page 139 because the marked condition is exactly the normalization condition. We see that the maximum of the entropy lies on the line at exactly $(1/2, 1/2)$.

As soon as the value of a condition deviates from the one derived from the uniform distribution, the probabilities of the worlds shift. We show this in a further example. With the same descriptions as used above we assume that only

$$P(B \mid A) = \beta$$

is known. Thus $P(A, B) = P(B \mid A)P(A) = \beta P(A)$, from which $p_1 = \beta\,(p_1 + p_2)$ follows and we derive the two constraints

[25] A set of probabilistic equations is called consistent if there is at least one solution, that is, one distribution which satisfies all equations.

$$\beta p_2 + (\beta - 1)p_1 = 0$$
$$p_1 + p_2 + p_3 + p_4 - 1 = 0$$

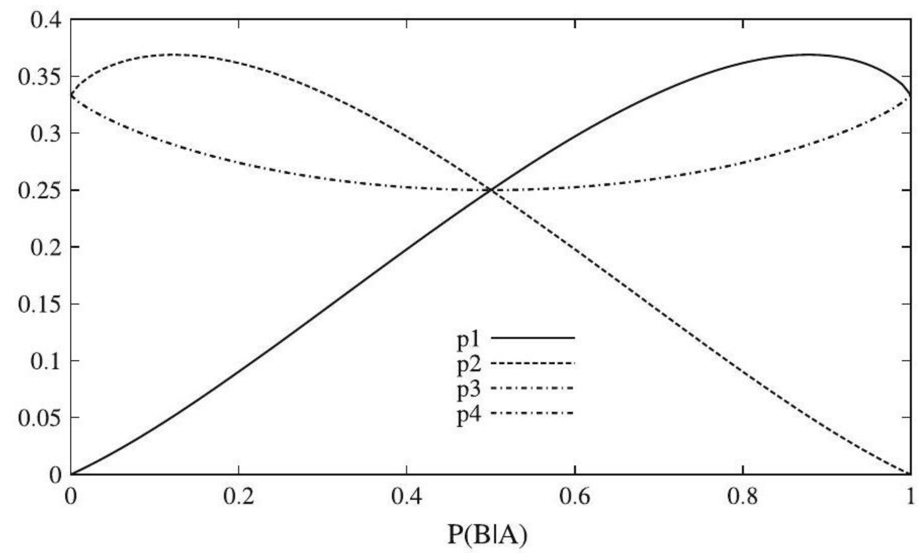


Figure 35: p_1, p_2, p_3, p_4 in dependence on β

Here the Lagrange equations can no longer be solved symbolically so easily. A numeric solution of the Lagrange equations yields the picture represented in Fig. 7.4, which shows that $p_3 = p_4$. We can already see this in the constraints, in which p_3 and p_4 are indifferent. For $P(B \mid A) = 1/2$ we obtain the uniform distribution, which is no surprise. This means that the constraint for this value does not imply a restriction on the distribution. Furthermore, we can see that for small $P(B \mid A), P(A, B)$ is also small.

0.28.4 Conditional Probability Versus Material Implication

We will now show that, for modeling reasoning, conditional probability is better than what is known in logic as material implication (to this end, also see [[**?**]]). First we observe the truth table shown in Table 7.1, in which the conditional probability and material implication for the extreme cases of probabilities zero and one are compared. In both cases with false premises (which, intuitively, are critical cases), $P(B \mid A)$ is undefined, which makes sense.

[26] The reader may calculate this result by maximization of the entropy under the normalization condition (Exercise 7.5 on page 132).

Table 4: Truth table for material implication and conditional probability for propositional logic limit

A	B	$A \Rightarrow B$	$P(A)$	$P(B)$	$P(B \mid A)$
t	t	t	1	1	1
t	f	f	1	0	0
f	t	t	0	1	Undefined
f	f	t	0	0	Undefined

Now we ask ourselves which value is taken on by $P(B \mid A)$ when arbitrary values $P(A) = \alpha$ and $P(B) = \gamma$ are given and no other information is known. Again we maximize entropy under the given constraints. As above we set

$$p_1 = P(A, B), \quad p_2 = P(A, \neg B), \quad p_3 = P(\neg A, B), \quad p_4 = P(\neg A, \neg B)$$

and obtain as constraints

$$p_1 + p_2 = \alpha, \tag{7.10}$$

$$p_1 + p_3 = \gamma, \tag{7.11}$$

$$p_1 + p_2 + p_3 + p_4 = 1. \tag{7.12}$$

With this we calculate using entropy maximization (see Exercise 7.8 on page 173)

$$p_1 = \alpha\gamma, \quad p_2 = \alpha(1 - \gamma), \quad p_3 = \gamma(1 - \alpha), \quad p_4 = (1 - \alpha)(1 - \gamma)$$

From $p_1 = \alpha\gamma$ it follows that $P(A, B) = P(A) \cdot P(B)$, which means that A and B are independent. Because there are no constraints connecting A and B, the MaxEnt principle results in the independence of these variables. The right half of Table 7.1 on page 142 makes this easier to understand. From the definition

$$P(B \mid A) = \frac{P(A, B)}{P(A)}$$

it follows for the case $P(A) \neq 0$, that is, when the premise is not false, because A and B are independent, that $P(B \mid A) = P(B)$. For the case $P(A) = 0$, $P(B \mid A)$ remains undefined.

0.28.5 MaxEnt-Systems

As previously mentioned, due to the nonlinearity of the entropy function, MaxEnt optimization usually cannot be carried out symbolically for non-trivial problems. Thus two systems were developed for numerical entropy maximization.

The first system, SPIRIT (Symmetrical Probabilistic Intensional Reasoning in Inference Networks in Transition, www.xspirit.de), [[**?**]] was built at Fernuniversität Hagen. The second, PIT (Probability Induction Tool) was developed at the Munich Technical University [[**?**], [**?**], [**?**]]. We will now briefly introduce PIT.

The PIT system uses the sequential quadratic programming (SQP) method to find an extremum of the entropy function under the given constraints. As input, PIT expects data containing the constraints. For example, the constraints $P(A) = \alpha$ and $P(B \mid A) = \beta$ from Sect. 7.2.1 have the form

```
var A{t,f}, B{t,f};
P([A=t]) = 0.6;
P([B=t] | [A=t]) = 0.3;
QP([B=t]);
QP([B=t] | [A=t]);
```

Because PIT performs a numerical calculation, we have to input explicit probability values. The second to last row contains the query QP ([B=t]). This means that $P(B)$ is the desired value. At www.pit-systems.de under ”Examples” we now put this input into a blank input page (”Blank Page”) and start PIT. As a result we get

Nr.	Truth value	Probability	Query
1	UNSPECIFIED	$3.800\text{e} - 01$	QP $([B = t])$;
2	UNSPECIFIED	$3.000\text{e} - 01$	QP $([A = t] - \mid > [B = t])$;

and from there read off $P(B) = 0.38$ and $P(B \mid A) = 0.3$.

0.28.6 The Tweety Example

We now show, using the Tweety example from Sect. 4.3, that probabilistic reasoning and in particular MaxEnt are non-monotonic and model everyday reasoning very well. We model the relevant rules with probabilities as follows:

$$
\begin{array}{lll}
P(\text{bird} \mid \text{penguin}) & = 1 & \text{“penguins are birds”} \\
P(\text{flies} \mid \text{bird}) & \in [0.95, 1] & \text{“(almost all) birds can fly”} \\
P(\text{flies} \mid \text{penguin}) & = 0 & \text{“penguins cannot fly”}
\end{array}
$$

The first and third rules represent firm predictions, which can also be easily formulated in logic. In the second, however, we express our knowledge that almost all birds can fly by means of a probability interval. With the PIT input data

```
var penguin{yes,no}, bird{yes,no}, flies{yes,no};
P([bird=yes] | [penguin=yes]) = 1;
P([flies=yes] | [bird=yes]) IN [0.95,1];
P([flies=yes] | [penguin=yes]) = 0;
QP([flies=yes]| [penguin=yes]);
```

we get back the correct answer

Nr.	Truthvalue	Probability	Query
1	UNSPECIFIED	$0.000e+00$	QP ([penguin = yes]− |> [flies = yes]);

with the proposition that penguins cannot fly. [7] The explanation for this is very simple. With $P(\text{flies} \mid \text{bird}) \in [0.95, 1]$ it is possible that there are non-flying birds. If this rule were replaced by $P(\text{flies} \mid \text{bird}) = 1$, then PIT would not be able to do anything and would output an error message about inconsistent constraints.

In this example we can easily see that probability intervals are often very helpful for modeling our ignorance about exact probability values. We could have made an even fuzzier formulation of the second rule in the spirit of "normally birds fly" with $P(\text{flies} \mid \text{bird}) \in (0.5, 1]$. The use of the half-open interval excludes the value 0.5.

It has already been shown in [Pea88] that this example can be solved using probabilistic logic, even without MaxEnt. In [Sch96] it is shown for a number of demanding benchmarks for non-monotonic reasoning that these can be solved elegantly with MaxEnt. In the following section we introduce a successful practical application of MaxEnt in the form of a medical expert system.

0.28.7 Lexmed, an Expert System for Diagnosing Appendicitis

The medical expert system Lexmed, which uses the MaxEnt method, was developed at the Ravensburg-Weingarten University of Applied Sciences by Manfred Schramm, Walter Rampf, and the author, in cooperation with the Weingarten 14-Nothelfer Hospital [SE00, [**?**]]. [8] The acronym Lexmed stands for learning expert system for medical diagnosis.

0.28.8 Appendicitis Diagnosis with Formal Methods

The most common serious cause of acute abdominal pain [[**?**]] is appendicitis—an inflammation of the appendix, a blind-ended tube connected to the cecum. Even today, diagnosis can be difficult in many cases [OFY^{+}95]. For example, up to about 20% of the removed appendices are without pathological findings, which means that the operations were unnecessary. Likewise, there are regularly cases in which an inflamed appendix is not recognized as such.

Since as early as the beginning of the 1970s, there have been attempts to automate the diagnosis of appendicitis, with the goal of reducing the rate of false

[7] QP ([penguin=yes]-|> [flies=yes]) is an alternative form of the PIT syntax for QP ([flies=yes] | [penguin=yes]).

[8] The project was financed by the German state of Baden-Württemberg, the health insurance company AOK Baden-Württemberg, the Ravensburg-Weingarten University of Applied Sciences, and the 14 Nothelfer Hospital in Weingarten.

diagnoses [dDLS+72, [?], [?]]. Especially noteworthy is the expert system for diagnosis of acute abdominal pain, developed by de Dombal in Great Britain. It was made public in 1972, thus distinctly earlier than the famous system MYCIN. Nearly all of the formal diagnostic processes used in medicine to date have been based on scores. Score systems are extremely easy to apply: For each value of a symptom (for example fever or lower right stomach pain) the doctor notes a certain number of points. If the sum of the points is over a certain value (threshold), a certain decision is recommended (for example operation). For n symptoms $S_1, \ldots, S_n$ a score for appendicitis can be described formally as

$$\text{Diagnose} = \begin{cases} \text{Appendicitis} & \text{if } w_1 S_1 + \cdots + w_n S_n > \Theta, \\ \text{negative} & \text{else.} \end{cases}$$

With scores, a linear combination of symptom values is thus compared with a threshold Θ. The weights of the symptoms are extracted from databases using statistical methods. The advantage of scores is their simplicity of application. The weighted sum of the points can be computed by hand easily and a computer is not needed for the diagnosis.

Because of the linearity of this method, scores are too weak to model complex relationships. Since the contribution $w_i S_i$ of a symptom S_i to the score is calculated independently of the other symptoms, it is clear that score systems cannot take any "context" into account. Principally, they cannot distinguish between combinations of symptoms, for example they cannot distinguish between the white blood cell count of an old patient and that of a young patient.

For a fixed given set of symptoms, conditional probability is much more powerful than scores for making predictions because the latter cannot describe the dependencies between different symptoms. We can show that scores implicitly assume that all symptoms are independent.

When using scores, yet another problem comes up. To arrive at a good diagnosis quality, we must put strict requirements on the databases used to statistically determine the weights w_i. In particular they must be representative of the set of patients in the area in which the diagnosis system is used. This is often difficult, if not impossible, to guarantee. In such cases, scores and other statistical methods either cannot be used, or will have a high rate of errors.

0.28.9 Hybrid Probabilistic Knowledge Base

Complex probabilistic relationships appear frequently in medicine. With Lexmed, these relationships can be modeled well and calculated quickly. Here the use of probabilistic propositions, with which uncertain and incomplete information can be expressed and processed in an intuitive and mathematically grounded way, is essential. The following question may serve as a typical query against the expert system: "How high is the probability of an inflamed appendix if the patient is a 23-year-old man with pain in the right lower abdomen and a white blood cell count

Table 7.2 Symptoms used for the query in Lexmed and their values. The number of values for the each symptom is given in the column marked #

Symptom	Values	#	Short
Gender	Male, female	2	Sex2
Age	0-5, 6-10, 11-15, 16-20, 21-25, 26-35, 36-45, 46-55, 56-65, 65-	10	Age10
Pain 1st Quad.	Yes, no	2	P1Q2
Pain 2nd Quad.	Yes, no	2	P2Q2
Pain 3rd Quad.	Yes, no	2	P3Q2
Pain 4th Quad.	Yes, no	2	P4Q2
Guarding	Local, global, none	3	Gua3
Rebound tenderness	Yes, no	2	Reb2
Pain on tapping	Yes, no	2	Tapp2
Rectal pain	Yes, no	2	RecP2
Bowel sounds	Weak, normal, increased, none	4	BowS4
Abnormal ultrasound	Yes, no	2	Sono2
Abnormal urine sedim.	Yes, no	2	Urin2
Temperature (rectal)	-37.3, 37.4-37.6, 37.7-38.0, 38.1-38.4, 38.5-38.9, 39.0-	6	TRec6
Leukocytes	0-6k, 6k-8k, 8k-10k, 10k-12k, 12k-15k, 15k-20k, 20k-	7	Leuko7
Diagnosis	Inflamed, perforated, negative, other	4	Diag4

of 13,000 ?" Formulated as conditional probability, using the names and value ranges for the symptoms used in Table 7.2, this reads

$$P(\text{ Diag } 4 = \text{ inflamed } \vee \text{ Diag } 4 = \text{ perforated } \mid$$
$$\text{Sex } 2 = \text{ male } \wedge \text{ Age } 10 \in 21 - 25 \wedge \text{ Leuko } 7 \in 12k - 15k).$$

By using probabilistic propositions, Lexmed has the ability to use information from non-representative databases because this information can be complemented appropriately from other sources. Underlying Lexmed is a database which only contains data about patients whose appendixes were surgically removed. With statistical methods, (about 400) rules are generated which compile the knowledge contained in the database into an abstracted form [ES99]. Because there are no patients in this database who were suspected of having appendicitis but had negative diagnoses (that is, not requiring treatment), [9] there is no knowledge about negative patients in the database. Thus knowledge from other sources must be added in. In Lexmed therefore the rules gathered from the database are complemented by (about 100) rules from medical experts and the medical literature. This results in a hybrid probabilistic database, which contains knowledge extracted from data as well as knowledge explicitly formulated by experts. Because both types of rules are formulated as conditional probabilities (see for example (7.14) on page 152), they can be easily combined, as shown in Fig. 7.5 on page 148 and with more details in Fig. 7.7 on page 150.

[29] These negative diagnoses are denoted "non-specific abdominal pain" (NSAP).

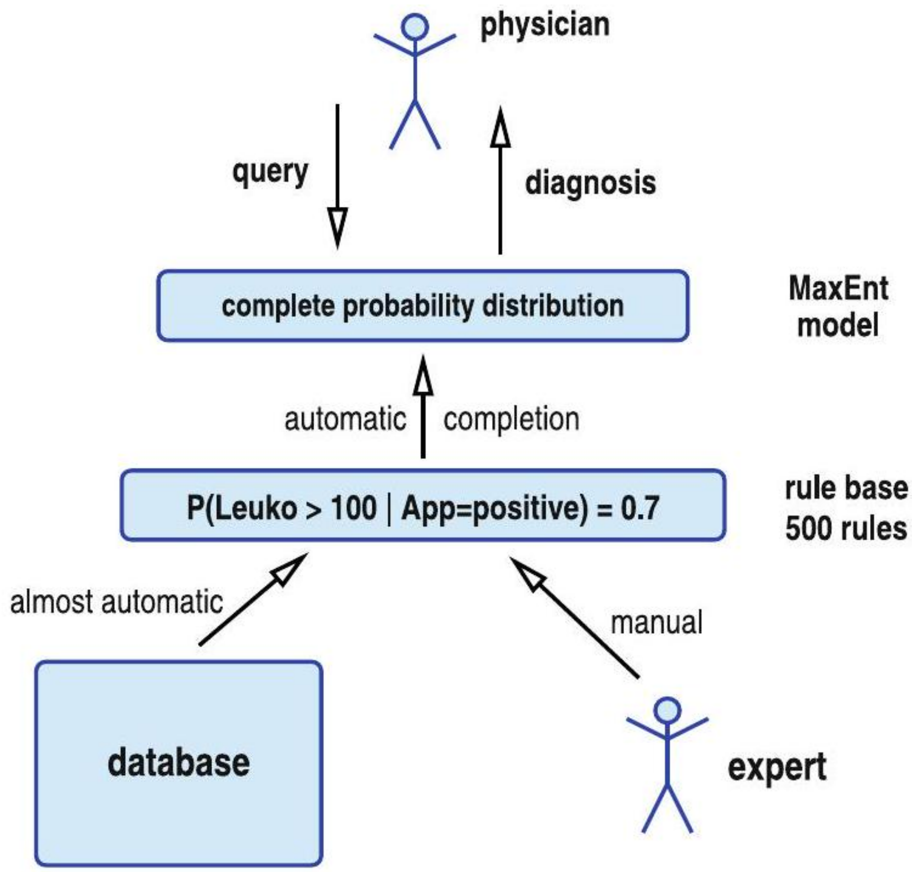


Figure 36: Probabilistic rules are generated from data and expert knowledge, which are integrated in a rule base (knowledge base) and finally made complete using the MaxEnt method

Lexmed calculates the probabilities of various diagnoses using the probability distribution of all relevant variables (see Table 7.2 on page 147). Because all 14 symptoms used in Lexmed and the diagnoses are modeled as discrete variables (even continuous variables like the leukocyte value are divided into ranges), the size of the distribution (that is, the size of the event space) can be determined using Table 7.2 on page 147 as the product of the number of values of all symptoms, or

$$2^{10} \cdot 10 \cdot 3 \cdot 4 \cdot 6 \cdot 7 \cdot 4 = 20643840$$

elements. Due to the normalization condition from Theorem 7.1 on page 129, it thus has 20643839 independent values. Every rule set with fewer than 20643839 probability values potentially does not completely describe this event space. To be able to answer any arbitrary query, the expert system needs a complete distribution. The construction of such an extensive, consistent distribution using statistical methods is very difficult. [10] To require from a human expert all 20643839 values for the distribution (instead of the aforementioned 100 rules) would essentially be impossible.

Here the MaxEnt method comes into play. The generalization of about 500 rules to a complete probability model is done in Lexmed by maximizing the entropy with the 500 rules as constraints. An efficient encoding of the resulting MaxEnt distribution leads to response times for the diagnosis of around one second.

Personal Details	unknown	values
Gender	–	✓ male ∼ female
Age-group	r	($0.5 \sim 6 - 10 \sim 11 - 15 \sim 16 - 20 \cdot 21 - 25 \sim 26 - 35 \sim 36 - 45 \sim 4$
Results of examination	not done	values
1st quadrant	-	✓ yes ∼ no
2nd quadrant	-	(yes) no
3rd quadrant	c	- yes ~ no
4th quadrant	o	✓ yes ∼ no
guarding	c	- local Γ global C none
rebound tenderness	r	- yes ∼ no
pain on tapping	c	- yes ∼ no
rectal pain	-	r yes r no
bowel sounds	c	✓ weak ∗ normal ∼ increased ∼ none
abnormal ultrasound	-	✓ yes ∼ no
abnormal urine sediment	-	✓ yes ∼ no
temperature range (rectal)	r	$(37.3)37.437.6)37.738.0 \sim 38.138.4 \cdot 38.538.9 \sim 39.0$
leucocyte count	c	$\sim 0 - 6k \sim 6k - 8k \sim 8k - 10k \sim 10k - 12k \sim 12k - 15k \sim 15k - 2$
		Result of the PIT diagnosis
Diagnosis	App. inflamed	App. perforated
Probability	0.70	0.17

Fig. 7.6 The Lexmed input mask for input of the examined symptoms and below it the output of the resulting diagnosis probabilities

0.28.10 Application of Lexmed

The usage of Lexmed is simple and self-explanatory. The doctor visits the Lexmed home page at www.lexmed.de. [11] For an automatic diagnosis, the doctor inputs

[210] The task of generating a function from a set of data is known as machine learning. We will cover this thoroughly in Chap. 8.

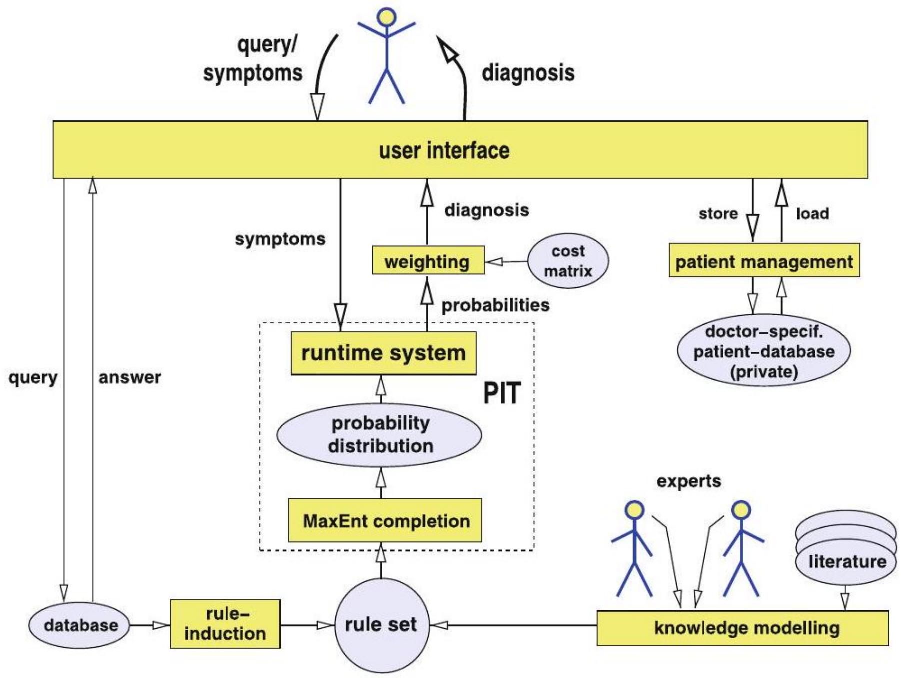


Figure 37: Rules are generated from the database as well as from expert knowledge. From these, MaxEnt creates a complete probability distribution. For a user query, the probability of every possible diagnosis is calculated. Using the cost matrix (see Sect. 7.3.5) a decision is then suggested

the results of his examination into the input form in Fig. 7.6. After one or two seconds he receives the probabilities for the four different diagnoses as well as a suggestion for a treatment (Sect. 7.3.5). If certain examination results are missing as input (for example the sonogram results), then the doctor chooses the entry not examined. Naturally the certainty of the diagnosis is higher when more symptom values are input.

Each registered user has access to a private patient database, in which input data can be archived. Thus data and diagnoses from earlier patients can be easily compared with those of a new patient. An overview of the processes in Lexmed is given in Fig. 7.7.

0.28.11 Function of Lexmed

Knowledge is formalized using probabilistic propositions. For example, the proposition

[211] A version with limited functionality is accessible without a password.

$$P(\text{ Leuko } 7 > 20000 \mid \text{ Diag } 4 = \text{ inflamed }) = 0.09$$

gives a frequency of 9% for a leukocyte value of more than 20,000 in case of an inflamed appendix. [12]

0.29 Learning of Rules by Statistical Induction

The raw data in Lexmed's database contain 54 different (anonymized) values for 14,646 patients. As previously mentioned, only patients whose appendixes were surgically removed are included in this database. Of the 54 attributes used in the

database, after a statistical analysis the 14 symptoms shown in Table 7.2 on page 147 were used. Now the rules are created from this database in two steps. The first step determines the dependency structure of the symptoms. The second step fills this structure with the respective probability rules. [13]

Determining the Dependency Graph The graph in Fig. 7.8 contains for each variable (the symptom and the diagnosis) a node and directed edges which connect various nodes. The thickness of the edges between the variables represents a measure of the statistical dependency or correlation of the variables. The correlation of two independent variables is equal to zero. The pair correlation for each of the 14 symptoms with Diag4 was computed and listed in the graph. Furthermore, all triple correlations between the diagnosis and two symptoms were calculated. Of these, only the strongest values have been drawn as additional edges between the two participating symptoms.

```
P([Leuco7=0-6k] | [Diag4=negativ] * [Age10=16-20]) = [0.132,0.156];
P([Leuco7=6-8k] | [Diag4=negativ] * [Age10=16-20]) = [0.257,0.281];
P([Leuco7=8-10k] | [Diag4=negativ] * [Age10=16-20]) = [0.250,0.274];
P([Leuco7=10-12k] | [Diag4=negativ] * [Age10=16-20]) = [0.159,0.183];
P([Leuco7=12-15k] [Diag4=negativ] * [Age10=16-20]) = [0.087,0.112];
P([Leuco7=15-20k] | [Diag4=negativ] * [Age10=16-20]) = [0.032,0.056];
P([Leuco7=20k-] | [Diag4=negativ] * [Age10=16-20]) = [0.000,0.023];
P([Leuco7=0-6k] | [Diag4=negativ] * [Age10=21-25]) = [0.132,0.172];
P([Leuco7=6-8k] | [Diag4=negativ] * [Age10=21-25]) = [0.227,0.266];
P([Leuco7=8-10k] | [Diag4=negativ] * [Age10=21-25]) = [0.211,0.250];
P([Leuco7=10-12k] | [Diag4=negativ] * [Age10=21-25]) = [0.166,0.205];
P([Leuco7=12-15k] | [Diag4=negativ] * [Age10=21-25]) = [0.081,0.120];
P([Leuco7=15-20k] | [Diag4=negativ] * [Age10=21-25]) = [0.041,0.081];
P([Leuco7=20k-] | [Diag4=negativ] * [Age10=21-25]) = [0.004,0.043];
```

Fig. 7.9 Some of the Lexmed rules with probability intervals. "*" stands for " $\wedge$ " here

[212] Instead of individual numerical values, intervals can also be used here (for example [0.06, 0.12]).

[213] For a systematic introduction to machine learning we refer the reader to Chap. 8.

Estimating the Rule Probabilities The structure of the dependency graph describes the structure of the learned rules. [14] The rules here have different complexities: there are rules which only describe the distribution of the possible diagnoses (a priori rules, for example (7.13)), rules which describe the dependency between the diagnosis and a symptom (rules with simple conditions, for example (7.14)), and finally rules which describe the dependency between the diagnosis and two symptoms, as given in Fig. 7.9 in Pit syntax.

$$\begin{aligned} P(\text{ Diag } 4 = \text{ inflamed }) &= 0.40, \\ P(\text{ Sono } 2 = \text{ yes } \mid \text{ Diag } 4 = \text{ inflamed }) &= 0.43, \\ P(P4Q2 = \text{ yes } \mid \text{ Diag } 4 = \text{ inflamed } \wedge P2Q2 = \text{ yes }) &= 0.61. \end{aligned} \tag{7.15}$$

To keep the context dependency of the saved knowledge as small as possible, all rules contain the diagnosis in their conditions and not as conclusions. This is quite similar to the construction of many medical books with formulations of the kind ”With appendicitis we usually see ...”. As previously shown in Example 7.6 on page 133, however, this does not present a problem because, using the Bayesian formula, Lexmed automatically puts these rules into the right form.

The numerical values for these rules are estimated by counting their frequency in the database. For example, the value in (7.14) is given by counting and calculating

$$\frac{\mid \text{ Diag4 } = \text{ inflamed } \wedge \text{ Sono } 2 = \text{ yes } \mid}{\mid \text{ Diag } 4 = \text{ inflamed } \mid} = 0.43.$$

0.30 Expert Rules

Because the appendicitis database only contains patients who have undergone the operation, rules for non-specific abdominal pain (NSAP) receive their values from propositions of medical experts. The experiences in Lexmed confirm that the probabilistic rules are easy to read and can be directly translated into natural language. Statements by medical experts about frequency relationships of specific symptoms and the diagnosis, whether from the literature or as the result of an interview, can therefore be incorporated into the rule base with little expense. To model the uncertainty of expert knowledge, the use of probability intervals has proven effective. The expert knowledge was primarily acquired from the participating surgeons, Dr. Rampf and Dr. Hontschik, and their publications [[**?**]]. Once the expert rules have been created, the rule base is finished. Then the complete probability model is calculated with the method of maximum entropy by the Pit-system.

[214] The difference between this and a Bayesian network is, for example, that the rules are equipped with probability intervals and that only after applying the principle of maximum entropy is a unique probability model produced.

0.31 Diagnosis Queries

Using its efficiently stored probability model, Lexmed calculates the probabilities for the four possible diagnoses within a few seconds. For example, we assume the following output:

	Results of the PIT diagnosis			
Diagnosis	Appendix inflamed	Appendix perforated	Negative	Other
Probability	0.24	0.16	0.57	0.03

A decision must be made based on these four probability values to pursue one of the four treatments: operation, emergency operation, stationary observation, or ambulant observation. [15] While the probability for a negative diagnosis in this case outweighs the others, sending the patient home as healthy is not a good decision. We can clearly see that, even when the probabilities of the diagnoses have been calculated, the diagnosis is not yet finished.

Rather, the task is now to derive an optimal decision from these probabilities. To this end, the user can have Lexmed calculate a recommended decision.

0.31.1 Risk Management Using the Cost Matrix

How can the computed probabilities now be translated optimally into decisions? A naive algorithm would assign a decision to each diagnosis and ultimately select the decision that corresponds to the highest probability. Assume that the computed probabilities are 0.40 for the diagnosis appendicitis (inflamed or perforated), 0.55 for the diagnosis negative, and 0.05 for the diagnosis other. A naive algorithm would now choose the (too risky) decision ”no operation” because it corresponds to the diagnosis with the higher probability. A better method consists of comparing

Table 7.3 The cost matrix of LeXMED together with a patient's computed diagnosis probabilities

Therapy	Probability of various diagnoses				
	Inflamed	Perforated	Negative	Other	
	0.25	0.15	0.55	0.05	
Operation	0	500	5800	6000	3565
Emergency operation	500	0	6300	6500	3915
Ambulant observ.	12000	150000	0	16500	26325
Other	3000	5000	1300	0	2215
Stationary observ.	3500	7000	400	600	2175

the costs of the possible errors that can occur for each decision. The error is quantified in the form of ”(hypothetical) additional cost of the current decision

[215] Ambulant observation means that the patient is released to stay at home.

compared to the optimum". The given values contain the costs to the hospital, to the insurance company, the patient (for example risk of post-operative complications), and to other parties (for example absence from work), taking into account long term consequences. These costs are given in Table 7.3.

The entries are finally averaged for each decision, that is, summed while taking into account their frequencies. These are listed in the last column in Table 7.3. Finally, the decision with the smallest average cost of error is suggested. In Table 7.3 the matrix is given together with the probability vector calculated for a patient (in this case: $(0.25, 0.15, 0.55, 0.05)$). The last column of the table contains the result of the calculations of the average expected costs of the errors. The value of Operation in the first row is thus calculated as $0.25 \cdot 0 + 0.15 \cdot 500 + 0.55 \cdot 5800 + 0.05 \cdot 6000 = 3565$, a weighted average of all costs. The optimal decisions are entered with (additional) costs of 0. The system decides on the treatment with the minimal average cost. It thus is an example of a cost-oriented agent.

0.32 Cost Matrix in the Binary Case

To better understand the cost matrix and risk management we will now restrict the Lexmed system to the two-value decision between the diagnosis appendicitis with probability

$$p_1 = P(\text{ appendicitis }) = P(\text{ Diag } 4 = \text{ inflamed }) + P(\text{ Diag } 4 = \text{ perforated })$$

and NSAP with the probability

$$p_2 = P(NSAP) = P(\text{ Diag } 4 = \text{ negative }) + P(\text{ Diag } 4 = \text{ other })$$

The only available treatments are operation and ambulant observation. The cost matrix is thus a 2×2 matrix of the form

$$\begin{pmatrix} 0 & k_2 \\ k_1 & 0 \end{pmatrix}$$

The two zeroes in the diagonal stand for the correct decision operation in the case of appendicitis and ambulant observation for NSAP. The parameter k_2 stands for the expected costs which occur when a patient without an inflamed appendix is operated on. This error is called a false positive. On the other hand, the decision ambulant observation in the case of appendicitis is a false negative. The probability vector $(p_1, p_2)^T$ is now multiplied by this matrix and we obtain the vector

$$(k_2 p_2, k_1 p_1)^T$$

with the average additional cost for the two possible treatments. Because the decision only takes into account the relationship of the two components, the vector can be multiplied by any scalar factor. We choose $1/k_1$ and obtain $((k_2/k_1)\,p_2, p_1)$. Thus only the relationship $k = k_2/k_1$ is relevant here. The same result is obtained by the simpler cost matrix

$$\begin{pmatrix} 0 & k \\ 1 & 0 \end{pmatrix}$$

which only contains the variable k. This parameter is very important because it determines risk management. By changing k we can fit the "working point" of the diagnosis system. For $k \to \infty$ the system is put in an extremely risky setting because no patient will ever be operated on, with the consequence that it gives no false positive classifications, but many False negatives. In the case of $k = 0$ the conditions are in exact reverse and all patients are operated upon.

0.32.1 Performance

Lexmed is intended for use in a medical practice or ambulance. Prerequisites for the use of Lexmed are acute abdominal pain for several hours (but less than five days). Furthermore, Lexmed is (currently) specialized for appendicitis, which means that for other illnesses the system contains very little information.

In the scope of a prospective study, a representative database with 185 cases was created in the 14 Nothelfer Hospital. It contains the hospital's patients who came to the clinic after several hours of acute abdominal pain and suspected appendicitis. From these patients, the symptoms and the diagnosis (verified from a tissue sample in the case of an operation) is noted.

If the patients were released to go home (without operation) after a stay of several hours or 1-2 days with little or no complaint, it was afterwards inquired by telephone whether the patient remained free of symptoms or whether a positive diagnosis was found in subsequent treatment.

To simplify the representation and make for a better comparison to similar studies, Lexmed was restricted to the two-value distinction between appendicitis and NSAP, as described in Sect. 7.3.5. Now k is varied between zero and infinity
and for every value of k the sensitivity and specificity are measured against the test data. Sensitivity measures

$$P(\text{ classified positive } \mid \text{ positive }) = \frac{\mid \text{ positive and classified positive } \mid}{\mid \text{ positive } \mid}, \tag{7.16}$$

that is, the relative portion of positive cases which are correctly identified. It indicates how sensitive the diagnostic system is. Specificity, on the other hand, measures

$$P(\text{ classified negative } \mid \text{ negative }) = \frac{\mid \text{ negative and classified negative } \mid}{\mid \text{ negative } \mid}, \tag{7.17}$$

that is, the relative portion of negative cases which are correctly identified. We give the results of the sensitivity and specificity in Fig. 7.10 for $0 \leq k < \infty$. This curve is denoted the ROC curve, or receiver operating characteristic. Before we come to the analysis of the quality of Lexmed, a few words about the meaning of the ROC curve. The line bisecting the diagram diagonally is drawn in for orientation. All points on this line correspond to a random decision. For example, the point (0.2, 0.2) corresponds to a specificity of 0.8 with a sensitivity of 0.2 . We can arrive at this quite easily by classifying a new case, without looking at it, with probabilities 0.2 for positive and 0.8 for negative. Every knowledge-based diagnosis system must therefore generate a ROC which clearly lies above the diagonal.

The extreme values in the ROC curve are also interesting. At point $(0, 0)$ all three curves intersect. The corresponding diagnosis system would classify all cases as negative. The other extreme value $(1, 1)$ corresponds to a system which would decide to do the operation for every patient and thus has a sensitivity of 1. We could call the ROC curve the characteristic curve for two-value diagnostic systems. The ideal diagnostic system would have a characteristic curve which consists only of the point $(0, 1)$, and thus has 100% specificity and 100% sensitivity. Now let us analyse the ROC curve. At a sensitivity of 88%, Lexmed attains a specificity of $87\%(k = 0.6)$. For comparison, the Ohmann score, an established, well-known score for appendicitis is given [[**?**], [**?**]]. Because Lexmed is above or to the left of the Ohmann score almost everywhere, its average quality of diagnosis is clearly better. This is not surprising because scores are simply too weak to model interesting propositions. In Sect. 8.7 and in Exercise 8.17 on page 242 we will show that scores are equivalent to the special case of naive Bayes, that is, to the assumption that all symptoms are pairwise independent when the diagnosis is known. When comparing Lexmed with scores it should, however, be mentioned that a statistically representative database was used for the Ohmann score, but a non-representative database enhanced with expert knowledge was used for Lexmed. To get an idea of the quality of the Lexmed data in comparison to the Ohmann data, a linear score was calculated using the least squares method (see Sect. 9.4.1), which is also drawn for comparison. Furthermore, a neural network was trained on the Lexmed data with the RProp algorithm (see Sect. 9.5). The strength of combining data and expert knowledge is displayed clearly in the difference between the Lexmed curve and the curves of the score system and the RProp algorithm.

0.32.2 Application Areas and Experiences

Lexmed should not replace the judgment of an experienced surgeon. However, because a specialist is not always available in a clinical setting, a Lexmed query offers a substantive second opinion. Especially interesting and worthwhile is the application of the system in a clinical ambulance and for general practitioners.

The learning capability of Lexmed, which makes it possible to take into account further symptoms, further patient data, and further rules, also presents new possibilities in the clinic. For especially rare groups which are difficult to diagnose, for example children under six years of age, Lexmed can use data from pediatricians or other special databases, to support even experienced surgeons.

Aside from direct use in diagnosis, Lexmed also supports quality assurance measures. For example, insurance companies can compare the quality of diagnosis of hospitals with that of expert systems. By further developing the cost matrix created in Lexmed (with the consent of doctors, insurance, and patients), the quality of physician diagnoses, computer diagnoses, and other medical institutions will become easier to compare.

Lexmed has pointed to a new way of constructing automatic diagnostic systems. Using the language of probability theory and the MaxEnt algorithm, inductively,
statistically derived knowledge is combined with knowledge from experts and from the literature. The approach based on probabilistic models is theoretically elegant, generally applicable, and has given very good results in a small study.

Lexmed has been in practical use in the 14 Nothelfer Hospital in Weingarten since 1999 and has performed there very well. It is also available at www.lexmed.de, without warranty, of course. Its quality of diagnosis is comparable with that of an experienced surgeon and is thus better than that of an average general practitioner, or that of an inexperienced doctor in the clinic.

Despite this success it has become evident that it is very difficult to market such a system commercially in the German medical system. One reason for this is that there is no free market to promote better quality (here better diagnoses) through its selection mechanisms. Furthermore, in medicine the time for broad use of intelligent techniques is not yet at hand-even in 2010. One cause of this could be conservative teachings in this regard in German medical school faculties.

A further issue is the desire of many patients for personal advice and care from the doctor, together with the fear that, with the introduction of expert systems, the patient will only communicate with the machine. This fear, however, is wholly unfounded. Even in the long term, medical expert systems cannot replace the doctor. They can, however, just like laser surgery and magnetic resonance imaging, be used advantageously for all participants. Since the first medical computer diagnostic system of de Dombal in 1972, almost 40 years have passed. It remains to be seen whether medicine will wait another 40 years until computer diagnostics becomes an established medical tool.

0.32.3 Reasoning with Bayesian Networks

One problem with reasoning using probability in practice was already pointed out in Sect. 7.1. If d variables $X_1, \ldots, X_d$ with n values each are used, then the associated probability distribution has n^d total values. This means that in the worst case the memory use and computation time for determining the specified probabilities grows exponentially with the number of variables.

In practice the applications are usually very structured and the distribution contains many redundancies. This means that it can be heavily reduced with the appropriate methods. The use of Bayesian networks has proved its power here and is one of the AI techniques which have been successfully used in practice. Bayesian networks utilize knowledge about the independence of variables to simplify the model.

0.32.4 Independent Variables

In the simplest case, all variables are pairwise independent and it is the case that

$$\boldsymbol{P}\left(X_1, \ldots, X_d\right) = \boldsymbol{P}\left(X_1\right) \cdot \boldsymbol{P}\left(X_2\right) \cdots \boldsymbol{P}\left(X_d\right)$$

All entries in the distribution can thus be calculated from the d values $P\left(X_1\right), \ldots, P\left(X_d\right)$. Interesting applications, however, can usually not be modeled because conditional probabilities become trivial. [16] Because of

$$P(A \mid B) = \frac{P(A, B)}{P(B)} = \frac{P(A)P(B)}{P(B)} = P(A)$$

all conditional probabilities are reduced to the a priori probabilities. The situation becomes more interesting when only a portion of the variables are independent or independent under certain conditions. For reasoning in AI, the dependencies between variables happen to be important and must be utilized.

We would like to outline reasoning with Bayesian networks through a simple and very illustrative example by J. Pearl [Pea88], which became well known through [RN10] and is now basic AI knowledge.

Example 7.10 (Alarm-Example) Bob, who is single, has had an alarm system installed in his house to protect against burglars. Bob cannot hear the alarm when he is working at the office. Therefore he has asked his two neighbors, John in the house next door to the left, and Mary in the house to the right, to call him at his office if they hear his alarm. After a few years Bob knows how reliable John and Mary are and models their calling behavior using conditional probability as follows.[17]

$$\begin{aligned} P(J \mid \text{Al}) &= 0.90 & P(M \mid \text{Al}) &= 0.70 \\ P(J \mid \neg\text{Al}) &= 0.05 & P(M \mid \neg\text{Al}) &= 0.01 \end{aligned}$$

Because Mary is hard of hearing, she fails to hear the alarm more often than John. However, John sometimes mixes up the alarm at Bob's house with the alarms at other houses. The alarm is triggered by a burglary, but can also be triggered by a (weak) earthquake, which can lead to a false alarm because Bob only wants to know about burglaries while at his office. These relationships are modeled by

$$P(\text{Al} \mid \text{Bur}, \text{Ear}) = 0.95,$$
$$P(\text{Al} \mid \text{Bur}, \neg\text{Ear}) = 0.94,$$
$$P(\text{Al} \mid \neg\text{Bur}, \text{Ear}) = 0.29,$$
$$P(\text{Al} \mid \neg\text{Bur}, \neg\text{Ear}) = 0.001,$$

as well as the a priori probabilities $P(\text{Bur}) = 0.001$ and $P(\text{Ear}) = 0.002$. These two variables are independent because earthquakes do not make plans based on the habits of burglars, and conversely there is no way to predict earthquakes, so burglars do not have the opportunity to set their schedule accordingly.

Queries are now made against this knowledge base. For example, Bob might be interested in $P(\text{Bur} \mid J \vee M)$, $P(J \mid \text{Bur})$ or $P(M \mid \text{Bur})$. That is, he wants to know how sensitively the variables J and M react to a burglary report.

0.32.5 Graphical Representation of Knowledge as a Bayesian Network

We can greatly simplify practical work by graphically representing knowledge that is formulated as conditional probability. Figure 7.11 shows the Bayesian network for the alarm example. Each node in the network represents a variable and every directed edge a statement of conditional probability. The edge from Al to J, for example, represents the two values $P(J \mid Al)$ and $P(J \mid \neg Al)$, which is given in the form of a table, the so-called CPT (conditional probability table). The CPT of a node lists all the conditional probabilities of the node's variable conditioned on all the nodes connected by incoming edges.

While studying the network, we might ask ourselves why there are no other edges included besides the four that are drawn in. The two nodes Bur and Ear are not linked since the variables are independent. All other nodes have a parent node, which makes the reasoning a little more complex. We first need the concept of conditional independence.

0.32.6 Conditional Independence

Analogously to independence of random variables, we give

Definition 7.6 Two variables A and B are called conditionally independent, given C if

$$\boldsymbol{P}(A, B \mid C) = \boldsymbol{P}(A \mid C) \cdot \boldsymbol{P}(B \mid C)$$

[216] In the naive Bayes method, the independence of all attributes is assumed, and this method has been successfully applied to text classification (see Sect. 8.7).

[17] The binary variables J and M stand for the two events "John calls", and "Mary calls", respectively, *Al* for "alarm siren sounds", Bur for "burglary" and Ear for "earthquake".

This equation is true for all combinations of values for all three variables (that is, for the distribution), which we see in the notation. We now look at nodes J and M in the alarm example, which have the common parent node Al. If John and Mary independently react to an alarm, then the two variables J and M are independent given Al, that is:

$$\boldsymbol{P}(J, M \mid Al) = \boldsymbol{P}(J \mid Al) \cdot \boldsymbol{P}(M \mid Al)$$

If the value of Al is known, for example because an alarm was triggered, then the variables J and M are independent (under the condition $Al = w$). Because of the conditional independence of the two variables J and M, no edge between these two nodes is added. However, J and M are not independent (see Exercise 7.11 on page 173).

Quite similar is the relationship between the two variables J and Bur, because John does not react to a burglary, rather the alarm. This could be, for example, because of a high wall that blocks his view on Bob's property, but he can still hear the alarm. Thus J and Bur are independent given Al and

$$\boldsymbol{P}(J, Bur \mid Al) = \boldsymbol{P}(J \mid Al) \cdot \boldsymbol{P}(Bur \mid Al)$$

Given an alarm, the variables J and Ear, M and Bur, as well as M and Ear are also independent. For computing with conditional independence, the following characterizations, which are equivalent to the above definition, are helpful:

Theorem 7.5 The following equations are pairwise equivalent, which means that each individual equation describes the conditional independence for the variables A and B given C.

$$\boldsymbol{P}(A, B \mid C) = \boldsymbol{P}(A \mid C) \cdot \boldsymbol{P}(B \mid C) \quad (7.18)$$

$$\boldsymbol{P}(A \mid B, C) = \boldsymbol{P}(A \mid C) \quad (7.19)$$

$$\boldsymbol{P}(B \mid A, C) = \boldsymbol{P}(B \mid C) \quad (7.20)$$

Proof On one hand, using conditional independence (7.18) we can conclude that

$$\boldsymbol{P}(A, B, C) = \boldsymbol{P}(A, B \mid C)\boldsymbol{P}(C) = \boldsymbol{P}(A \mid C)\boldsymbol{P}(B \mid C)\boldsymbol{P}(C)$$

On the other hand, the product rule gives us

$$\boldsymbol{P}(A, B, C) = \boldsymbol{P}(A \mid B, C)\boldsymbol{P}(B \mid C)\boldsymbol{P}(C)$$

Thus $\boldsymbol{P}(A \mid B, C) = \boldsymbol{P}(A \mid C)$ is equivalent to (7.18). We obtain (7.20) analogously by swapping A and B in this derivation.

0.32.7 Practical Application

Now we turn again to the alarm example and show how the Bayesian network in Fig. 7.11 can be used for reasoning. Bob is interested, for example, in the sensitivity of his two alarm reporters John and Mary, that is, in $P(J \mid Bur)$ and $P(M \mid Bur)$. However, the values $P(Bur \mid J)$ and $P(Bur \mid M)$, as well as $P(Bur \mid J, M)$ are even more important to him. We begin with $P(J \mid Bur)$ and calculate

$$P(J \mid Bur) = \frac{P(J, Bur)}{P(Bur)} = \frac{P(J, Bur, Al) + P(J, Bur, \neg Al)}{P(Bur)} \tag{7.21}$$

and

$$\boldsymbol{P}(J, Bur, Al) = \boldsymbol{P}(J \mid Bur, Al)\boldsymbol{P}(Al \mid Bur)\boldsymbol{P}(Bur) = \boldsymbol{P}(J \mid Al)\boldsymbol{P}(Al \mid Bur)\boldsymbol{P}(Bur), \tag{7.22}$$

where for the last two equations we have used the product rule and the conditional independence of J and Bur given Al. Inserted in (7.21) we obtain

$$\begin{aligned} P(J \mid Bur) &= \frac{P(J \mid Al)P(Al \mid Bur)P(Bur) + P(J \mid \neg Al)P(\neg Al \mid Bur)P(Bur)}{P(Bur)} \\ &= P(J \mid Al)P(Al \mid Bur) + P(J \mid \neg Al)P(\neg Al \mid Bur). \end{aligned} \tag{7.23}$$

Here $P(Al \mid Bur)$ and $P(\neg Al \mid Bur)$ are missing. Therefore we calculate

$$\begin{aligned} P(Al \mid Bur) &= \frac{P(Al, Bur)}{P(Bur)} = \frac{P(Al, Bur, Ear) + P(Al, Bur, \neg Ear)}{P(Bur)} \\ &= \frac{P(Al \mid Bur, Ear)P(Bur)P(Ear) + P(Al \mid Bur, \neg Ear)P(Bur)P(\neg Ear)}{P(Bur)} \\ &= P(Al \mid Bur, Ear)P(Ear) + P(Al \mid Bur, \neg Ear)P(\neg Ear) \\ &= 0.95 \cdot 0.002 + 0.94 \cdot 0.998 = 0.94 \end{aligned}$$

as well as $P(\neg Al \mid Bur) = 0.06$ and insert this into (7.23) which gives the result

$$P(J \mid Bur) = 0.9 \cdot 0.94 + 0.05 \cdot 0.06 = 0.849.$$

Analogously we calculate $P(M \mid Bur) = 0.659$. We now know that John calls for about 85% of all break-ins and Mary for about 66% of all break-ins. The probability that both of them call is calculated, due to conditional independence, as

$$\begin{aligned}P(J,M \mid Bur) &= P(J,M \mid Al)P(Al \mid Bur) + P(J,M \mid \neg Al)P(\neg Al \mid Bur)\\ &= P(J \mid Al)P(M \mid Al)P(Al \mid Bur) + P(J \mid \neg Al)P(M \mid \neg Al)P(\neg Al \mid Bur)\\ &= 0.9 \cdot 0.7 \cdot 0.94 + 0.05 \cdot 0.01 \cdot 0.06 = 0.5922.\end{aligned}$$

More interesting, however, is the probability of a call from John or Mary

$$\begin{aligned}&P(J \vee M \mid Bur) = P(\neg(\neg J, \neg M) \mid Bur) = 1 - P(\neg J, \neg M \mid Bur)\\ &\quad = 1 - [P(\neg J \mid Al)P(\neg M \mid Al)P(Al \mid Bur) + P(\neg J \mid \neg Al)P(\neg M \mid \neg Al)P(\neg Al \mid Bur)]\\ &\quad = 1 - [0.1 \cdot 0.3 \cdot 0.94 + 0.95 \cdot 0.99 \cdot 0.06] = 1 - 0.085 = 0.915.\end{aligned}$$

Bob thus receives a notification for about 92% of all burglaries. Now to calculate $P(Bur \mid J)$, we apply Bayes' theorem, which gives us

$$P(Bur \mid J) = \frac{P(J \mid Bur)P(Bur)}{P(J)} = \frac{0.849 \cdot 0.001}{0.052} = 0.016.$$

Evidently only about 1.6% of all calls from John are actually due to a break-in. Because the probability of false alarms is five times smaller for Mary, with $P(Bur \mid M) = 0.056$, we have significantly higher confidence given a call from Mary. Bob should only be seriously concerned about his home if both of them call, because $P(Bur \mid J, M) = 0.284$ (see Exercise 7.11 on page 173).

In (7.23) on page 162 we showed with

$$P(J \mid Bur) = P(J \mid Al)P(Al \mid Bur) + P(J \mid \neg Al)P(\neg Al \mid Bur)$$

how we can "slide in" a new variable. This relationship holds in general for two variables A and B given the introduction of an additional variable C and is called conditioning:

$$P(A \mid B) = \sum_c P(A \mid B, C = c)P(C = c \mid B).$$

If furthermore A and B are conditionally independent given C, this formula simplifies to

$$P(A \mid B) = \sum_c P(A \mid C = c)P(C = c \mid B).$$

0.32.8 Software for Bayesian Networks

We will give a brief introduction to two tools using the alarm example. We are already familiar with the system PIT. We input the values from the CPTs in PIT syntax into the online input window at www.pit-systems.de. After the input shown in Fig. 7.12 on page 164 we receive the answer:

```
P([Einbruch=t] | [John=t] AND [Mary=t]) = 0.2841.

var Alarm{t,f}, Burglary{t,f}, Earthquake{t,f}, John{t,f}, Mary{t,f};
P([Earthquake=t]) = 0.002;
P([Burglary=t]) = 0.001;
P([Alarm=t] | [Burglary=t] AND [Earthquake=t]) = 0.95;
P([Alarm=t] | [Burglary=t] AND [Earthquake=f]) = 0.94;
P([Alarm=t] | [Burglary=f] AND [Earthquake=t]) = 0.29;
P([Alarm=t] | [Burglary=f] AND [Earthquake=f]) = 0.001;
P([John=t] | [Alarm=t]) = 0.90;
P([John=t] | [Alarm=f]) = 0.05;
P([Mary=t] | [Alarm=t]) = 0.70;
P([Mary=t] | [Alarm=f]) = 0.01;
QP([Burglary=t] | [John=t] AND [Mary=t]);
```

Fig. 7.12 PIT input for the alarm example

While PIT is not a classical Bayesian network tool, it can take arbitrary conditional probabilities and queries as input and calculate correct results. It can be shown [Sch96], that on input of CPTs or equivalent rules, the MaxEnt principle implies the same conditional independences and thus also the same answers as a Bayesian network. Bayesian networks are thus a special case of MaxEnt.

Next we will look at JavaBayes [[**?**]], a classic system also freely available on the Internet with the graphical interface shown in Fig. 7.13. With the graphical network editor, nodes and edges can be manipulated and the values in the CPTs edited. Furthermore, the values of variables can be assigned with "Observe" and the values of other variables called up with "Query". The answers to queries then appear in the console window.

The professional, commercial system Hugin is significantly more powerful and convenient. For example, Hugin can use continuous variables in addition to discrete variables. It can also learn Bayesian networks, that is, generate the network fully automatically from statistical data (see Sect. 8.5).

0.32.9 Development of Bayesian Networks

A compact Bayesian network is very clear and significantly more informative for the reader than a full probability distribution. Furthermore, it requires much less memory. For the variables $v_1, \dots, v_n$ with $|v_1|, \dots, |v_n|$ different values each, the distribution has a total of

$$\prod_{i=1}^{n} |v_i| - 1$$

independent entries. In the alarm example the variables are all binary. Thus for all variables $|v_i| = 2$, and the distribution has $2^5 - 1 = 31$ independent entries. To calculate the number of independent entries for the Bayesian network, the total number of all entries of all CPTs must be determined. For a node v_i with k_i parent nodes $e_{i1}, \ldots, e_{ik_i}$, the associated CPT has

$$(|v_i| - 1) \prod_{j=1}^{k_i} |e_{ij}|$$

entries. Then all CPTs in the network together have

$$\sum_{i=1}^{n} (|v_i| - 1) \prod_{j=1}^{k_i} |e_{ij}| \tag{7.24}$$

entries.[3] For the alarm example the result is then

$$2 + 2 + 4 + 1 + 1 = 10$$

independent entries which uniquely describe the network. The comparison in memory complexity between the full distribution and the Bayesian network becomes clearer when we assume that all n variables have the same number b of values and each node has k parent nodes. Then (7.24) can be simplified and all CPTs together have $n(b-1)b^k$ entries. The full distribution contains $b^n - 1$ entries. A significant gain is only made then if the average number of parent nodes is much smaller than the number of variables. This means that the nodes are only locally connected. Because of the local connection, the network becomes modularized, which—as in software engineering—leads to a reduction in complexity. In the alarm example the alarm node separates the nodes Bur and Ear from the nodes J and M. We can also see this clearly in the Lexmed example.

0.33 Lexmed as a Bayesian Network

The Lexmed system described in Sect. 7.3 can also be modeled as a Bayesian network. By making the outer, thinly-drawn lines directed (giving them arrows), the independence graph in Fig. 7.8 on page 151 can be interpreted as a Bayesian network. The resulting network is shown in Fig. 7.14.

In Sect. 7.3.2 the size of the distribution for Lexmed was calculated as the value 20643839. The Bayesian network on the other hand can be fully described with only 521 values. This value can be determined by entering the variables from Fig. 7.14 into (7.24) on page 165. In the order (Leuko, TRek, Gua, Age, Reb, Sono, Tapp, BowS, Sex, P4Q, P1Q, P2Q, RecP, Urin, P3Q, Diag4) we cal-

[3] For the case of a node without ancestors the product in this sum is empty. For this we substitute the value 1 because the CPT for nodes without ancestors contains, with its a priori probability, exactly one value.

culate

$$\sum_{i=1}^{n}(|v_i|-1)\prod_{j=1}^{k_i}|e_{ij}| = 6\cdot 6\cdot 4+5\cdot 4+2\cdot 4+9\cdot 7\cdot 4+1\cdot 3\cdot 4+1\cdot 4+1\cdot 2\cdot 4$$
$$+3\cdot 3\cdot 4+1\cdot 4+1\cdot 4\cdot 2+1\cdot 4\cdot 2+1\cdot 4+1\cdot 4+1\cdot 4$$
$$+1\cdot 4+1 = 521$$

This example demonstrates that it is practically impossible to build a full distribution for real applications. A Bayesian network with 22 edges and 521 probability values on the other hand is still manageable.

0.34 Causality and Network Structure

Construction of a Bayesian network usually proceeds in two stages.

1. Design of the network structure: This step is usually performed manually and will be described in the following.

2. Entering the probabilities in the CPTs: Manually entering the values in the case of many variables is very tedious. If (as for example with Lexmed) a database is available, this step can be automated through estimating the CPT entries by counting frequencies.
   We will now describe the construction of the network in the alarm example (see Fig. 7.15). At the beginning we know the two causes Burglary and Earthquake and the two symptoms John and Mary. However, because John and Mary do not directly react to a burglar or earthquake, rather only to the alarm, it is appropriate to add this as an additional variable which is not observable by Bob. The process of adding edges starts with the causes, that is, with the variables that have no parent nodes. First we choose Burglary and next Earthquake. Now we must check whether Earthquake is independent of Burglary. This is given, and thus no edge is added from Burglary to Earthquake. Because Alarm is directly dependent on Burglary and Earthquake, these variables are chosen next and an edge is added from both Burglary and Earthquake to Alarm. Then we choose John. Because Alarm and John are not independent, an edge is added from alarm to John. The same is true for Mary. Now we must check whether John is conditionally independent of Burglary given Alarm. If this is not the case, then another edge must be inserted from Burglary to John. We must also check whether edges are needed from Earthquake to John and from Burglary or Earthquake to Mary. Because of conditional independence, these four edges are not necessary. Edges between John and Mary are also unnecessary because John and Mary are conditionally independent given Alarm.

The structure of the Bayesian network heavily depends on the chosen variable ordering. If the order of variables is chosen to reflect the causal relationship beginning with the causes and proceeding to the diagnosis variables, then the result will be a simple network. Otherwise the network may contain significantly more edges. Such non-causal networks are often very difficult to understand and have a higher complexity for reasoning. The reader may refer to Exercise 7.11 on page 173 for better understanding.

0.34.1 Semantics of Bayesian Networks

As we have seen in the previous section, no edge is added to a Bayesian network between two variables A and B when A and B are independent or conditionally independent given a third variable C. This situation is represented in Fig. 7.16.

We now require the Bayesian network to have no cycles and we assume that the variables are numbered such that no variable has a lower number than any variable that precedes it. This is always possible when the network has no cycles.[4] Then, when using all conditional independencies, we have

$$\boldsymbol{P}\left(X_n \mid X_1, \ldots, X_{n-1}\right) = \boldsymbol{P}\left(X_n \mid \text{Parents}\left(X_n\right)\right).$$

This equation thus is a proposition that an arbitrary variable X_i in a Bayesian network is conditionally independent of its ancestors, given its parents. The somewhat more general proposition depicted in Fig. 7.17 on page 169 can be stated compactly as

Theorem 7.6 A node in a Bayesian network is conditionally independent from all non-successor nodes, given its parents.

Now we are able to greatly simplify the chain rule ((7.1) on page 132):

$$\boldsymbol{P}\left(X_1, \ldots, X_n\right) = \prod_{i=1}^{n} \boldsymbol{P}\left(X_i \mid X_1, \ldots, X_{i-1}\right) = \prod_{i=1}^{n} \boldsymbol{P}\left(X_i \mid \text{Parents}\left(X_i\right)\right). \tag{7.25}$$

Using this rule we could, for example, write (7.22) on page 162 directly

$$\boldsymbol{P}(J, Bur, Al) = \boldsymbol{P}(J \mid Al)\, \boldsymbol{P}(Al \mid Bur)\, \boldsymbol{P}(Bur).$$

We now know the most important concepts and foundations of Bayesian networks. Let us summarize them [Jen01]:

Definition 7.7 A Bayesian network is defined by:

- A set of variables and a set of directed edges between these variables.
- Each variable has finitely many possible values.
- The variables together with the edges form a directed acyclic graph (DAG). A DAG is a graph without cycles, that is, without paths of the form $(A, \ldots, A)$.
- For every variable A the CPT (that is, the table of conditional probabilities $\boldsymbol{P}(A \mid \text{Parents}(A))$) is given.

[4] If for example three nodes X_1, X_2, X_3 form a cycle, then there are the edges $(X_1, X_2), (X_2, X_3)$ and (X_3, X_1) where X_1 has X_3 as a successor.

Two variables A and B are called conditionally independent given C if $\boldsymbol{P}(A, B \mid C) = \boldsymbol{P}(A \mid C) \cdot \boldsymbol{P}(B \mid C)$ or, equivalently, if $\boldsymbol{P}(A \mid B, C) = \boldsymbol{P}(A \mid C)$.
Besides the foundational rules of computation for probabilities, the following rules are also true:
Bayes' Theorem: $P(A \mid B) = \dfrac{P(B \mid A) \cdot P(A)}{P(B)}$
Marginalization: $P(B) = P(A, B) + P(\neg A, B) = P(B \mid A) \cdot P(A) + P(B \mid \neg A) \cdot P(\neg A)$
Conditioning: $P(A \mid B) = \sum_c P(A \mid B, C = c)\, P(C = c \mid B)$
A variable in a Bayesian network is conditionally independent of all non-successor variables given its parent variables. If $X_1, \ldots, X_{n-1}$ are no successors of X_n, we have $\boldsymbol{P}\left(X_n \mid X_1, \ldots, X_{n-1}\right) = \boldsymbol{P}\left(X_n \mid \text{Parents}\left(X_n\right)\right)$. This condition must be honored during the construction of a network.
During construction of a Bayesian network the variables should be ordered according to causality. First the causes, then the hidden variables, and the diagnosis variables last.
Chain rule: $\boldsymbol{P}\left(X_1, \ldots, X_n\right) = \prod_{i=1}^{n} \boldsymbol{P}\left(X_i \mid \text{Parents}\left(X_i\right)\right)$

In [Pea88] and [Jen01] the term d-separation is introduced for Bayesian networks, from which a theorem similar to Theorem 7.6 on page 168 follows. We will refrain from introducing this term and thereby reach a somewhat simpler, though not a theoretically as clean representation.

0.34.2 Summary

In a way that reflects the prolonged, sustained trend toward probabilistic systems, we have introduced probabilistic logic for reasoning with uncertain knowledge. After introducing the language—propositional logic augmented with probabilities or probability intervals—we chose the natural, if unusual approach via the method of maximum entropy as an entry point and showed how we can model non-monotonic reasoning with this method. Bayesian networks were then introduced as a special case of the MaxEnt method.

Why are Bayesian networks a special case of MaxEnt? When building a Bayesian network, assumptions about independence are made which are unnecessary for the MaxEnt method. Furthermore, when building a Bayesian network, all CPTs must be completely filled in so that a complete probability distribution can be constructed. Otherwise reasoning is restricted or impossible. With MaxEnt, on the other hand, the developer can formulate all the knowledge he has at his disposal in the form of probabilities. MaxEnt then completes the model and generates the distribution. Even if (for example when interviewing an expert) only vague propositions are available, this can be suitably modeled. A proposition such as "I am pretty sure that A is true." can for example be modeled using $P(A) \in [0.6, 1]$ as a probability interval. When building a Bayesian network, a concrete value must be given for the probability, if necessary by guessing. This means, however, that the expert or the developer put ad hoc information into the system. One further advantage of MaxEnt is the possibility of formulating (almost) arbitrary propositions. For Bayesian networks the CPTs must be filled.

The freedom that the developer has when modeling with MaxEnt can be a disadvantage (especially for a beginner) because, in contrast to the Bayesian approach, it is not necessarily clear what knowledge should be modeled. When modeling with Bayesian networks the approach is quite clear: according to causal dependencies, from the causes to the effects, one edge after the other is entered into the network by testing conditional independence.[5] At the end all CPTs are filled with values.

However, the following interesting combinations of the two methods are possible: we begin by building a network according to the Bayesian methodology, enter all the edges accordingly and then fill the CPTs with values. Should certain values for the CPTs be unavailable, then they can be replaced with intervals or by other probabilistic logic formulas. Naturally such a network—or better: a rule set—no longer has the special semantics of a Bayesian network. It must then be processed and completed by a MaxEnt system. The ability to use MaxEnt with arbitrary rule sets has a downside, though. Similarly to the situation in logic, such rule sets can be inconsistent. For example, the two rules $P(A) = 0.7$ and $P(A) = 0.8$ are inconsistent. While the MaxEnt system PIT for example can recognize the inconsistency, if cannot give a hint about how to remove the problem.

We introduced the medical expert system Lexmed, a classic application for reasoning with uncertain knowledge, and showed how it can be modeled and implemented using MaxEnt and Bayesian networks, and how these tools can replace the well-established, but too weak linear scoring systems used in medicine. 21

In the Lexmed example we showed that it is possible to build an expert system for reasoning under uncertainty that is capable of discovering (learning) knowledge from the data in a database. We will give more insight into the methods of learning of Bayesian networks in Chap. 8, after the necessary foundations for machine learning have been laid.

Today Bayesian reasoning is a large, independent field, which we can only briefly describe here. We have completely left out the handling of continuous variables. For the case of normally distributed random variables there are procedures and systems. For arbitrary distributions, however, the computational complexity is a big problem. In addition to the directed networks that are heavily based on causality, there are also undirected networks. Connected with this is a discussion about the meaning and usefulness of causality in Bayesian networks. The interested reader is directed to excellent textbooks such as [Pea88, Jen01, Whi96, [**?**]], as well as the proceedings of the annual conference of the Association for Uncertainty in Artificial Intelligence (AUAI) (www.auai.org).

[5]This is also not always quite so simple.

0.34.3 Exercises

Exercise 7.1 Prove the proposition from Theorem 7.1 on page 129.
Exercise 7.2 The gardening hobbyist Max wants to statistically analyze his yearly harvest of peas. For every pea pod he picks he measures its length x_i in centimeters and its weight y_i in grams. He divides the peas into two classes, the good and the bad (empty pods). The measured data (x_i, y_i) are

good peas:

x	1	2	2	3	3	4	4	5	6
y	2	3	4	4	5	5	6	6	6

bad peas:

x	4	6	6	7
y	2	2	3	3

(a) From the data, compute the probabilities $P(y > 3 \mid \text{Class} = \text{good})$ and $P(y \leq 3 \mid \text{Class} = \text{good})$. Then use Bayes' formula to determine $P(\text{Class} = \text{good} \mid y > 3)$ and $P(\text{Class} = \text{good} \mid y \leq 3)$.
(b) Which of the probabilities computed in subproblem (a) contradicts the statement "All good peas are heavier than 3 grams"?

Exercise 7.3 You are supposed to predict the afternoon weather using a few simple weather values from the morning of this day. The classical probability calculation for this requires a complete model, which is given in the following table.

Sky	*Bar*	*Prec*	*P(Sky, Bar, Prec)*
Clear	Rising	Dry	0.40
Clear	Rising	Raining	0.07
Clear	Falling	Dry	0.08
Clear	Falling	Raining	0.10
Cloudy	Rising	Dry	0.09
Cloudy	Rising	Raining	0.11
Cloudy	Falling	Dry	0.03

Sky:	The sky is clear or cloudy in the morning
Bar:	Barometer rising or falling in the morning
Prec:	Raining or dry in the afternoon

(a) How many events are in the distribution for these three variables?
(b) Compute $P(\text{Prec} = \text{dry} \mid \text{Sky} = \text{clear}, \text{Bar} = \text{rising})$.
(c) Compute $P(\text{Prec} = \text{rain} \mid \text{Sky} = \text{cloudy})$.
(d) What would you do if the last row were missing from the table?

[521] In Sect. 8.7 and in Exercise 8.17 on page 242 we will show that the scores are equivalent to the special case naive Bayes, that is, to the assumption that all symptoms are conditionally independent given the diagnosis.

- Exercise 7.4 In a television quiz show, the contestant must choose between three closed doors. Behind one door the prize awaits: a car. Behind both of the other doors are goats. The contestant chooses a door, e.g. number one. The host, who knows where the car is, opens another door, e.g. number three, and a goat appears. The contestant is now given the opportunity to choose between the two remaining doors (one and two). What is the better choice from his point of view? To stay with the door he originally chose or to switch to the other closed door?

Exercise 7.5 Using the Lagrange multiplier method, show that, without explicit constraints, the uniform distribution $p_1 = p_2 = \cdots = p_n = 1/n$ represents maximum entropy. Do not forget the implicitly ever-present constraint $p_1 + p_2 + \cdots + p_n = 1$. How can we show this same result using indifference?

Exercise 7.6 Use the PIT system (http://www.pit-systems.de) or SPIRIT (http://www.xspirit.de) to calculate the MaxEnt solution for $P(B)$ under the constraint $P(A) = \alpha$ and $P(B \mid A) = \beta$. Which disadvantage of PIT, compared with calculation by hand, do you notice here?

Exercise 7.7 Given the constraints $P(A) = \alpha$ and $P(A \vee B) = \beta$, manually calculate $P(B)$ using the MaxEnt method. Use PIT to check your solution.

- Exercise 7.8 Given the constraints from (7.10), (7.11), (7.12): $p_1 + p_2 = \alpha$, $p_1 + p_3 = \gamma$, $p_1 + p_2 + p_3 + p_4 = 1$. Show that $p_1 = \alpha\gamma$, $p_2 = \alpha(1 - \gamma)$, $p_3 = \gamma(1-\alpha)$, $p_4 = (1-\alpha)(1-\gamma)$ represents the entropy maximum under these constraints.

- Exercise 7.9 A probabilistic algorithm calculates the likelihood p that an inbound email is spam. To classify the emails in classes delete and read, a cost matrix is then applied to the result.

(a) Give a cost matrix (2×2 matrix) for the spam filter. Assume here that it costs the user 10 cents to delete an email, while the loss of an email costs 10 dollars (compare this to Example 1.1 on page 17 and Exercise 1.7 on page 21).
(b) Show that, for the case of a 2×2 matrix, the application of the cost matrix is equivalent to the application of a threshold on the spam probability and determine the threshold.

Exercise 7.10 Given a Bayesian network with the three binary variables A, B, C and $P(A) = 0.2$, $P(B) = 0.9$, as well as the CPT shown below:
(a) Compute $P(A \mid B)$.
(b) Compute $P(C \mid A)$.

A	B	$P(C)$
t	f	0.1
t	t	0.2
f	t	0.9
f	f	0.4

Exercise 7.11 For the alarm example (Example 7.10 on page 159), calculate the following conditional probabilities:
(a) Calculate the a priori probabilities $P(Al)$, $P(J)$, $P(M)$.
(b) Calculate $P(M \mid Bur)$ using the product rule, marginalization, the chain rule, and conditional independence.
(c) Use Bayes' formula to calculate $P(Bur \mid M)$.
(d) Compute $P(Al \mid J, M)$ and $P(Bur \mid J, M)$.
(e) Show that the variables J and M are not independent.
(f) Check all of your results with JavaBayes and with PIT (see [[**?**]] for demo programs).
(g) Design a Bayesian network for the alarm example, but with the altered variable ordering M, Al, Ear, Bur, J. According to the semantics of Bayesian networks, only the necessary edges must be drawn in. (Hint: the variable order given here does NOT represent causality. Thus it will be difficult to intuitively determine conditional independences.)
(h) In the original Bayesian network of the alarm example, the earthquake nodes is removed. Which CPTs does this change? (Why these in particular?)
(i) Calculate the CPT of the alarm node in the new network.

Exercise 7.12 A diagnostic system is to be made for a dynamo-powered bicycle light using a Bayesian network. The variables in the following table are given.

Abbr.	Meaning	Values
Li	Light is on	t/f
Str	Street condition	dry, wet, snow_covered
Flw	Dynamo flywheel worn out	t/f
R	Dynamo sliding	t/f
V	Dynamo shows voltage	t/f
B	Light bulb o.k.	t/f
K	Cable o.k.	t/f

The following variables are pairwise independent: Str, Flw, B, K. Furthermore: (R, B), (R, K), (V, B), (V, K) are independent and the following equation holds:

$$P(Li \mid V, R) = P(Li \mid V)$$
$$P(V \mid R, \text{Str}) = P(V \mid R)$$
$$P(V \mid R, \text{Flw}) = P(V \mid R)$$

(a) Draw all of the edges into the graph (taking causality into account).
(b) Enter all missing CPTs into the graph (table of conditional probabilities). Freely insert plausible values for the probabilities.
(c) Show that the network does not contain an edge (Str, Li).
(d) Compute $P(V \mid \text{Str} = \text{snow\_covered})$.

V	B	K	$P(Li)$
t	t	t	0.99
t	t	f	0.01
t	f	t	0.01
t	f	f	0.001
f	t	t	0.3
f	t	f	0.005
f	f	t	0.005
f	f	f	0

0.35 Machine Learning and Data Mining

If we define AI as is done in Elaine Rich's book [Ric83]:
Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.
and if we consider that the computer's learning ability is especially inferior to that of humans, then it follows that research into learning mechanisms, and the development of machine learning algorithms is one of the most important branches of AI.

There is also demand for machine learning from the viewpoint of the software developer who programs, for example, the behavior of an autonomous robot. The structure of the intelligent behavior can become so complex that it is very difficult or even impossible to program close to optimally, even with modern high-level languages such as PROLOG and Python.[6] Machine learning algorithms are even used today to program robots in a way similar to how humans learn (see Chap. 10 or [[**?**], [**?**]]), often in a hybrid mixture of programmed and learned behavior.

The task of this chapter is to describe the most important machine learning algorithms and their applications. The topic will be introduced in this section, followed by important fundamental learning algorithms in the next sections. Theory and terminology will be built up in parallel to this. The chapter will close with a summary and overview of the various algorithms and their applications. We will restrict ourselves in this chapter to supervised and unsupervised learning algorithms. As an important class of learning algorithms, neural networks will be dealt with in Chap. 9. Due to its special place and important role for autonomous robots, reinforcement learning will also have its own dedicated chapter (Chap. 10).

What Is Learning? Learning vocabulary of a foreign language, or technical terms, or even memorizing a poem can be difficult for many people. For computers, however, these tasks are quite simple because they are little more than saving text in a file. Thus memorization is uninteresting for AI. In contrast, the acquisition of mathematical skills is usually not done by memorization. For

[6] Python is a modern scripting language with very readable syntax, powerful data types, and extensive standard libraries, which can be used to this end.

addition of natural numbers this is not at all possible, because for each of the terms in the sum $x + y$ there are infinitely many values. For each combination of the two values x and y, the triple $(x, y, x+y)$ would have to be stored, which is impossible. For decimal numbers, this is downright impossible. This poses the question: how do we learn mathematics? The answer reads: The teacher explains the process and the students practice it on examples until they no longer make mistakes on new examples. After 50 examples the student (hopefully) understands addition. That is, after only 50 examples he can apply what was learned to infinitely many new examples, which to that point were not seen. This process is known as generalization. We begin with a simple example.

Example 8.1 A fruit farmer wants to automatically divide harvested apples into the merchandise classes A and B. The sorting device is equipped with sensors to measure two features, size and color, and then decide which of the two classes the apple belongs to. This is a typical classification task. Systems which are capable of dividing feature vectors into a finite number of classes are called classifiers.

To configure the machine, apples are hand-picked by a specialist, that is, they are classified. Then the two measurements are entered together with their class

Table 8.1 Training data for the apple sorting agent

Size [cm]	8	8	6	3	...
Color	0.1	0.3	0.9	0.8	...
Merchandise class	B	A	A	B	...

label in a table (Table 8.1. The size is given in the form of diameter in centimeters and the color by a numeric value between 0 (for green) and 1 (for red). A visualization of the data is listed as points in a scatterplot diagram in the right of Fig. 8.2.

The task in machine learning consists of generating a function from the collected, classified data which calculates the class value (A or B) for a new apple from the two features size and color. In Fig. 8.3 such a function is shown by the dividing line drawn through the diagram. All apples with a feature vector to the bottom left of the line are put into class B, and all others into class A.

In this example it is still very simple to find such a dividing line for the two classes. It is clearly a more difficult, and above all much less visualizable task, when the objects to be classified are described by not just two, but many features. In practice 30 or more features are usually used. For n features, the task consists of finding an $n-1$ dimensional hyperplane within the n-dimensional feature space

which divides the classes as well as possible. A "good" division means that the percentage of falsely classified objects is as small as possible.

A classifier maps a feature vector to a class value. Here it has a fixed, usually small, number of alternatives. The desired mapping is also called target function. If the target function does not map onto a finite domain, then it is not a classification, but rather an approximation problem. Determining the market value of a stock from given features is such an approximation problem. In the following sections we will introduce several learning agents for both types of mappings.

The Learning Agent We can formally describe a learning agent as a function which maps a feature vector to a discrete class value or in general to a real number. This function is not programmed, rather it comes into existence or changes itself during the learning phase, influenced by the training data. In Fig. 8.4 such an agent is presented in the apple sorting example. During learning, the agent is fed with the already classified data from Table 8.1 on page 177. Thereafter the agent constitutes as good a mapping as possible from the feature vector to the function value (e.g. merchandise class).

We can now attempt to approach a definition of the term "machine learning". Tom Mitchell [[**?**]] gives this definition:

Machine Learning is the study of computer algorithms that improve automatically through experience.

Drawing on this, we give

Definition 8.1 An agent is a learning agent if it improves its performance (measured by a suitable criterion) on new, unknown data over time (after it has seen many training examples).

It is important to test the generalization capability of the learning algorithm on unknown data, the test data. Otherwise every system that just saved the training data would appear to perform optimally just by calling up the saved data. A learning agent is characterized by the following terms:

Task: the task of the learning algorithm is to learn a mapping. This could for example be the mapping from an apple's size and color to its merchandise class,

but also the mapping from a patient's 15 symptoms to the decision of whether or not to remove his appendix.

Variable agent: (more precisely a class of agents): here we have to decide which learning algorithm will be worked with. If this has been chosen, thus the class of all learnable functions is determined.

Training data: (experience): the training data (sample) contain the knowledge which the learning algorithm is supposed to extract and learn. With the choice of training data one must ensure that it is a representative sample for the task to be learned.

Test data: important for evaluating whether the trained agent can generalize well from the training data to new data.

Performance measure: for the apple sorting device, the number of correctly classified apples. We need it to test the quality of an agent. Knowing the performance measure is usually much easier than knowing the agent's function. For example, it is easy to measure the performance (time) of a 10,000 meter runner. However, this does not at all imply that the referee who measures the time can run as fast. The referee only knows how to measure the performance, but not the "function" of the agent whose performance he is measuring.

What Is Data Mining? The task of a learning machine to extract knowledge

from training data. Often the developer or the user wants the learning machine to make the extracted knowledge readable for humans as well. It is still better if the
developer can even alter the knowledge. The process of induction of decision trees in Sect. 8.4 is an example of this type of method. Similar challenges come from electronic business and knowledge management. A classic problem presents itself here: from the actions of visitors to his web portal, the owner of an Internet business would like to create a relationship between the characteristics of a customer and the class of products which are interesting to that customer. Then a seller will be able to place customer-specific advertisements. This is demonstrated in a telling way at www.amazon.com, where the customer is recommended products which are similar to those seen in the previous visit. In many areas of advertisement and marketing, as well as in customer relationship management (CRM), data mining techniques are coming into use. Whenever large amounts of data are available, one can attempt to use these data for the analysis of customer preferences in order to show customer-specific advertisements. The emerging field of preference learning is dedicated to this purpose.

The process of acquiring knowledge from data, as well as its representation and application, is called data mining. The methods used are usually taken from statistics or machine learning and should be applicable to very large amounts of data at reasonable cost.

In the context of acquiring information, for example on the Internet or in an intranet, text mining plays an increasingly important role. Typical tasks include finding similar text in a search engine or the classification of texts, which for example is applied in spam filters for email. In Sect. 8.7.1 we will introduce the widespread naive Bayes algorithm for the classification of text. A relatively new challenge for data mining is the extraction of structural, static, and dynamic information from graph structures such as social networks, traffic networks, or Internet traffic.

Because the two described tasks of machine learning and data mining are formally very similar, the basic methods used in both areas are for the most part identical. Therefore in the description of the learning algorithms, no distinction will be made between machine learning and data mining.

Because of the huge commercial impact of data mining techniques, there are now many sophisticated optimizations and a whole line of powerful data mining systems, which offer a large palette of convenient tools for the extraction of knowledge from data. Such a system is introduced in Sect. 8.10.

0.35.1 Data Analysis

Statistics provides a number of ways to describe data with simple parameters. From these we choose a few which are especially important for the analysis of training data and test these on a subset of the Lexmed data from Sect. 7.3. In this example dataset, the symptoms $x_1, \ldots, x_{15}$ of $N = 473$ patients, concisely described in

Table 8.2 Description of variables $x_1, \ldots, x_{16}$. A slightly different formalization was used in Table 7.2 on page 147

Var. num.	Description	Values
1	Age	Continuous
2	Sex ($1 =$ male, $2 =$ female)	1, 2
3	Pain quadrant 1	0,1
4	Pain quadrant 2	0,1
5	Pain quadrant 3	0,1
6	Pain quadrant 4	0,1
7	Local muscular guarding	0,1
8	Generalized muscular guarding	0,1
9	Rebound tenderness	0,1
10	Pain on tapping	0,1
11	Pain during rectal examination	0,1
12	Axial temperature	Continuous
13	Rectal temperature	Continuous
14	Leukocytes	Continuous
15	Diabetes mellitus	0,1
16	Appendicitis	0,1

Table 8.2 on page 181, as well as the class label—that is, the diagnosis (appendicitis positive/negative)—are listed. Patient number one, for example, is described by the vector

$$x^1 = (26, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 37.9, 38.8, 23100, 0, 1)$$

and patient number two by

$$\boldsymbol{x}^2 = (17, 2, 0, 0, 1, 0, 1, 0, 1, 1, 0, 36.9, 37.4, 8100, 0, 0)$$

Patient number two has the leukocyte value $x^2_{14} = 8100$. For each variable x_i, its average $\bar{x}_i$ is defined as

$$\bar{x}_i := \frac{1}{N} \sum_{p=1}^{N} x_i^p$$

and the standard deviation s_i as a measure of its average deviation from the average value as

$$s_i := \sqrt{\frac{1}{N-1} \sum_{p=1}^{N} (x_i^p - \bar{x}_i)^2}.$$

The question of whether two variables x_i and x_j are statistically dependent (correlated) is important for the analysis of multidimensional data. For example, the covariance

Table 5: Correlation matrix for the 16 appendicitis variables measured in 473 cases

1.	-0.009	0.14	0.037	-0.096	0.12	0.018	0.051	-0.034	-0.041	0.034	0.037
-0.009	1.	-0.0074	-0.019	-0.06	0.063	-0.17	0.0084	-0.17	-0.14	-0.13	-0.01
0.14	-0.0074	1.	0.55	-0.091	0.24	0.13	0.24	0.045	0.18	0.028	0.02
0.037	-0.019	0.55	1.	-0.24	0.33	0.051	0.25	0.074	0.19	0.087	0.11
-0.096	-0.06	-0.091	-0.24	1.	0.059	0.14	0.034	0.14	0.049	0.057	0.064
0.12	0.063	0.24	0.33	0.059	1.	0.071	0.19	0.086	0.15	0.048	0.11
0.018	-0.17	0.13	0.051	0.14	0.071	1.	0.16	0.4	0.28	0.2	0.24
0.051	0.0084	0.24	0.25	0.034	0.19	0.16	1.	0.17	0.23	0.24	0.19
-0.034	-0.17	0.045	0.074	0.14	0.086	0.4	0.17	1.	0.53	0.25	0.19
-0.041	-0.14	0.18	0.19	0.049	0.15	0.28	0.23	0.53	1.	0.24	0.15
0.034	-0.13	0.028	0.087	0.057	0.048	0.2	0.24	0.25	0.24	1.	0.17
0.037	-0.017	0.02	0.11	0.064	0.11	0.24	0.19	0.19	0.15	0.17	1.
0.05	-0.034	0.045	0.12	0.058	0.12	0.36	0.24	0.27	0.19	0.17	0.72
-0.037	-0.14	0.03	0.11	0.11	0.063	0.29	0.27	0.27	0.23	0.22	0.26
0.37	0.045	0.11	0.14	0.017	0.21	-0.0001	0.083	0.026	0.02	0.098	0.035
0.012	-0.2	0.045	-0.0091	0.14	0.053	0.33	0.084	0.38	0.32	0.17	0.15

$$\sigma_{ij} = \frac{1}{N-1} \sum_{p=1}^{N} \left(x_i^p - \bar{x}_i\right)\left(x_j^p - \bar{x}_j\right)$$

gives information about this. In this sum, the summand returns a positive entry for the pth data vector exactly when the deviations of the ith and jth components from the average both have the same sign. If they have different signs, then the entry is negative. Therefore the covariance $\sigma_{12,13}$ of the two different fever values should be quite clearly positive.

However, the covariance also depends on the absolute value of the variables, which makes comparison of the values difficult. To be able to compare the degree of dependence in the case of multiple variables, we therefore define the correlation coefficient

$$K_{ij} = \frac{\sigma_{ij}}{s_i \cdot s_j}$$

for two values x_i and x_j, which is nothing but a normalized covariance. The matrix $\boldsymbol{K}$ of all correlation coefficients contains values between -1 and 1, is symmetric, and all of its diagonal elements have the value 1. The correlation matrix for all 16 variables is given in Table 8.3.

This matrix becomes somewhat more readable when we represent it as a density plot. Instead of the numerical values, the matrix elements in Fig. 8.6 on page 183 are filled with gray values. In the right diagram, the absolute values are shown. Thus we can very quickly see which variables display a weak or strong dependence. We can see, for example, that the variables 7, 9, 10 and 14 show the strongest correlation with the class variable appendicitis and therefore are more important for the diagnosis than the other variable. We also see, however, that the variables 9 and 10 are strongly correlated. This could mean that one of these two values is potentially sufficient for the diagnosis.

0.35.2 The Perceptron, a Linear Classifier

In the apple sorting classification example, a curved dividing line is drawn between the two classes in Fig. 8.3 on page 177. A simpler case is shown in Fig. 8.7. Here the two-dimensional training examples can be separated by a straight line. We call such a set of training data linearly separable. In n dimensions a hyperplane is needed for the separation. This represents a linear subspace of dimension $n-1$. Because every $(n-1)$-dimensional hyperplane in $\mathbb{R}^n$ can be described by an equation

$$\sum_{i=1}^{n} a_i x_i = \theta$$

it makes sense to define linear separability as follows.

Definition 8.2 Two sets $M_1 \subset \mathbb{R}^n$ and $M_2 \subset \mathbb{R}^n$ are called linearly separable if real numbers $a_1, \dots, a_n, \theta$ exist with

$$\sum_{i=1}^{n} a_i x_i > \theta \quad \text{for all } \boldsymbol{x} \in M_1 \quad \text{and} \quad \sum_{i=1}^{n} a_i x_i \leq \theta \quad \text{for all } \boldsymbol{x} \in M_2$$

The value θ is denoted the threshold.

In Fig. 8.8 we see that the AND function is linearly separable, but the XOR function is not. For AND, for example, the line $-x_1 + 3/2$ separates true and false interpretations of the formula $x_1 \wedge x_2$. In contrast, the XOR function does not have a straight line of separation. Clearly the XOR function has a more complex structure than the AND function in this regard.

With the perceptron, we present a very simple learning algorithm which can separate linearly separable sets.

Definition 8.3 Let $\boldsymbol{w} = (w_1, \dots, w_n) \in \mathbb{R}^n$ be a weight vector and $\boldsymbol{x} \in \mathbb{R}^n$ an input vector. A perceptron represents a function $P : \mathbb{R}^n \to \{0, 1\}$ which corresponds to the following rule:

$$P(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \boldsymbol{w}x = \sum_{i=1}^{n} w_i x_i > 0, \\ 0 & \text{else.} \end{cases}$$

The perceptron [[**?**], MP69] is a very simple classification algorithm. It is equivalent to a two-layer neural network with activation by a threshold function, shown in Fig. 8.9 on page 185. As shown in Chap. 9, each node in the network represents a neuron, and every edge a synapse. For now, however, we will only view

the perceptron as a learning agent, that is, as a mathematical function which maps a feature vector to a function value. Here the input variables x_i are denoted features.

As we can see in the formula $\sum_{i=1}^{n} w_i x_i > 0$, all points $\boldsymbol{x}$ above the hyperplane $\sum_{i=1}^{n} w_i x_i = 0$ are classified as positive ($P(\boldsymbol{x}) = 1$), and all others as negative $P(\boldsymbol{x}) = 0$. The separating hyperplane goes through the origin because $\theta = 0$. We will use a little trick to show that the absence of an arbitrary threshold represents no restriction of power. First, however, we want to introduce a simple learning algorithm for the perceptron.

0.35.3 The Learning Rule

With the notation M_+ and M_- for the sets of positive and negative training patterns respectively, the perceptron learning rule reads [MP69]

PerceptronLearning[M_+, M_-]
$\boldsymbol{w}$ = arbitrary vector of real numbers
Repeat
For all $\boldsymbol{x} \in M_+$
If $\boldsymbol{wx} \leq 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}$
For all $\boldsymbol{x} \in M_-$
If $\boldsymbol{wx} > 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}$
Until all $\boldsymbol{x} \in M_+ \cup M_-$ are correctly classified

The perceptron should output the value 1 for all $\boldsymbol{x} \in M_+$. By Definition 8.3 on page 184 this is true when $\boldsymbol{wx} > 0$. If this is not the case then $\boldsymbol{x}$ is added to the weight vector $\boldsymbol{w}$, whereby the weight vector is changed in exactly the right direction. We see this when we apply the perceptron to the changed vector $\boldsymbol{w}+\boldsymbol{x}$ because

$$(\boldsymbol{w} + \boldsymbol{x}) \cdot \boldsymbol{x} = \boldsymbol{wx} + \boldsymbol{x}^2.$$

If this step is repeated often enough, then at some point the value $\boldsymbol{wx}$ will become positive, as it should be. Analogously, we see that, for negative training data, the perceptron calculates an ever smaller value

$$(\boldsymbol{w} - \boldsymbol{x}) \cdot \boldsymbol{x} = \boldsymbol{wx} - \boldsymbol{x}^2$$

which at some point becomes negative. [2]

Example 8.2 A perceptron is to be trained on the sets $M_+ = \{(0, 1.8), (2, 0.6)\}$ and $M_- = \{(-1.2, 1.4), (0.4, -1)\}$.$\boldsymbol{w} = (1, 1)$ was used as an initial weight vector. The training data and the line defined by the weight vector $\boldsymbol{wx} = x_1 + x_2 = 0$ are shown in Fig. 8.10 on page 187 in the first picture in the top row. In addition, the weight vector is drawn as a dashed line. Because $\boldsymbol{wx} = 0$, this is orthogonal to the line.

In the first iteration through the loop of the learning algorithm, the only falsely classified training example is $(-1.2, 1.4)$ because

$$(-1.2, 1.4) \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = 0.2 > 0$$

This results in $\boldsymbol{w} = (1, 1) - (-1.2, 1.4) = (2.2, -0.4)$, as drawn in the second image in the top row in Fig. 8.10 on page 187. The other images show how, after a total of five changes, the dividing line lies between the two classes. The perceptron thus classifies all data correctly. We clearly see in the example that every incorrectly classified data point from M_+ "pulls" the weight vector $\boldsymbol{w}$ in its direction and every incorrectly classified point from M_- "pushes" the weight vector in the opposite direction.

It has been shown [MP69] that the perceptron always converges for linearly separable data. We have

Theorem 8.1 Let classes M_+ and M_- be linearly separable by a hyperplane $\boldsymbol{wx} = 0$. Then PerceptronLearning converges for every initialization of the vector $\boldsymbol{w}$. The perceptron P with the weight vector so calculated divides the classes M_+ and M_-, that is:

$$P(\boldsymbol{x}) = 1 \quad \Leftrightarrow \quad \boldsymbol{x} \in \boldsymbol{M}_+$$

and

$$P(\boldsymbol{x}) = 0 \quad \Leftrightarrow \quad \boldsymbol{x} \in M_-.$$

As we can clearly see in Example 8.2, perceptrons as defined above cannot divide arbitrary linearly separable sets, rather only those which are divisible by a line through the origin, or in $\mathbb{R}^n$ by a hyperplane through the origin, because the constant term θ is missing from the equation $\sum_{i=1}^{n} w_i x_i = 0$.

With the following trick we can generate the constant term. We hold the last component x_n of the input vector $\boldsymbol{x}$ constant and set it to the value 1. Now the weight $w_n := -\theta$ works like a threshold because

$$\sum_{i=1}^{n} w_i x_i = \sum_{i=1}^{n-1} w_i x_i - \theta > 0 \Leftrightarrow \sum_{i=1}^{n-1} w_i x_i > \theta$$

Such a constant value $x_n = 1$ in the input is called a bias unit. Because the associated weight causes a constant shift of the hyperplane, the term "bias" fits well.

In the application of the perceptron learning algorithm, a bit with the constant value 1 is appended to the training data vector. We observe that the weight w_n, or the threshold θ, is learned during the learning process.

Now it has been shown that a perceptron $P_\theta : \mathbb{R}^{n-1} \rightarrow \{0, 1\}$

$$P_\theta\left(x_1, \ldots, x_{n-1}\right) = \begin{cases} 1 & \text{if } \sum_{i=1}^{n-1} w_i x_i > \theta \\ 0 & \text{else} \end{cases} \tag{8.1}$$

with an arbitrary threshold can be simulated by a perceptron $P : \mathbb{R}^n \rightarrow \{0, 1\}$ with the threshold 0. If we compare (8.1) with the definition of linearly separable, then we see that both statements are equivalent. In summary, we have shown that:

Theorem 8.2 *A* function $f : \mathbb{R}^n \rightarrow \{0, 1\}$ can by represented by a perceptron if and only if the two sets of positive and negative input vectors are linearly separable. Example 8.3 We now train a perceptron with a threshold on six simple, graphical binary patterns, represented in Fig. 8.11, with 5×5 pixels each.

[62] Caution! This is not a proof of convergence for the perceptron learning rule. It only shows that the perceptron converges when the training dataset consists of a single example.

The training data can be learned by PerceptronLearning in four iterations over all patterns. Patterns with a variable number of inverted bits introduced as noise are used to test the system's generalization capability. The inverted bits in the test pattern are in each case in sequence one after the other. In Fig. 8.12 the percentage of correctly classified patterns is plotted as a function of the number of false bits.

After about five consecutive inverted bits, the correctness falls off sharply, which is not surprising given the simplicity of the model. In the next section we will present an algorithm that performs much better in this case.

0.35.4 Optimization and Outlook

As one of the simplest neural-network-based learning algorithms, the two-layer perceptron can only divide linearly separable classes. In Sect. 9.5 we will see that multi-layered networks are significantly more powerful. Despite its simple structure, the perceptron in the form presented converges very slowly. It can be accelerated by normalization of the weight-altering vector. The formulas $\boldsymbol{w} = \boldsymbol{w} \pm \boldsymbol{x}$ are replaced by $\boldsymbol{w} = \boldsymbol{w} \pm \boldsymbol{x}/|\boldsymbol{x}|$. Thereby every data point has the same weight during learning, independent of its value.

The speed of convergence heavily depends on the initialization of the vector $\boldsymbol{w}$. Ideally it would not need to be changed at all and the algorithm would converge after one iteration. We can get closer to this goal by using the heuristic initialization

$$w_{\mathbf{0}} = \sum_{x \in M_+} x - \sum_{x \in M_-} x,$$

which we will investigate more closely in Exercise 8.5 on page 239.

If we compare the perceptron formula with the scoring method presented in Sect. 7.3.1, we immediately see their equivalence. Furthermore, the perceptron, as the simplest neural network model, is equivalent to naive Bayes, the simplest type of Bayesian network (see Exercise 8.17 on page 242). Thus evidently several very different classification algorithms have a common origin.

In Chap. 9 we will become familiar with a generalization of the perceptron in the form of the back-propagation algorithm, which can divide non linearly separable sets through the use of multiple layers, and which possesses a better learning rule.

0.35.5 The Nearest Neighbor Method

For a perceptron, knowledge available in the training data is extracted and saved in a compressed form in the weights w_i. Thereby information about the data is lost. This is exactly what is desired, however, if the system is supposed to generalize from the training data to new data. Generalization in this case is a time-intensive process with the goal of finding a compact representation of data in the form of a function which classifies new data as good as possible.

Memorization of all data by simply saving them is quite different. Here the learning is extremely simple. However, as previously mentioned, the saved knowledge is not so easily applicable to new, unknown examples. Such an approach is very unfit for learning how to ski, for example. A beginner can never become a good skier just by watching videos of good skiers. Evidently, when learning movements of this type are automatically carried out, something similar happens as in the case of the perceptron. After sufficiently long practice, the knowledge stored in training examples is transformed into an internal representation in the brain.

However, there are successful examples of memorization in which generalization is also possible. During the diagnosis of a difficult case, a doctor could try to remember similar cases from the past. If his memory is sound, then he might hit upon this case, look it up in his files and finally come a similar diagnosis. For this approach the doctor must have a good feeling for similarity, in order to remember the most similar case. If he has found this, then he must ask himself whether it is similar enough to justify the same diagnosis.

Fig. 8.13 In this example with negative and positive training examples, the nearest neighbor method groups the new point marked in black into the negative class

What does similarity mean in the formal context we are constructing? We represent the training samples as usual in a multidimensional feature space and define: The smaller their distance in the feature space, the more two examples are similar.

We now apply this definition to the simple two-dimensional example from Fig. 8.13. Here the next neighbor to the black point is a negative example. Thus it is assigned to the negative class.

The distance $d(\boldsymbol{x}, \boldsymbol{y})$ between two points $\boldsymbol{x} \in \mathbb{R}^n$ and $\boldsymbol{y} \in \mathbb{R}^n$ can for example be measured by the Euclidean distance

$$d(\boldsymbol{x}, \boldsymbol{y}) = |\boldsymbol{x} - \boldsymbol{y}| = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Because there are many other distance metrics besides this one, it makes sense to think about alternatives for a concrete application. In many applications, certain features are more important than others. Therefore it is often sensible to scale the features differently by weights w_i. The formula then reads

$$d_w(\boldsymbol{x}, \boldsymbol{y}) = |\boldsymbol{x} - \boldsymbol{y}| = \sqrt{\sum_{i=1}^{n} w_i (x_i - y_i)^2}.$$

The following simple nearest neighbor classification program searches the training data for the nearest neighbor $\boldsymbol{t}$ to the new example s and then classifies s exactly like t^3

NEARESTNEIGHBOR[$M_+, M_-, \boldsymbol{s}$]

$\boldsymbol{t} = \text{argmin}_{\boldsymbol{x} \in M_+ \cup M_-} \{d(\boldsymbol{s}, \boldsymbol{x})\}$
If $\boldsymbol{t} \in M_+$ **Then Return** $(+)$
Else Return $(-)$

In contrast to the perceptron, the nearest neighbor method does not generate a line that divides the training data points. However, an imaginary line separating the two classes certainly exists. We can generate this by first generating the so-called Voronoi diagram. In the Voronoi diagram, each data point is surrounded by a convex polygon, which thus defines a neighborhood around it. The Voronoi diagram has the property that for an arbitrary new point, the nearest neighbor among the data points is the data point, which lies in the same neighborhood. If the Voronoi diagram for a set of training data is determined, then it is simple to find the nearest neighbor for a new point which is to be classified. The class membership will then be taken from the nearest neighbor. In Fig. 8.14 we see clearly that the nearest neighbor method is significantly more powerful than the perceptron. It is capable of correctly realizing arbitrarily complex dividing lines (in general: hyperplanes). However, there is a danger here. A single erroneous point can in certain circumstances lead to very bad classification results. Such a case occurs in Fig. 8.15 during the classification of the black point. The nearest neighbor method may classify this wrong. If the black point is immediately next to a positive point that is an outlier of the positive class, then it will be classified positive rather than negative as would be intended here. An erroneous fitting to random errors (noise) is called overfitting.

```
K-NearestNeighbor $\left(M_{+}, M_{-}, \boldsymbol{s}\right)$
    $V=\left\{k\right.$ nearest neighbors in $\left.M_{+} \cup M_{-}\right\}$
    If $\quad\left|M_{+} \cap V\right|>\left|M_{-} \cap V\right|$ Then Return(,,+")
    ElseIf $\left|M_{+} \cap V\right|<\left|M_{-} \cap V\right|$ Then Return(,,-")
    Else Return(Random(,,+",,-"))
```

Fig. 8.16 The k-NearestNeighbor Algorithm

To prevent false classifications due to single outliers, it is recommended to smooth out the division surface somewhat. This can be accomplished by, for example, with the k-NearestNeighbor algorithm in Fig. 8.16, which makes a majority decision among the k nearest neighbors.

Example 8.4 We now apply NearestNeighbor to Example 8.3 on page 188. Because we are dealing with binary data, we use the Hamming distance as the distance metric. [4] As a test example, we again use modified training examples with n consecutive inverted bits each. In Fig. 8.17 the percentage of correctly classified test examples is shown as a function of the number of inverted bits b. For up to eight inverted bits, all patterns are correctly identified. Past that point, the number of errors quickly increases. This is unsurprising because training

[63] The functionals argmin and argmax determine, similarly to min and max, the minimum or maximum of a set or function. However, rather than returning the value of the maximum or minimum, they give the position, that is, the argument in which the extremum appears.

pattern number 2 from Fig. 8.11 on page 188 from class M_+ has a hamming distance of 9 to the two training examples, numbers 4 and 5 from the other class. This means that the test pattern is very likely close to the patterns of the other class. Quite clearly we see that nearest neighbor classification is superior to the perceptron in this application for up to eight false bits.

0.35.6 Two Classes, Many Classes, Approximation

Nearest neighbor classification can also be applied to more than two classes. Just like the case of two classes, the class of the feature vector to be classified is simply set as the class of the nearest neighbor. For the k nearest neighbor method, the class is to be determined as the class with the most members among the k nearest neighbors.

If the number of classes is large, then it usually no longer makes sense to use classification algorithms because the size of the necessary training data grows quickly with the number of classes. Furthermore, in certain circumstances important information is lost during classification of many classes. This will become clear in the following example.

Example 8.5 An autonomous robot with simple sensors similar to the Braitenberg vehicles presented in Fig. 1.1 on page 2 is supposed to learn to move away from light. This means it should learn as optimally as possible to map its sensor values onto a steering signal which controls the driving direction. The robot is equipped with two simple light sensors on its front side. From the two sensor signals (with s_l for the left and s_r for the right sensor), the relationship $x = s_r/s_l$ is calculated. To control the electric motors of the two wheels from this value x, the difference $v = U_r - U_l$ of the two voltages U_r and U_l of the left and right motors, respectively. The learning agent's task is now to avoid a light signal. It must therefore learn a mapping f which calculates the "correct" value $v = f(x)$.[5]

For this we carry out an experiment in which, for a few measured values x, we find as optimal a value v as we can. These values are plotted as data points in Fig. 8.18 and shall serve as training data for the learning agent. During nearest neighbor classification each point in the feature space (that is, on the x-axis) is classified exactly like its nearest neighbor among the training data. The function for steering the motors is then a step function with large jumps (Fig. 8.18 middle). If we want finer steps, then we must provide correspondingly more training data. On the other hand, we can obtain a continuous function if we approximate a smooth function to fit the five points (Fig. 8.18 on page 193 right). Requiring the function f to be continuous leads to very good results, even with no additional data points.

For the approximation of functions on data points there are many mathematical methods, such as polynomial interpolation, spline interpolation, or the

[64] The Hamming distance between two bit vectors is the number of different bits of the two vectors.

[65] To keep the example simple and readable, the feature vector $\boldsymbol{x}$ was deliberately kept one-dimensional.

method of least squares. The application of these methods becomes problematic in higher dimensions. The special difficulty in AI is that model-free approximation methods are needed. That is, a good approximation of the data must be made without knowledge about special properties of the data or the application. Very good results have been achieved here with neural networks and other nonlinear function approximators, which are presented in Chap. 9.

The k nearest neighbor method can be applied in a simple way to the approximation problem. In the algorithm k-NearestNeighbor, after the set $V = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_k\}$ is determined, the k nearest neighbors average function value

$$\hat{f}(\boldsymbol{x}) = \frac{1}{k} \sum_{i=1}^{k} f(\boldsymbol{x}_i) \tag{8.2}$$

is calculated and taken as an approximation $\hat{f}$ for the query vector $\boldsymbol{x}$. The larger k becomes, the smoother the function $\hat{f}$ is.

0.35.7 Distance Is Relevant

In practical application of discrete as well as continuous variants of the k nearest neighbor method, problems often occur. As k becomes large, there typically exist more neighbors with a large distance than those with a small distance. Thereby the calculation of $\hat{f}$ is dominated by neighbors that are far away. To prevent this, the k neighbors are weighted such that the more distant neighbors have lesser influence on the result. During the majority decision in the algorithm k-NearestNeighbor, the "votes" are weighted with the weight

$$w_i = \frac{1}{1 + \alpha d(\boldsymbol{x}, \boldsymbol{x_i})^2}, \tag{8.3}$$

which decreases with the square of the distance. The constant α determines the speed of decrease of the weights. Equation (8.2) is now replaced by

$$\hat{f}(\boldsymbol{x}) = \frac{\sum_{i=1}^{k} w_i f(\boldsymbol{x}_i)}{\sum_{i=1}^{k} w_i}$$

For uniformly distributed concentration of points in the feature space, this ensures that the influence of points asymptotically approaches zero as distance increases. Thereby it becomes possible to use many or even all training data to classify or approximate a given input vector.

To get a feeling for these methods, in Fig. 8.19 the k-nearest neighbor method (in the upper row) is compared with its distance weighted optimization. Due to the averaging, both methods can generalize, or in other words, cancel out noise, if the number of neighbors for k-nearest neighbor or the parameter α is set appropriately. The diagrams show nicely that the distance weighted method gives a much smoother approximation than k-nearest neighbor. With respect to approximation quality, this very simple method can compete well with sophisticated approximation algorithms such as nonlinear neural networks, support vector machines, and Gaussian processes. There are many alternatives to the weight function (also called kernel) given in (8.3) on page 194. For example a Gaussian or similar bell-shaped function can be used. For most applications, the results are not very sensible on the selection of the kernel. However, the width parameter α which for all these functions has to be set manually has great influence on the results, as shown in Fig. 8.19. To avoid such an inconvenient manual adaptation, optimization methods have been developed for automatically setting this parameter [[**?**], [**?**]].

0.35.8 Computation Times

As previously mentioned, training is accomplished in all variants of the nearest neighbor method by simply saving all training vectors together with their labels (class values), or the function value $f(\boldsymbol{x})$. Thus there is no other learning algorithm that learns as quickly. However, answering a query for classification or approximation of a vector $\boldsymbol{x}$ can become very expensive. Just finding the k nearest neighbors for n training data requires a cost which grows linearly with n. For classification or approximation, there is additionally a cost which is linear in k. The total computation time thus grows as $\Theta(n+k)$. For large amounts of training data, this can lead to problems.

0.35.9 Summary and Outlook

Because nothing happens in the learning phase of the presented nearest neighbor methods, such algorithms are also denoted lazy learning, in contrast to eager learning, in which the learning phase can be expensive, but application to new examples is very efficient. The perceptron and all other neural networks, decision tree learning, and the learning of Bayesian networks are eager learning methods. Since the lazy learning methods need access to the memory with all training data for approximating a new input, they are also called memory-based learning.

To compare these two classes of learning processes, we will use as an example the task of determining the current avalanche hazard from the amount of newly fallen snow in a certain area of Switzerland. [6] In Fig. 8.20 values determined by experts are entered, which we want to use as training data. During the application of a eager learning algorithm which undertakes a linear approximation of the data, the dashed line shown in the figure is calculated. Due to the restriction to a straight line, the error is relatively large with a maximum of about 1.5 hazard levels. During lazy learning, nothing is calculated before a query for the current hazard level arrives. Then the answer is calculated from several nearest neighbors, that is, locally. It could result in the curve shown in the figure, which is put together from line segments and shows much smaller errors. The advantage of the lazy method is its locality. The approximation is taken locally from the current new snow level and not globally. Thus for fundamentally equal classes of functions (for example linear functions), the lazy algorithms are better.